

TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA CÔNG NGHỆ THÔNG TIN

Nguyễn Chí Danh - Lý Trường Nam - Phạm
Tân Nghĩa - Hà Gia Phúc - Nguyễn Trường
Tân - Trần Nhật Tân

HỆ THỐNG HỖ TRỢ THÔNG TIN HỌC
TẬP CHO SINH VIÊN SỬ DỤNG AI
AGENTS

ĐỒ ÁN TỐT NGHIỆP CỬ NHÂN
CHƯƠNG TRÌNH CHUẨN

Tp. Hồ Chí Minh, tháng 04/2026

TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA CÔNG NGHỆ THÔNG TIN

Nguyễn Chí Danh - 22120048

Lý Trường Nam - 22120218

Phạm Tấn Nghĩa - 22120230

Hà Gia Phúc - 22120272

Nguyễn Trường Tân - 22120326

Trần Nhật Tân - 22120328

HỆ THỐNG HỖ TRỢ THÔNG TIN HỌC TẬP CHO SINH VIÊN SỬ DỤNG AI AGENTS

ĐỒ ÁN TỐT NGHIỆP CỬ NHÂN
CHƯƠNG TRÌNH CHUẨN

GIẢNG VIÊN HƯỚNG DẪN
TS. Nguyễn Trần Minh Thư

Tp. Hồ Chí Minh, tháng 04/2026

Lời cam đoan

Nhóm xin cam đoan Dự án “Hệ thống hỗ trợ thông tin học tập cho sinh viên sử dụng AI AGENTS” là công trình nghiên cứu độc lập của nhóm, với sự hướng dẫn tận tình của cô Nguyễn Trần Minh Thư.

Công trình được nhóm nghiên cứu và hoàn thành tại Trường Đại học Khoa học Tự nhiên từ tháng 01 năm 2026 đến tháng 6 năm 2026. Các tài liệu tham khảo, các số liệu thống kê phục vụ mục đích nghiên cứu công trình này được sử dụng đúng quy định, không vi phạm quy chế bảo mật của Nhà nước.

Những số liệu, bảng biểu phục vụ cho việc phân tích và dẫn dắt đề tài này được thu thập từ các nguồn tài liệu khác nhau được trích dẫn một cách đầy đủ, đồng thời ghi rõ ràng về nguồn gốc theo quy định của nhà trường.

Ngoài ra, đối với các tài liệu diễn giải để làm rõ thêm các luận điểm đã phân tích và trích dẫn trong phần phụ lục cũng được chú thích nguồn gốc dữ liệu. Nếu những lời cam đoan trên của nhóm không chính xác, nhóm xin chịu hoàn toàn trách nhiệm và chịu mọi hình thức kỷ luật từ khoa và nhà trường

Thuyết minh chỉnh sửa đề tài

Bản thuyết minh chỉnh sửa đề tài (nếu có thay đổi tên đề tài, nội dung báo cáo, ... trong cuốn báo cáo sau bảo vệ)

Nhận xét hướng dẫn

Bản nhận xét của giảng viên hướng dẫn (có chữ ký) do giáo vụ cung cấp.

Nhận xét phản biện

Bản nhận xét của giảng viên phản biện (có chữ ký) do giáo vụ cung cấp.

Lời cảm ơn

Dự án thực tập tốt nghiệp chuyên ngành Hệ Thống Thông Tin trên đánh dấu cột mốc hoàn thành.

Chúng em xin gửi lời cảm ơn chân thành đến Quý Ban Giám hiệu và toàn thể cán bộ giảng viên của Trường Đại học Khoa học Tự nhiên - Đại học Quốc gia TP.HCM, vì đã cung cấp cho chúng em môi trường học tập và nghiên cứu tuyệt vời trong suốt khoảng thời gian học tập và làm việc tại nhà trường.

Và chúng em xin chân thành cảm ơn sâu sắc dành cho cô Nguyễn Trần Minh Thư vì sự tận tâm, kiến thức sâu rộng và sự hỗ trợ quý báu trong suốt quá trình thực hiện đồ án tốt nghiệp. Cô đã dành thời gian và nỗ lực để chỉ dẫn, định hướng và đánh giá công việc của chúng em. Sự tận tâm và kiến thức chuyên môn của cô đã giúp chúng em vượt qua những khó khăn và phát triển kỹ năng nghiên cứu và thực hiện đồ án của mình.

Mặc dù chúng em đã cố gắng hoàn thành dự án thực tập tốt nghiệp với kết quả tốt nhất có thể trong khả năng, nhưng vẫn có những khiếm khuyết, sai sót nhất định do thiếu kinh nghiệm thực tế. Chúng em mong nhận được sự góp ý chân thành và dạy bảo của quý Thầy Cô để có thể hoàn thiện bản thân hơn nữa.

Lời cảm ơn cuối cùng dành cho gia đình, bạn bè và những người thân yêu đã luôn động viên và đồng hành cùng chúng em trong suốt quá trình nghiên cứu và thực hiện đồ án tốt nghiệp. Sự ủng hộ và động viên của họ là nguồn động lực vô cùng quan trọng giúp chúng em vượt qua khó khăn và hoàn thành công việc một cách tốt nhất.

Đề cương chi tiết

Bản đề cương đã thực hiện và nộp cho giáo vụ trước đó *(phải có chữ ký của giảng viên hướng dẫn)*.

Mục lục

Lời cam đoan	i
Thuyết minh chỉnh sửa đề tài	ii
Nhận xét của GV hướng dẫn	iii
Nhận xét của GV phản biện	iv
Lời cảm ơn	v
Đề cương	vi
Mục lục	vii
Danh sách viết tắt	xi
Danh sách hình	xiv
Danh sách bảng	xvi
Tóm tắt	xvi
1 Giới thiệu	1
2 Các công trình liên quan	3
2.1 Tổng quan về hệ thống hỗ trợ học tập thông minh	3
2.1.1 Các hướng tiếp cận truyền thống	3

2.1.2	Ứng dụng AI trong giáo dục hiện đại	4
2.2	Mô hình ngôn ngữ lớn (LLMs) trong giáo dục	5
2.2.1	Ứng dụng LLM trong hệ thống hỏi đáp học tập	5
2.2.2	Hạn chế của LLM (Hallucination, thiếu context, v.v.)	6
2.3	Retrieval-Augmented Generation (RAG)	7
2.3.1	Kiến trúc RAG truyền thống	7
2.3.2	Các thành phần chính: Retriever – Generator	8
2.3.3	Ưu điểm và hạn chế của Naive RAG	9
2.4	GraphRAG và Knowledge Graph	10
2.4.1	Tổng quan về Knowledge Graph	10
2.5	Các hệ thống RAG dựa trên Đồ thị tri thức	11
2.6	Lý do lựa chọn hệ thống LightRAG	12
2.7	Hiện trạng của LightRAG (Phiên bản 1.4.11)	12
2.8	Hệ thống Đa tác nhân (Multi-Agent Systems) và Luồng suy luận (Agentic Workflow)	14
2.8.1	Khái niệm AI Agent	15
2.8.2	Tích hợp LightRAG và Multi-Agent	15
2.9	Đánh giá hệ thống RAG và Multi-Agent	16
2.9.1	Các chỉ số đánh giá RAG (RAGAS)	16
2.9.2	Đánh giá hệ thống đa agent (Multi-Agent Evaluation)	17
2.9.3	Các phương pháp benchmark hiện nay	17
3	Phương pháp đề xuất	19
3.1	Quy trình xử lý dữ liệu	20
3.1.1	Tiền xử lý dữ liệu tri thức	21
3.1.2	Luồng logic xử lý và cơ chế kiểm tra trùng lặp	23
3.1.3	Chiến lược phân đoạn và chuẩn hóa văn bản	27
3.1.4	Hệ thống thu thập tri thức từ trang web	35
3.1.5	Quy trình điều phối và phân đoạn dữ liệu	37
3.1.6	Cơ chế trích xuất thực thể và xây dựng quan hệ	39

3.1.7	Luồng xử lý Prompt linh hoạt cho đa dạng nguồn dữ liệu	42
3.1.8	Kiểm chứng tri thức bằng chuẩn SHACL và cơ chế Staging	45
3.1.9	Xử lý thực thể trùng lặp trong văn bản học thuật .	49
3.1.10	Quản lý và duy trì bền vững hệ thống	54
3.2	Mô hình hóa trạng thái hệ thống (Agent State)	58
3.2.1	Kiến trúc các tác vụ (Agents)	58
3.2.2	Quy trình điều phối và Luồng dữ liệu	59
3.2.3	Cơ chế truy xuất tri thức (LightRAG Integration) .	60
3.2.4	Thuật toán điều phối (Pseudocode)	61
3.3	Thiết lập thực nghiệm	62
3.3.1	Thiết lập thực nghiệm phân hệ trích xuất (MinerU)	62
3.3.2	Thiết lập thực nghiệm Hệ thống Đồ thị tri thức và Đa tác nhân	63
4	Kết quả thí nghiệm và Ứng dụng	67
4.1	Đánh giá chất lượng MinerU trên tập tài liệu đầu vào . . .	67
4.2	Đánh giá chất lượng đồ thị tri thức	68
4.3	Đánh giá chất lượng truy xuất và sinh văn bản (RAG Metrics)	70
4.4	Đánh giá hiệu năng điều phối Đa tác nhân (Agentic Metrics)	70
4.5	Đánh giá hệ thống từ phía người dùng cuối	71
4.6	Thiết kế và xây dựng ứng dụng	71
4.6.1	Ý tưởng ứng dụng	71
4.6.2	Phân tích và đặc tả yêu cầu	72
4.6.3	Mô hình Use Case	74
4.6.4	Kiến trúc hệ thống	78
4.6.5	Thiết kế chi tiết các thành phần	81
4.6.6	Thiết kế giao diện	90
4.6.7	Hiện thực phân hệ Ứng dụng di động (Mobile Application)	97

5	Kết luận	105
5.1	Kết quả đạt được	105
5.2	Hạn chế của hệ thống	106
5.3	Hướng phát triển trong tương lai	106
	Danh mục công trình của tác giả	108
	Tài liệu tham khảo	109
A	Ngữ pháp tiếng Việt	112
B	Ngữ pháp tiếng Nôm	113

Danh sách viết tắt

STT	Viết tắt	Viết đầy đủ	Chú thích
1	AI	Artificial Intelligence	Trí tuệ nhân tạo, liên quan đến tự động hóa hành vi thông minh.
2	API	Application Programming Interface	Là cơ chế giao tiếp giữa backend và frontend bằng một tập hợp các định nghĩa và giao thức.
3	BLEU	Bilingual Evaluation Understudy	Độ đo đánh giá chất lượng văn bản sinh ra bằng máy.
4	CER	Character Error Rate	Tỷ lệ lỗi ký tự, thường dùng đánh giá hệ thống nhận dạng.
5	CSDL	Cơ sở dữ liệu	Tập hợp các dữ liệu được tổ chức có cấu trúc để lưu trữ.
6	HCMUS	Ho Chi Minh City University of Science	Trường Đại học Khoa học Tự nhiên, ĐHQG-HCM.

STT	Viết tắt	Viết đầy đủ	Chú thích
7	HTML	HyperText Markup Language	Ngôn ngữ đánh dấu siêu văn bản dùng tạo trang web.
8	JSON	JavaScript Object Notation	Định dạng hoán đổi dữ liệu nhẹ, dễ đọc.
9	KG	Knowledge Graph	Đồ thị tri thức, biểu diễn dữ liệu dạng thực thể và quan hệ.
10	KV	Key-Value	Mô hình lưu trữ dữ liệu theo từng cặp khóa - giá trị.
11	LLM	Large Language Model	Mô hình ngôn ngữ lớn, được huấn luyện trên dữ liệu khổng lồ.
12	MFR	Mathematical Formula Recognition	Nhận diện và trích xuất công thức toán học từ tài liệu.
13	OCR	Optical Character Recognition	Nhận dạng ký tự quang học từ hình ảnh sang văn bản.
14	PDF	Portable Document Format	Định dạng tài liệu di động phổ biến độc lập nền tảng.
15	RAG	Retrieval-Augmented Generation	Sinh văn bản tăng cường truy xuất kết hợp LLM và tìm kiếm.

STT	Viết tắt	Viết đầy đủ	Chú thích
16	RAGAS	Retrieval-Augmented Generation Assessment	Khung đánh giá tự động chuyên biệt cho hệ thống RAG.
17	SHACL	Shapes Constraint Language	Ngôn ngữ xác thực các ràng buộc cấu trúc cho đồ thị RDF.
18	TEDS	Tree-Edit-Distance-based Similarity	Độ đo tương đồng cấu trúc bảng dựa trên khoảng cách cây.
19	UI/UX	User Interface / User Experience	Giao diện người dùng và Trải nghiệm người dùng.
20	VLM	Vision-Language Model	Mô hình đa phương thức xử lý hình ảnh và ngôn ngữ.
21	WER	Word Error Rate	Tỷ lệ lỗi từ, dùng đánh giá độ chính xác của văn bản.

Danh sách hình

3.1	Quy trình tổng quát từ văn bản thô đến đồ thị tri thức . .	20
3.2	Quy trình xử lý Hybrid-Auto-Engine trong MinerU mineru_2024	22
3.3	Luồng logic kiểm tra trùng lặp bằng cơ chế băm nội dung .	24
3.4	Quy trình thu thập tri thức từ trang web với Smart Scraper	25
3.5	Quy trình thu thập tri thức từ trang web với Smart Scraper	35
3.6	Luồng logic xử lý từ dữ liệu thô đến giai đoạn phân đoạn .	38
3.7	Luồng logic quy trình trích xuất tri thức và cơ chế Gleaning	41
3.8	Luồng điều phối lời nhắc linh hoạt theo miền tri thức (Dy- namic Prompt Flow)	45
3.9	Quy trình kiểm chứng tri thức ba lớp bảo vệ (3-Guard) dựa trên SHACL và đối chiếu bằng chứng	48
3.10	Kiến trúc hai giai đoạn của module phân giải thực thể. Giai đoạn Candidate Retrieval sử dụng embedding similarity để truy hồi các thực thể ngữ nghĩa tương đồng từ vector store. Giai đoạn LLM Decision chỉ được kích hoạt khi có ứng viên vượt ngưỡng τ , giúp tiết kiệm chi phí gọi LLM cho những thực thể hoàn toàn mới.	50

3.11	Ví dụ cấu trúc knowledge graph sau khi áp dụng phân giải thực thể trên corpus học thuật HCMUS. Các nút màu đậm là thực thể dạng chuẩn (<i>canonical</i>); các nút màu nhạt là hình thức tên thay thế (<i>alias</i>) được liên kết về nút chuẩn qua quan hệ SAME_AS (đường nét đứt cam). Quan hệ ngữ nghĩa thông thường (mũi tên xanh) chỉ xuất phát từ nút chuẩn, đảm bảo tri thức không bị phân tán.	52
3.12	Quy trình truy vấn đa chế độ và cơ chế tối ưu phản hồi trong LightRAG	56
4.1	Sơ đồ Use Case hệ thống	74
4.2	Mô hình kiến trúc hệ thống tổng thể	79
4.3	Mô hình sơ đồ cơ sở dữ liệu quan hệ người dùng	82
4.4	Mô hình khái niệm các loại thực thể và quan hệ chính trong đồ thị tri thức học vụ	85
4.5	Ví dụ subgraph trích xuất từ đề cương môn học trong Knowledge Graph	87
4.6	Tổng quan kiến trúc lưu trữ và triển khai LightRAG	88
4.7	Giao diện Màn hình Đăng ký	90
4.8	Giao diện Màn hình Đăng nhập	91
4.9	Giao diện Màn hình Quên mật khẩu - Yêu cầu mã xác thực	93
4.10	Giao diện Màn hình Quên mật khẩu - Xác nhận từ Email	93
4.11	Giao diện Màn hình Thông tin cá nhân	95
4.12	Giao diện Màn hình AI Chat	96
4.13	Giao diện tương tác phân hệ Trợ lý ảo AI trên ứng dụng di động	99
4.14	Giao diện quản lý Tiến trình tích lũy và Gợi ý học tập cá nhân hóa	102
4.15	Giao diện Hồ sơ sinh viên và Quản lý thông tin học vụ cá nhân	104

Danh sách bảng

3.1	So sánh hai chiến lược phân đoạn trong hệ thống	31
3.2	So sánh các thành phần thiết kế với công trình liên quan .	53
4.1	Kết quả chi tiết đánh giá MinerU trên các tập tài liệu Chương trình đào tạo (2022-2025)	68
4.2	Tổng hợp kết quả đánh giá đồ thị tri thức trên các cấp độ lấy mẫu khác nhau	69
4.3	Danh sách yêu cầu chức năng	73
4.4	Yêu cầu phi chức năng	73
4.5	Danh sách Actor	74
4.6	Danh sách Use Case	75
4.10	Danh sách các bảng trong cơ sở dữ liệu quan hệ	82
4.11	Mô tả khóa ngoại của các bảng trong CSDL quan hệ	83
4.12	Danh sách các Collection trong cơ sở dữ liệu hcmus_langgraph	84
4.13	Thuộc tính của node thực thể trong Knowledge Graph (Neo4j)	86
4.14	Thuộc tính của relationship trong Knowledge Graph (Neo4j)	87
4.15	Các thành phần lưu trữ trong kiến trúc LightRAG	89

Tóm tắt

Nhiều sinh viên của trường Đại học Khoa học Tự nhiên – ĐHQG TPHCM đang học tập và làm việc trong tình trạng không có định hướng cho bản thân. Hệ thống hỗ trợ cung cấp thông tin học tập ra đời sẽ giúp sinh viên có góc nhìn tổng quan hơn trong con đường học vấn. Hệ thống sẽ đưa ra được các thông tin học tập theo chuyên ngành hiện tại, mong muốn sau này của sinh viên, gợi ý lộ trình học tập rõ ràng, chi tiết để đạt được hiệu quả học tập tốt nhất trong quá trình học. Nghiên cứu này áp dụng Knowledge Graph (KG) để lưu các thông tin liên quan tới học tập, kết hợp AI AGENTS” hỗ trợ trong mapping các tri thức thành đồ thị và các AI Agents chuyên biệt cho quá trình tìm kiếm thông tin học tập, phân tích, đánh giá và đưa ra kết quả tối ưu nhất cho người dùng. Phương pháp, đánh giá và kết quả (Viết thêm sau khi hoàn thành)

Chương 1

Giới thiệu

Trong bối cảnh hiện nay sinh viên ngành Công nghệ thông tin tại Trường Đại học Khoa học Tự nhiên đang phải đối mặt với một lượng kiến thức cực kỳ lớn và rời rạc. Tuy tài liệu có sẵn rất nhiều trên mạng nhưng chúng thường tồn tại ở dạng các mẫu tin riêng biệt khiến người học khó có thể tự mình kết nối chúng lại thành một thông tin tổng hợp học tập hoàn chỉnh. Các công cụ tìm kiếm thông thường hiện nay chỉ giúp tìm ra thông tin của một môn học cụ thể chứ không thể chỉ ra mối quan hệ giữa các môn hay giải thích tại sao kiến thức này lại cần thiết cho kiến thức kia. Thêm vào đó các mô hình trí tuệ nhân tạo phổ biến đôi khi vẫn cung cấp thông tin không chính xác hoặc không hiểu rõ các quy định và chương trình học đặc thù của nhà trường.

Vì những lý do đó đề tài này được ra đời nhằm xây dựng một trợ lý học tập thông minh có khả năng hiểu sâu về lộ trình và tài liệu của sinh viên. Mục đích cuối cùng là tạo ra một trợ lý ảo có thể trả lời chính xác các thắc mắc về học vụ đồng thời hiểu được sự liên kết logic giữa các khái niệm khác nhau. Điểm thú vị của bài toán này nằm ở việc biến những tài liệu thô cứng thành một mạng lưới tri thức giúp sinh viên không còn cảm thấy lạc lõng giữa biển thông tin. Chúng ta cần giải quyết vấn đề này vì việc định hướng đúng đắn ngay từ đầu sẽ giúp sinh viên tiết kiệm rất nhiều thời gian và công sức trong suốt quá trình theo học tại trường.

Tuy nhiên việc xây dựng một hệ thống như vậy không hề đơn giản vì

nó đòi hỏi trí tuệ nhân tạo phải có khả năng tư duy và kết nối thông tin như con người. Đề tài lựa chọn hướng giải quyết thông qua việc kết hợp giữa một nhóm AI Agents phối hợp với nhau cùng một phương pháp tổ chức dữ liệu theo dạng đồ thị tri thức. Những hướng đi trước đây thường chỉ tập trung vào việc tìm kiếm từ khóa nên thường bỏ sót các mối quan hệ ngầm giữa các môn học. Bằng cách xây dựng một bản đồ kiến thức chặt chẽ hệ thống mới sẽ khắc phục được tình trạng trả lời thiếu chiều sâu hoặc thông tin không liên quan.

Nghiên cứu này tập trung chủ yếu vào sinh viên Công nghệ thông tin tại HCMUS với dữ liệu là các môn học cơ sở, môn chuyên ngành và chương trình đào tạo. Thông qua việc trả lời các câu hỏi về cách học và lộ trình phát triển đề tài mang lại một công cụ hỗ trợ thiết thực giúp cá nhân hóa trải nghiệm học tập cho từng người. Đóng góp lớn nhất của nghiên cứu là tạo ra một nền tảng tri thức tin cậy nơi mọi thông tin học tập được tổng hợp và kết nối một cách logic nhất phục vụ hiệu quả cho việc nghiên cứu và rèn luyện của sinh viên trong kỷ nguyên số.

Chương 2

Các công trình liên quan

2.1 Tổng quan về hệ thống hỗ trợ học tập thông minh

Trong môi trường giáo dục đại học, khối lượng kiến thức và cấu trúc chương trình đào tạo ngày càng trở nên phức tạp. Việc xây dựng một lộ trình học tập tối ưu không chỉ dừng lại ở việc hoàn thành các tín chỉ, mà còn phải đáp ứng đúng định hướng nghề nghiệp và năng lực cốt lõi của từng cá nhân. Các hệ thống hỗ trợ học tập thông minh (Intelligent Tutoring/Learning Support Systems) ra đời nhằm giải quyết bài toán này, đóng vai trò như một cầu nối thông tin giữa nhà trường, chương trình đào tạo và sinh viên [28].

2.1.1 Các hướng tiếp cận truyền thống

Trước khi có sự bùng nổ của Trí tuệ Nhân tạo và các Mô hình Ngôn ngữ Lớn (LLMs), các hệ thống hỗ trợ học vụ tại các trường đại học chủ yếu hoạt động dựa trên các quy tắc tĩnh (rule-based) và kiến trúc dữ liệu phẳng (flat data representations). Một số đặc điểm và hạn chế của các hệ thống này có thể được phân tích qua ba khía cạnh chính. Một trong những điểm yếu rõ rệt nhất nằm ở các hệ thống cổng thông tin tĩnh (Static Portals).

Tại đây, thông tin về chương trình đào tạo, đề cương chi tiết (syllabus), môn học tiên quyết và song hành thường được cung cấp dưới dạng các tệp văn bản hoặc PDF rời rạc. Do sinh viên phải tự tra cứu thủ công, điều này dẫn đến tình trạng phân mảnh thông tin và khiến họ khó hình dung được bức tranh toàn cảnh của mạng lưới môn học. Bên cạnh đó, các chatbot dựa trên quy tắc (Rule-based Chatbots) thế hệ cũ thường chỉ hoạt động dựa trên bộ từ khóa (keywords) và các kịch bản hội thoại được lập trình sẵn. Do đó, hệ thống này gặp hạn chế lớn trong việc xử lý ngôn ngữ tự nhiên, hoàn toàn không hiểu được ngữ cảnh tổng thể hay các truy vấn phức tạp của sinh viên. Ngoài ra, các phương pháp truyền thống còn thiếu khả năng mô hình hóa các mối quan hệ đa chiều, dẫn đến hạn chế trong việc liên kết dữ liệu. Việc kết nối giữa điểm số, sở thích cá nhân và môn học hầu như không được tự động hóa. Hậu quả là làm mất đi tính cá nhân hóa, khiến sinh viên dễ bị mất phương hướng khi tự lập kế hoạch học tập.

2.1.2 Ứng dụng AI trong giáo dục hiện đại

Sự phát triển của Trí tuệ Nhân tạo (AI), đặc biệt là Xử lý Ngôn ngữ Tự nhiên (NLP) và Học máy (Machine Learning), đã tạo ra những bước nhảy vọt trong việc phát triển các hệ thống giáo dục thông minh. Thay vì chỉ cung cấp thông tin thụ động, các hệ thống hiện đại đã chuyển dịch sang hướng tương tác và cá nhân hóa sâu sắc:

Tiêu biểu là các hệ thống gợi ý và cá nhân hóa (Recommender Systems in Education). Tại đây, AI cho phép phân tích dữ liệu lịch sử học tập (điểm số, tiến độ) kết hợp với dữ liệu phi cấu trúc (nguyện vọng, mục tiêu nghề nghiệp) nhằm đưa ra các gợi ý môn học phù hợp nhất. Nhờ đó, các thuật toán này giúp sinh viên vạch ra lộ trình học tập linh hoạt theo từng học kỳ, bám sát với định hướng chuyên môn. Song song đó là sự chuyển đổi mạnh mẽ từ Chatbot sang AI Agents (Tác nhân AI). Trái ngược với chatbot thông thường chỉ phản hồi truy vấn đơn lẻ, AI Agents (đặc biệt

là kiến trúc Multi-Agent) được trang bị khả năng tự động hóa chuỗi suy luận, giúp giải quyết các yêu cầu phức tạp hơn. Hơn nữa, việc ứng dụng Đồ thị hóa tri thức giáo dục (Educational Knowledge Graphs) cũng đang trở thành xu hướng. Nhằm khắc phục tình trạng AI tạo ra thông tin sai lệch (“ảo giác” - hallucination) và thiếu ngữ cảnh, các nghiên cứu hiện đại đang tích cực kết hợp AI với Đồ thị tri thức [11]. Vì việc số hóa và biểu diễn cấu trúc chương trình đào tạo dưới dạng các thực thể và mối quan hệ sẽ giúp hệ thống có khả năng truy xuất thông tin đa chiều, AI từ đó có thể giải thích rõ ràng lý do tại sao một môn học lại cần thiết.

2.2 Mô hình ngôn ngữ lớn (LLMs) trong giáo dục

2.2.1 Ứng dụng LLM trong hệ thống hỏi đáp học tập

Với khả năng hiểu và sinh ngôn ngữ tự nhiên, LLMs đã nhanh chóng được tích hợp vào các hệ thống giáo dục, đặc biệt là trong các hệ thống hỏi đáp (Q&A):

Một trong những ứng dụng nổi bật là khả năng giao tiếp bằng ngôn ngữ tự nhiên (Natural Language Interface). Thay vì phải sử dụng các từ khóa cứng nhắc hay câu lệnh phức tạp, sinh viên hoàn toàn có thể đặt câu hỏi theo cách giao tiếp thông thường (ví dụ: “Nếu em trượt môn Cơ sở dữ liệu thì kỳ sau em có được học Hệ quản trị cơ sở dữ liệu không?”). Nhờ vào LLMs, hệ thống có thể hiểu chính xác ý định và đưa ra câu trả lời trực tiếp. Đồng thời, LLMs còn đóng vai trò đắc lực trong việc hỗ trợ giải thích và tóm tắt kiến thức. Với khả năng phân tích các đề cương môn học phức tạp, mô hình có thể tóm tắt lại những kỹ năng cốt lõi mà sinh viên sẽ đạt được. Hơn thế nữa, nó có thể đóng vai trò như một người hướng dẫn, giải thích các khái niệm học thuật bằng nhiều cách diễn đạt khác nhau. Ngoài ra, tính năng hỗ trợ sinh tự động (Generative Assistance) cũng là

một điểm cộng lớn. LLMs có thể hỗ trợ sinh viên trong việc phác thảo kế hoạch học tập tạm thời, gợi ý cách viết email trao đổi học thuật với giảng viên, hoặc thậm chí sinh ra các câu hỏi trắc nghiệm để sinh viên tự ôn tập dựa trên tài liệu được cung cấp.

2.2.2 Hạn chế của LLM (Hallucination, thiếu context, v.v.)

Mặc dù sở hữu những khả năng vượt trội, việc triển khai trực tiếp các LLMs thuần túy (vanilla LLMs) vào hệ thống tư vấn học vụ gặp phải những rủi ro và rào cản, đặc biệt là trong môi trường đại học yêu cầu tính chính xác tuyệt đối:

Nguy cơ nghiêm trọng nhất là hiện tượng ảo giác AI (Hallucination). Khi không có đủ thông tin hoặc câu hỏi vượt quá dữ liệu huấn luyện, LLM có xu hướng bịa ra các câu trả lời nghe có vẻ rất hợp lý nhưng thực chất lại sai sự thật [12]. Nếu hệ thống AI tự sáng tác ra một môn học tiên quyết không tồn tại hoặc cung cấp sai quy chế học vụ, điều này sẽ dẫn đến hậu quả rất nghiêm trọng, làm sai lệch toàn bộ lộ trình học tập của sinh viên. Một hạn chế khác đến từ việc thiếu ngữ cảnh chuyên biệt và tính cập nhật (Lack of Domain-specific Context & Up-to-date Knowledge). Do LLMs được huấn luyện trên dữ liệu đại trà tính đến một thời điểm cắt (cut-off date) nhất định, chúng không thể tự động nắm bắt những thay đổi mới nhất trong chương trình đào tạo, quy định chuẩn đầu ra, hay danh sách giảng viên phụ trách của nhà trường. Thêm vào đó, LLMs thuần túy bộc lộ sự yếu kém trong suy luận đa bước phức tạp (Complex Multi-hop Reasoning). Để vạch ra một lộ trình học tập, hệ thống cần phải duyệt qua một mạng lưới chằng chịt các điều kiện ràng buộc (ví dụ: môn A là tiên quyết của môn B, môn B lại học song hành với môn C). Vì thế, nếu thông tin chỉ được biểu diễn dưới dạng văn bản phẳng, mô hình sẽ gặp rất nhiều rào cản trong việc xâu chuỗi các mối quan hệ đa tầng này. Một rào cản nữa là sự khó khăn trong việc xác minh nguồn gốc (Lack of Traceability).

Các câu trả lời được LLMs sinh ra tự động thường không đi kèm với nguồn trích dẫn rõ ràng, khiến người dùng không biết thông tin được lấy từ văn bản quy chế nào. Hệ quả là điều này làm giảm đi đáng kể độ tin cậy của hệ thống trong mắt sinh viên và giảng viên.

2.3 Retrieval-Augmented Generation (RAG)

Để giải quyết các hạn chế chí mạng của Mô hình ngôn ngữ lớn (LLMs) như hiện tượng “ảo giác” (hallucination) và thiếu thông tin cập nhật (đã phân tích ở mục 2.2), kiến trúc sinh văn bản tăng cường truy xuất (Retrieval-Augmented Generation - RAG) [17] đã được đề xuất và nhanh chóng trở thành tiêu chuẩn trong việc xây dựng các hệ thống AI chuyên biệt. Trong bối cảnh đề tài, RAG đóng vai trò là cơ chế cốt lõi giúp AI Agent truy cập vào cơ sở dữ liệu học vụ thực tế của Trường Đại học Khoa học Tự nhiên trước khi đưa ra câu trả lời cho sinh viên.

2.3.1 Kiến trúc RAG truyền thống

Kiến trúc RAG truyền thống (còn gọi là Naive RAG) là một phương pháp kết hợp giữa khả năng truy xuất thông tin và khả năng sinh ngôn ngữ (Text Generation). Thay vì ép buộc LLM phải tự nhớ mọi kiến thức trong các trọng số tham số (parametric memory) của nó, RAG cung cấp cho LLM một “bộ nhớ ngoài” (non-parametric memory) chứa dữ liệu thực tế.

Quy trình hoạt động cơ bản của Naive RAG diễn ra qua các bước cụ thể. Giai đoạn khởi đầu của quy trình này là lập chỉ mục (Indexing). Tại đây, dữ liệu thô (như file PDF đề cương môn học, quy chế học vụ) sẽ được chia nhỏ thành các đoạn văn bản ngắn (chunks). Ngay sau đó, những đoạn văn bản này được chuyển đổi thành các vector số học (embeddings) và lưu trữ trực tiếp vào Cơ sở dữ liệu Vector (Vector Database). Một khi cơ sở

dữ liệu đã sẵn sàng, hệ thống sẽ bước vào quá trình truy xuất (Retrieval) mỗi khi nhận được yêu cầu. Cụ thể, khi sinh viên đặt câu hỏi, hệ thống sẽ tiến hành chuyển đổi câu hỏi đó thành vector, từ đó tìm kiếm các đoạn văn bản (chunks) mang độ tương đồng ngữ nghĩa cao nhất trong Vector Database. Bước trọng tâm cuối cùng là sinh văn bản (Generation). Các đoạn văn bản vừa được tìm thấy sẽ được hệ thống ghép nối vào câu lệnh (prompt) đóng vai trò như một “ngữ cảnh” (context). Nhờ có ngữ cảnh sát sườn này, LLM sẽ đọc và tổng hợp để sinh ra câu trả lời cuối cùng dựa trên các thông tin thực tế vừa được cung cấp.

2.3.2 Các thành phần chính: Retriever – Generator

Hệ thống RAG truyền thống được cấu thành từ hai module độc lập nhưng phối hợp chặt chẽ với nhau. Thành phần then chốt thứ nhất là Retriever (Bộ truy xuất), chịu trách nhiệm tìm kiếm và trích xuất thông tin. Do chất lượng của Retriever phụ thuộc hoàn toàn vào các mô hình nhúng (Embedding Models), nên các đoạn văn bản sẽ được biểu diễn ý nghĩa dưới dạng các điểm trong không gian vector đa chiều. Bằng cách áp dụng các thuật toán như Cosine Similarity (độ tương đồng Cosine) để đo lường khoảng cách, Retriever có thể tìm ra chính xác đoạn văn bản mô tả về môn học hoặc quy định học vụ mà sinh viên đang thắc mắc. Đóng vai trò hỗ trợ và hoàn thiện quy trình là Generator (Bộ sinh văn bản), thường là một Mô hình ngôn ngữ lớn (LLM) đã được tinh chỉnh cho tác vụ làm theo chỉ dẫn (Instruction-tuned LLM). Khác với các mô hình thông thường, nhiệm vụ của Generator không phải là tự nghĩ ra câu trả lời từ con số không. Thay vào đó, nó sẽ đọc hiểu các đoạn dữ liệu do Retriever mang về, sau đó tiến hành tổng hợp, tóm tắt và diễn đạt lại nhằm tạo ra một câu trả lời mạch lạc, tự nhiên và dễ hiểu nhất cho sinh viên.

2.3.3 Ưu điểm và hạn chế của Naive RAG

Ưu điểm: Ưu điểm sáng giá nhất của phương pháp này là khả năng giảm thiểu ảo giác (Hallucination). Bởi vì hệ thống buộc LLM phải dựa dẫm hoàn toàn vào dữ liệu học vụ nội bộ của Khoa, RAG gần như loại bỏ được rủi ro AI tự bịa ra thông tin sai lệch. Song song đó, hệ thống còn mang lại tính hiệu quả về mặt chi phí và vô cùng dễ dàng trong việc bảo trì. Thay vì phải tốn kém huấn luyện lại (fine-tuning) LLM mỗi khi chương trình đào tạo có sự điều chỉnh, nhà trường giờ đây chỉ việc đưa các file tài liệu mới vào Vector Database. Không chỉ vậy, tính minh bạch (Traceability) cũng được nâng cao đáng kể. Vì RAG cung cấp khả năng trích dẫn trực tiếp nguồn gốc của thông tin (ví dụ: “Theo mục 2 của Quy chế Đào tạo tín chỉ...”), hệ thống sẽ tạo dựng được mức độ tin cậy vững chắc hơn đối với sinh viên.

Hạn chế chí mạng đối với dữ liệu giáo dục đại học: Dù Naive RAG hoạt động rất tốt với các tác vụ hỏi đáp thông thường (Q&A), nhưng có nhiều điểm yếu khi xử lý với dữ liệu có tính liên kết chặt chẽ như chương trình đào tạo đại học. Khuyết điểm dễ thấy nhất xuất phát từ cách biểu diễn dữ liệu phẳng (Flat Data Representation). Do Naive RAG chia cắt tài liệu thành các khối (chunks) rời rạc, hệ thống đã vô tình cắt đứt các mối liên hệ logic phức tạp vốn có. Ví dụ, nếu môn A là tiên quyết của môn B, và môn B kết hợp với môn C, thì Vector Database chỉ lưu các đoạn văn bản rời rạc chứ không hề “hiểu” được bản đồ mạng lưới liên kết của các môn học này. Từ đặc điểm biểu diễn phẳng đó, hệ thống dẫn đến sự thất bại trong suy luận đa bước (Multi-hop Reasoning Failure). Cụ thể, Naive RAG sẽ tỏ ra bối rối và xử lý kém hiệu quả khi đối mặt với những câu hỏi đòi hỏi phải duyệt qua và tổng hợp dữ liệu từ hàng chục đề cương môn học khác nhau. Nghiêm trọng hơn là rủi ro bỏ sót thông tin. Trong trường hợp từ khóa truy vấn của sinh viên không khớp hoàn toàn về mặt ngữ nghĩa cục bộ với các chunk trong database, Retriever sẽ thất bại trong việc mang về ngữ cảnh chính xác. Hậu quả là Generator sẽ hoàn toàn không có đủ dữ

liệu nền tảng để đưa ra câu trả lời.

2.4 GraphRAG và Knowledge Graph

Để khắc phục những yêu điểm của hệ thống RAG truyền thống (Vector RAG) trong việc xử lý các truy vấn phức tạp và thiếu nhận thức về ngữ cảnh tổng thể, hướng tiếp cận kết hợp Đồ thị tri thức (Knowledge Graph) vào quá trình truy xuất đã ra đời, được gọi chung là GraphRAG. Trong bối cảnh giáo dục đại học, nơi thông tin có tính liên kết cao.

2.4.1 Tổng quan về Knowledge Graph

Đồ thị tri thức (Knowledge Graph - KG) là một cấu trúc dữ liệu mô hình hóa thông tin dưới dạng một mạng lưới [11]. Thay vì lưu trữ dữ liệu dưới dạng các bảng độc lập (như cơ sở dữ liệu quan hệ) hoặc các văn bản phẳng rời rạc, KG lưu trữ thông tin dưới dạng các Thực thể (Entities - Node) và các Mối quan hệ (Relationships - Edge) giữa chúng.

Trong hệ thống hỗ trợ học tập của Khoa Công nghệ Thông tin, Đồ thị tri thức có thể được thiết kế như sau. Thành phần cơ bản cấu thành nên đồ thị là các Thực thể (Nodes), đại diện cho các đối tượng cốt lõi như Sinh viên, Môn học (vd: Nhập môn Lập trình, Cơ Sở Trí tuệ Nhân tạo), và Kỹ năng (vd: Lập trình Python, Tư duy logic). Để gắn kết các thực thể rời rạc này lại với nhau, hệ thống sử dụng các Mối quan hệ (Edges/Relations). Các mối quan hệ này định nghĩa cách thức tương tác giữa các node, ví dụ như: Là tiên quyết của (IS PREREQUISITE OF), Cần thiết cho (REQUIRED FOR), hay Thuộc khối kiến thức (BELONGS TO).

2.5 Các hệ thống RAG dựa trên Đồ thị tri thức

Sự phát triển của RAG kết hợp với Đồ thị tri thức đã dẫn đến sự ra đời của nhiều hệ thống khác nhau nhằm giải quyết các bài toán về truy xuất và lập luận phức tạp. Các hệ thống này khác biệt chủ yếu ở cách mô hình hóa đồ thị, cơ chế truy vấn và khả năng cập nhật dữ liệu. Điển hình là hệ thống GraphRAG do Microsoft phát triển vào năm 2024 [5]. Kiến trúc này sử dụng đồ thị phân cấp dựa trên các cụm cộng đồng và thực hiện tóm tắt ở cấp độ cộng đồng để hỗ trợ tìm kiếm cục bộ lẫn toàn cục. Mặc dù thể hiện khả năng hiểu các mối quan hệ sâu và tóm tắt toàn cục rất tốt, GraphRAG đòi hỏi chi phí xây dựng chỉ mục cao và bị hạn chế trong việc cập nhật dữ liệu liên tục do phải tái cấu trúc lại các cụm mỗi khi có dữ liệu mới.

Một giải pháp khác là HippoRAG2 được giới thiệu tại NeurIPS 2025 [8], sử dụng kiến trúc đồ thị phức hợp bao gồm các node thực thể, node đoạn văn bản và cạnh đồng nghĩa. Hệ thống ứng dụng thuật toán PageRank cá nhân hóa để trích xuất văn bản và hỗ trợ lập luận đa bước. Điểm mạnh của HippoRAG2 là khả năng học liên tục không tham số và cập nhật tăng dần trực tuyến xuất sắc mà không cần lập chỉ mục lại toàn bộ, dù khả năng truy vấn toàn cục chỉ ở mức trung bình. Tiếp đến là PathRAG [24], một nghiên cứu tiếp cận bằng cách sử dụng đồ thị đường đi quan hệ kết hợp với kỹ thuật tỉa nhánh dựa trên luồng. Phương pháp này giảm bớt nhiễu cảnh không cần thiết trong khi vẫn duy trì độ chính xác cao cho các truy vấn đa bước, tuy nhiên lại gặp khó khăn và tốn kém khi đồ thị thay đổi vì cần phải thực hiện lại quá trình tỉa nhánh.

Các hệ thống chuyên biệt hơn bao gồm OG-RAG [21] và Hyper-GraphRAG [16]. OG-RAG hoạt động dựa trên đồ thị có ràng buộc lược đồ, giúp giảm đáng kể hiện tượng ảo giác và phù hợp cho các miền dữ liệu đặc thù, nhưng lại yêu cầu người dùng phải cập nhật lược đồ thủ công.

Ngược lại, HyperGraphRAG ứng dụng siêu đồ thị để nắm bắt các mối quan hệ đa thực thể phức tạp vượt khỏi giới hạn quan hệ nhị phân. Hệ thống này rất thích hợp cho các nguồn dữ liệu khoa học, nhưng hiện đòi hỏi chi phí tính toán cao và chưa hoàn toàn tối ưu hóa cho môi trường vận hành thực tế ¹.

2.6 Lý do lựa chọn hệ thống LightRAG

Qua quá trình phân tích chuyên sâu các kiến trúc hiện đại, LightRAG (được giới thiệu năm 2024 [7]) được lựa chọn làm giải pháp cốt lõi cho hệ thống tư vấn học tập. Quyết định này xuất phát từ việc LightRAG mang lại sự cân bằng hoàn hảo giữa hiệu năng truy xuất, khả năng thích ứng với dữ liệu động và tối ưu hóa chi phí vận hành, khắc phục được nhiều khuyết điểm chí mạng của các hệ thống tiền nhiệm. Những lợi ích to lớn về khả năng lập luận đa bước chính xác, hiệu suất truy vấn toàn cục nhanh chóng và chi phí tạo lập chỉ mục siêu thấp của LightRAG là yếu tố then chốt. Việc phân chia cấp độ truy vấn còn giúp đảm bảo hệ thống không bị ảo giác khi đối mặt với các câu hỏi phức hợp, đồng thời không gây lãng phí tài nguyên cho các câu hỏi đơn điệu. Tổng hợp tất cả các khía cạnh trên, LightRAG khẳng định được vị thế là lựa chọn khả thi, tiết kiệm và tối ưu nhất để xây dựng một môi trường tư vấn học thuật cá nhân hóa và đáng tin cậy.

2.7 Hiện trạng của LightRAG (Phiên bản 1.4.11)

Nhằm hiểu rõ hơn về tiềm năng và khả năng ứng dụng thực tế của LightRAG trong ngữ cảnh hiện tại, việc đánh giá chi tiết cả những ưu

¹Tham khảo dữ liệu so sánh từ báo cáo đánh giá các hệ thống RAG sử dụng Knowledge Graph năm 2025.

điểm nổi trội lẫn các hạn chế còn tồn đọng là vô cùng cần thiết. Hiện tại, nền tảng đang được xem xét dựa trên phiên bản phát hành 1.4.11.

Xét về ưu điểm, khác với cấu trúc phân cụm phức tạp gây tốn kém tài nguyên của GraphRAG hay mô hình lược đồ cứng nhắc của OG-RAG, LightRAG sử dụng cấu trúc đồ thị thực thể và mối quan hệ dạng phẳng kết hợp với cơ chế truy xuất đa cấp độ. Hệ thống thực hiện song song việc khớp từ khóa, tìm kiếm vector và truy vấn đồ thị cục bộ lẫn toàn cục. Nhờ chiến lược truy xuất theo hai luồng riêng biệt này, LightRAG không chỉ bao quát được toàn bộ ngữ cảnh từ diện rộng đến chi tiết mà còn đạt tốc độ phản hồi cực kỳ nhanh với độ trễ tối ưu chỉ khoảng tám mươi mili-giây [7]. Một lợi thế cạnh tranh mang tính quyết định khác của hệ thống nằm ở khả năng cập nhật tri thức linh hoạt. Trong môi trường học thuật, nơi quy chế đào tạo thay đổi thường xuyên, LightRAG hỗ trợ cập nhật tăng dần một cách mượt mà bằng cách trích xuất tự động và nối thêm các thực thể vào mạng lưới đồ thị hiện hữu mà không làm phá vỡ tổng thể. Sự linh hoạt này đi kèm với khả năng tương thích cao trên nhiều nền tảng lưu trữ như Neo4j hay PostgreSQL.

Bên cạnh những điểm mạnh vượt trội, hệ thống hiện tại (phiên bản 1.4.11) vẫn bộc lộ nhiều hạn chế cần được nghiên cứu khắc phục. Thứ nhất, quá trình trích xuất thông tin để xây dựng đồ thị phụ thuộc hoàn toàn vào chất lượng của mô hình ngôn ngữ lớn làm nền tảng. Nếu mô hình không hiểu sâu sát văn phong học vụ, hệ thống dễ nhận diện sai hoặc bỏ sót mối quan hệ. Thứ hai, cơ chế trích xuất nội dung văn bản mặc định của LightRAG chủ yếu dựa trên các thư viện đọc trực tiếp (như `pypdf`, `python-docx`, hoặc `docling`) để xử lý các tài liệu PDF, Word. Phương pháp này bộc lộ yếu điểm lớn khi áp dụng vào kho tài liệu thực tế của trường đại học, vốn chứa rất nhiều tài liệu dạng ảnh, các file PDF được scan từ văn bản giấy, hoặc các slide bài giảng có cấu trúc phức tạp. Do thiếu cơ chế Nhận dạng Ký tự Quang học (OCR) chuyên sâu được tối ưu hóa cho tài liệu Tiếng Việt dạng này, hệ thống không thể đọc được hoặc đọc sai lệch nội dung từ các hình ảnh, dẫn đến việc mất mát lượng lớn

thông tin quan trọng trước khi đưa vào mô hình học.

Thứ ba, LightRAG hiện đang thiếu hụt một hệ thống thu thập dữ liệu tự động từ các nguồn trực tuyến. Việc không có sẵn các công cụ cào dữ liệu từ các trang web của trường hay khoa khiến quá trình cập nhật thông tin trở nên thủ công và mất thời gian. Thứ tư, các câu lệnh định hướng (prompt) mặc định của hệ thống được phân bổ quá chung chung. Chúng hoàn toàn không phù hợp để xử lý các tài liệu đặc thù của trường đại học, vốn chứa đầy đặc các từ viết tắt, thuật ngữ chuyên ngành và các quy tắc học vụ riêng biệt. Thứ năm, cơ chế hợp nhất mặc định của LightRAG chỉ nhận diện hai thực thể là cùng nút khi tên của chúng khớp nhau sau khi chuẩn hóa về chữ thường. Điều này có nghĩa *HCMUS* và *Trường Đại học Khoa học Tự nhiên* sẽ tồn tại như hai nút hoàn toàn riêng biệt trong đồ thị. Hậu quả trực tiếp là tri thức về cùng một đối tượng bị phân tán, các quan hệ thuộc về một hình thức tên không được kết nối với các quan hệ thuộc hình thức tên khác, và hệ thống truy vấn không thể tổng hợp thông tin một cách đầy đủ. Cuối cùng, cấu trúc Đồ thị tri thức hiện tại của hệ thống vẫn còn thiếu các cơ chế ràng buộc hoặc kiểm chứng tính đúng đắn. Việc không có các quy tắc xác thực chặt chẽ khiến đồ thị dễ bị nhiễu bởi các thực thể sai lệch sinh ra từ quá trình trích xuất tự động, từ đó làm giảm đi độ tin cậy của toàn bộ hệ thống tư vấn. Trong môi trường corpus học thuật tiếng Việt, một thực thể trong thế giới thực thường xuất hiện với nhiều hình thức biểu diễn ngôn ngữ khác nhau tùy theo ngữ cảnh của từng tài liệu.

2.8 Hệ thống Đa tác nhân (Multi-Agent Systems) và Luồng suy luận (Agentic Workflow)

Mặc dù sự kết hợp giữa LLM và LightRAG đã giải quyết được bài toán truy xuất thông tin học vụ chính xác, bản thân mô hình ngôn ngữ vẫn chỉ

đóng vai trò thụ động (nhận câu hỏi và trả lời). Để hệ thống có thể tự động lập kế hoạch, tự kiểm tra lỗi và sử dụng các công cụ một cách linh hoạt, việc áp dụng kiến trúc AI Agent (Tác nhân AI) là thiết yếu.

2.8.1 Khái niệm AI Agent

AI Agent (Tác nhân AI) là một hệ thống sử dụng Mô hình ngôn ngữ lớn (LLM) làm trung tâm để điều khiển quá trình suy luận và đưa ra quyết định [26]. Khác với LLM truyền thống, một Agent hoàn chỉnh bao gồm 4 thành phần cốt lõi: Nền tảng của một Agent bắt đầu từ Hồ sơ (Persona/Profile), đóng vai trò thiết yếu nhằm định hình tính cách và nhiệm vụ cụ thể của nó (ví dụ như: Cố vấn học tập, Trợ giảng). Song song với hồ sơ, hệ thống không thể thiếu Bộ nhớ (Memory). Thành phần này đảm bảo khả năng lưu trữ ngữ cảnh ngắn hạn từ các đoạn hội thoại gần nhất, đồng thời duy trì kiến thức dài hạn liên quan đến thông tin, sở thích và điểm số của sinh viên. Để có thể hành động thực tế, Agent được trang bị các Công cụ (Tools). Nhờ đó, nó có khả năng tương tác trực tiếp với môi trường bên ngoài, ví dụ như thực hiện gọi API để truy xuất dữ liệu từ LightRAG, hay kích hoạt các công cụ tìm kiếm web. Yếu tố làm nên sự khác biệt cốt lõi của Agent chính là năng lực Lập kế hoạch (Planning). Khi đối mặt với một truy vấn phức tạp từ người dùng, Agent có khả năng tự động phân tích và chia nhỏ vấn đề thành nhiều bước xử lý tuần tự để giải quyết một cách triệt để.

2.8.2 Tích hợp LightRAG và Multi-Agent

Sự kết hợp giữa Multi-Agent và LightRAG tạo ra một hệ thống hỗ trợ học tập toàn diện. Trong đó, LightRAG đóng vai trò là “Bộ nhớ kiến thức dài hạn” (Long-term Knowledge Memory) với độ chính xác cao và chi phí tối ưu, còn Multi-Agent đóng vai trò là “Trung tâm xử lý logic” (Reasoning Engine). Tác nhân AI sẽ tự động quyết định khi nào cần sử dụng truy xuất cấp độ thấp (chỉ tìm thông tin 1 môn học) và khi nào cần truy xuất

cấp độ cao (tìm lộ trình tổng thể) trên nền tảng LightRAG, từ đó mang lại trải nghiệm tư vấn cá nhân hóa và sát với thực tế nhất cho sinh viên Khoa Công nghệ Thông tin.

2.9 Đánh giá hệ thống RAG và Multi-Agent

Sau khi thiết kế và triển khai hệ thống thông minh dựa trên kiến trúc LightRAG và Multi-Agent, việc thiết lập một quy trình đánh giá chuẩn xác là vô cùng cần thiết. Đặc thù của dữ liệu giáo dục đại học yêu cầu hệ thống không chỉ trả lời tự nhiên mà phải đảm bảo tính chính xác tuyệt đối, không gây ảnh hưởng sai lệch đến quyết định đăng ký môn học của sinh viên.

2.9.1 Các chỉ số đánh giá RAG (RAGAS)

Để đánh giá chất lượng của kiến trúc RAG một cách tự động và định lượng, RAGAS (Retrieval Augmented Generation Assessment) [6] hiện là framework tiêu chuẩn được cộng đồng nghiên cứu sử dụng rộng rãi. RAGAS đánh giá hệ thống thông qua hai khía cạnh chính: khả năng truy xuất (Retrieval) và khả năng sinh văn bản (Generation), với 4 chỉ số cốt lõi: Chỉ số trọng tâm nhất là Độ trung thực (Faithfulness), nhằm đo lường mức độ câu trả lời của AI bám sát vào ngữ cảnh (context) đã được hệ thống truy xuất. Giả sử hệ thống bịa ra một quy định chuẩn đầu ra Tiếng Anh hoàn toàn không có trong tài liệu của trường, chỉ số này sẽ lập tức bị đánh giá thấp, qua đó phơi bày tình trạng “ảo giác” (hallucination). Bên cạnh tính trung thực, Độ liên quan của câu trả lời (Answer Relevance) cũng cần được xem xét kỹ lưỡng. Tiêu chí này dùng để kiểm định xem câu trả lời do AI sinh ra có đi thẳng vào trọng tâm vấn đề mà sinh viên thắc mắc hay không, hay chỉ đang phản hồi một cách vòng vo và dư thừa thông tin. Mặt khác, đối với module truy xuất, Độ chính xác của ngữ cảnh (Context Precision) đóng vai trò định lượng chất lượng tìm kiếm. Nó cho

biết liệu các tài liệu hoặc các node trên đồ thị tri thức quan trọng nhất có được thuật toán ưu tiên xếp hạng ở những vị trí đầu tiên hay không. Bổ sung cho độ chính xác là Độ bao phủ của ngữ cảnh (Context Recall). Đây là thước đo đánh giá xem bộ truy xuất có thu thập “đủ” các thông tin cần thiết để phục vụ cho câu hỏi hay không. Nếu sinh viên hỏi một vấn đề đòi hỏi sự kết hợp giữa đề cương môn học và quy chế học vụ, chỉ số này sẽ phản ánh việc hệ thống có bỏ sót bất kỳ tài liệu quan trọng nào không.

2.9.2 Đánh giá hệ thống đa agent (Multi-Agent Evaluation)

Việc đánh giá một hệ thống Đa tác nhân (Multi-Agent) phức tạp hơn RAG thông thường, bởi nó không chỉ là đầu vào - đầu ra (input/output) mà là một quy trình suy luận gồm nhiều bước (Agentic workflow). Các tiêu chí đánh giá tập trung vào: Thước đo quan trọng hàng đầu là Tỷ lệ hoàn thành nhiệm vụ (Task Completion Rate). Chỉ số này tập trung đo lường tần suất các Agent có thể phối hợp thành công để giải quyết trọn vẹn một luồng công việc, ví dụ như đưa ra một kế hoạch học tập hoàn chỉnh cho sinh viên. Ngoài việc hoàn thành đúng nhiệm vụ, Hiệu suất và Chi phí (Efficiency & Cost) cũng là bài toán thực tế cần lời giải. Nhóm nghiên cứu cần tiến hành đánh giá chi tiết về độ trễ trung bình của hệ thống (Latency) cũng như chi phí tài nguyên (dựa trên số lượng API Token tiêu thụ cho mỗi phiên hội thoại). Nhờ vào sự tối ưu của kiến trúc, việc tích hợp LightRAG được kỳ vọng sẽ giảm thiểu đáng kể chỉ số chi phí này so với các mô hình GraphRAG truyền thống.

2.9.3 Các phương pháp benchmark hiện nay

Để tiến hành đo lường các chỉ số trên, nhóm dự kiến áp dụng các phương pháp benchmark (chấm điểm chuẩn) sau: Nền móng của toàn bộ quá trình đánh giá là việc xây dựng tập dữ liệu chuẩn (Golden Dataset

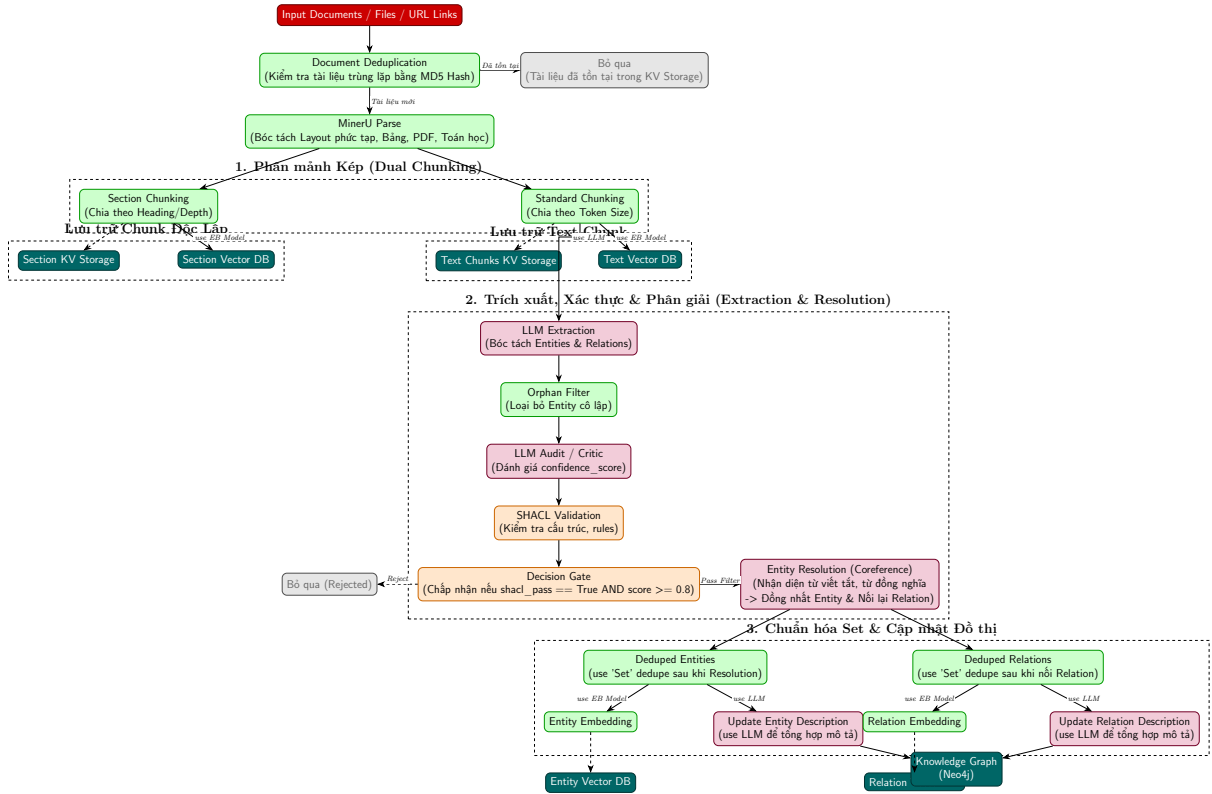
/ Ground Truth). Để thực hiện điều này, nhóm sẽ tự biên soạn một bộ khoảng 100-200 cặp câu hỏi - câu trả lời chuẩn xác, bao quát các kịch bản thực tế như đăng ký môn tiên quyết, tư vấn hướng chuyên ngành, hay quy định xét tốt nghiệp. Toàn bộ câu trả lời trong tập dữ liệu này sẽ được trích xuất thủ công và chính xác từ các văn bản chính thức của Khoa Công nghệ Thông tin, nhằm làm mốc tham chiếu để đối chiếu tự động với kết quả đầu ra của hệ thống. Để tự động hóa quá trình chấm điểm quy mô lớn, phương pháp LLM-as-a-Judge (Sử dụng LLM mạnh hơn để chấm điểm) sẽ được áp dụng. Thay vì phải tốn công sức chấm điểm thủ công hàng ngàn câu trả lời, nhóm sẽ sử dụng các mô hình ngôn ngữ tiên tiến nhất hiện nay (như GPT-4 hoặc Claude 3.5) để đóng vai trò là “Giám khảo” [29]. Vị giám khảo này sẽ có nhiệm vụ đọc câu hỏi, đối chiếu câu trả lời chuẩn (Ground Truth) với câu trả lời của hệ thống, từ đó chấm điểm các tiêu chí RAGAS một cách khách quan và nhất quán. Bước kiểm chứng cuối cùng và cũng mang tính quyết định nhất chính là Đánh giá từ người dùng thực tế (Human-in-the-loop Evaluation). Trong giai đoạn này, hệ thống sẽ được triển khai thử nghiệm (Beta testing) cho một nhóm sinh viên và giảng viên cố vấn học tập của Khoa. Thông qua các khảo sát về trải nghiệm người dùng (UX) và mức độ hữu ích của thông tin, những phản hồi thu thập được sẽ là cơ sở thực tiễn vô giá để nhóm tiếp tục tinh chỉnh các Prompt cũng như luồng hoạt động của Agent.

Chương 3

Phương pháp đề xuất

Trên cơ sở phân tích lý thuyết và yêu cầu thực tiễn, đề xuất kiến trúc hệ thống hỗ trợ học tập thông minh dựa trên sự LightRAG [9] một mã nguồn mở được phát triển từ GraphRAG [19] và LangGraph.

3.1 Quy trình xử lý dữ liệu



Hình 3.1: Quy trình tổng quát từ văn bản thô đến đồ thị tri thức

Đây là quy trình xử lý thông tin, đóng vai trò "nguyên liệu đầu vào" cho toàn bộ hệ thống. Hệ thống không chỉ lưu trữ dữ liệu thô mà còn thực hiện các phép biến đổi vector hóa phức tạp. Quản lý nguồn dữ liệu đa dạng bao gồm hồ sơ người học và các học liệu phi cấu trúc như bài giảng, tài liệu tham khảo.

Đồ thị tri thức (Knowledge Graph) [10]: đây là thành phần quan trọng nhất, lưu trữ tri thức dưới dạng cấu trúc mạng lưới. Các nút đại diện cho các khái niệm và các cạnh biểu diễn mối quan hệ sự phạm. Điều này giúp hệ thống hiểu được "vị trí" và "tầm quan trọng" của một khái niệm trong tổng thể chương trình học.

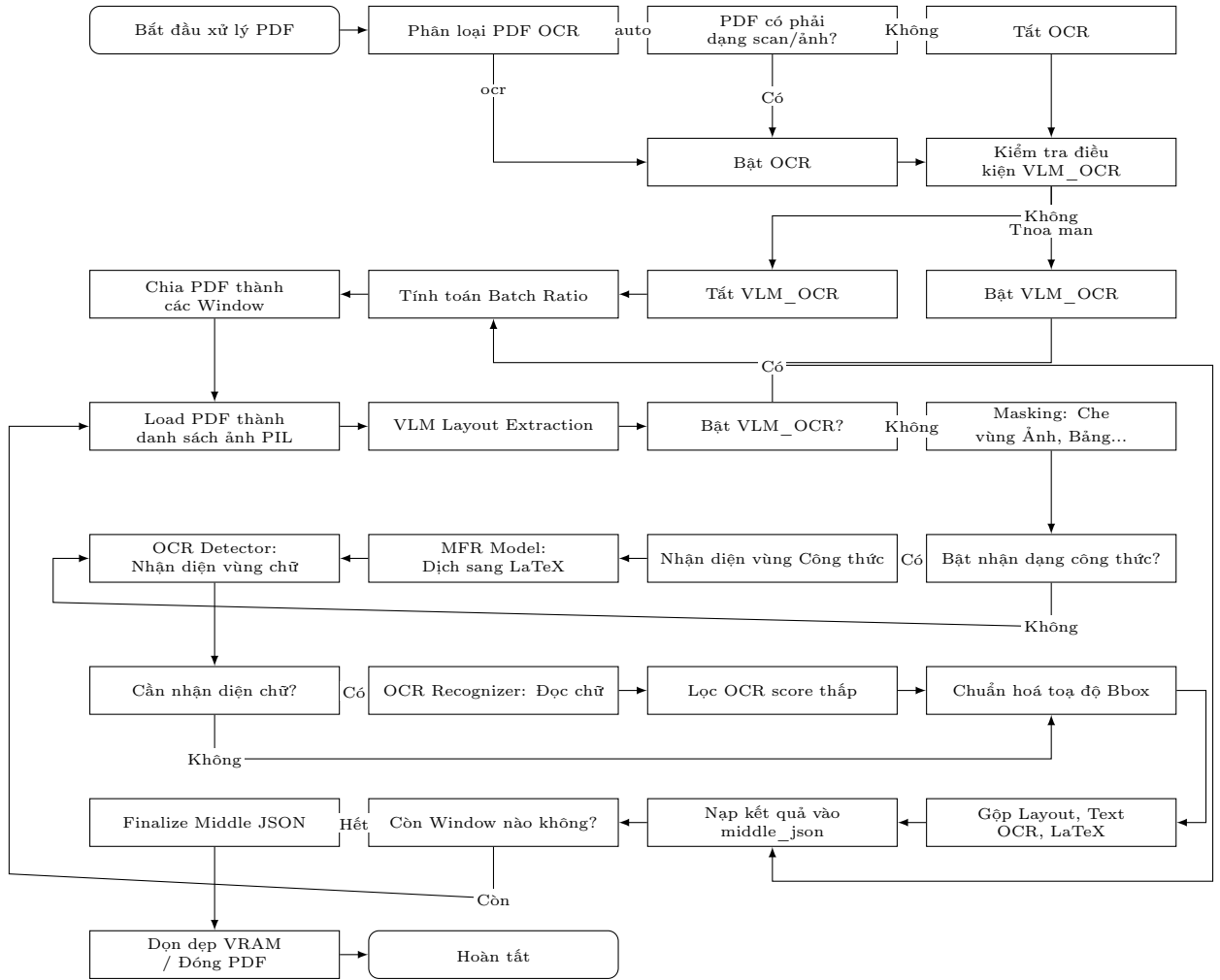
3.1.1 Tiền xử lý dữ liệu tri thức

3.1.1.1 Hệ thống MinerU

Cơ sở lựa chọn công cụ và vai trò trong hệ thống: Trong môi trường học thuật, nguồn dữ liệu đầu vào thường rất đa dạng và phức tạp, bao gồm các tệp PDF được scan từ văn bản giấy, tài liệu chứa nhiều biểu đồ, hình ảnh, và đặc biệt là các công thức toán học. Các công cụ trích xuất văn bản truyền thống thường xuyên gặp thất bại trong việc xử lý các tài liệu này, dẫn đến việc mất thông tin quan trọng hoặc làm xáo trộn cấu trúc bài giảng.

Để giải quyết triệt để thách thức này, hệ thống áp dụng công cụ MinerU **mineru_2024** (từ phiên bản 3.2.3 trở lên) làm giải pháp lõi cho quy trình tiền xử lý dữ liệu. Đây là một công cụ trích xuất dữ liệu đa phương thức chất lượng cao, tích hợp các mô hình tiên tiến như Vision-Language Model (VLM) cho việc phân tích bố cục (Layout Analysis), mô hình OCR để nhận diện văn bản trên ảnh, và mô hình Mathematical Formula Recognition (MFR) để chuyển đổi công thức toán học sang chuẩn LaTeX. Sự kết hợp này giúp MinerU không chỉ trích xuất chính xác văn bản mà còn bảo toàn hoàn hảo cấu trúc logic của tài liệu gốc, đặc biệt phù hợp với các giáo trình, bài báo khoa học và tài liệu lưu trữ của trường đại học.

Luồng xử lý tự động (Hybrid-Auto-Engine): Chế độ **-b hybrid -auto-engine** trong MinerU thực thi một quy trình xử lý khép kín, kết hợp giữa mô hình đa phương thức (VLM) để bóc tách cấu trúc và các mô hình truyền thống (OCR, MFR) để nhận dạng văn bản. Dưới đây là sơ đồ chi tiết về luồng hoạt động này:



Hình 3.2: Quy trình xử lý Hybrid-Auto-Engine trong MinerU mineru_2024

Quy trình xử lý được thực hiện tuần tự qua các giai đoạn. Đầu tiên, hệ thống tiến hành phân loại và phân tích định dạng của tệp đầu vào để xác định là văn bản gốc hay bản quét. Dựa trên kết quả này, cơ chế nhận dạng ký tự quang học sẽ tự động được kích hoạt nếu cần thiết. Tiếp theo, hệ thống đánh giá tài nguyên phần cứng hiện có, cụ thể là dung lượng bộ nhớ đồ họa VRAM, để tính toán tỷ lệ phân lô phù hợp, đảm bảo tối ưu hóa hiệu suất mà không gây tràn bộ nhớ.

Sau quá trình khởi tạo, tài liệu được phân chia thành các đoạn nhỏ để xử lý dần. Tại mỗi phân đoạn, hệ thống sử dụng một mô hình đa phương thức VLM mạnh mẽ để bóc tách cấu trúc tổng thể, nhận diện các khối văn

bản, bảng biểu, hình ảnh và công thức. Đối với các vùng chứa hình ảnh và bảng biểu, hệ thống sẽ tiến hành làm mờ nhằm không gây nhiễu cho các mô hình phía sau. Đồng thời, các công thức toán học nằm trong dòng văn bản sẽ được mô hình MFR chuyên biệt nhận diện và dịch đổi sang định dạng mã chuẩn LaTeX. Các khối chữ chưa xử lý được chuyển qua mô hình nhận diện ký tự quang học OCR. Nhằm đảm bảo độ chính xác, các kết quả có độ tin cậy thấp sẽ bị hệ thống tự động lọc bỏ.

Cuối cùng, ở giai đoạn tổng hợp, tọa độ của các khối thông tin được chuẩn hóa về một hệ quy chiếu chung. Tất cả các dữ liệu trích xuất từ mô hình đa phương thức, mô hình nhận diện chữ và mô hình công thức được gộp chung vào một cấu trúc dữ liệu trung gian. Quy trình này lặp lại cho đến khi xử lý hết các phân đoạn của tài liệu, sau đó hệ thống sẽ giải phóng tài nguyên bộ nhớ và hoàn tất việc tạo ra một văn bản đầu ra đồng nhất và giữ nguyên cấu trúc logic gốc.

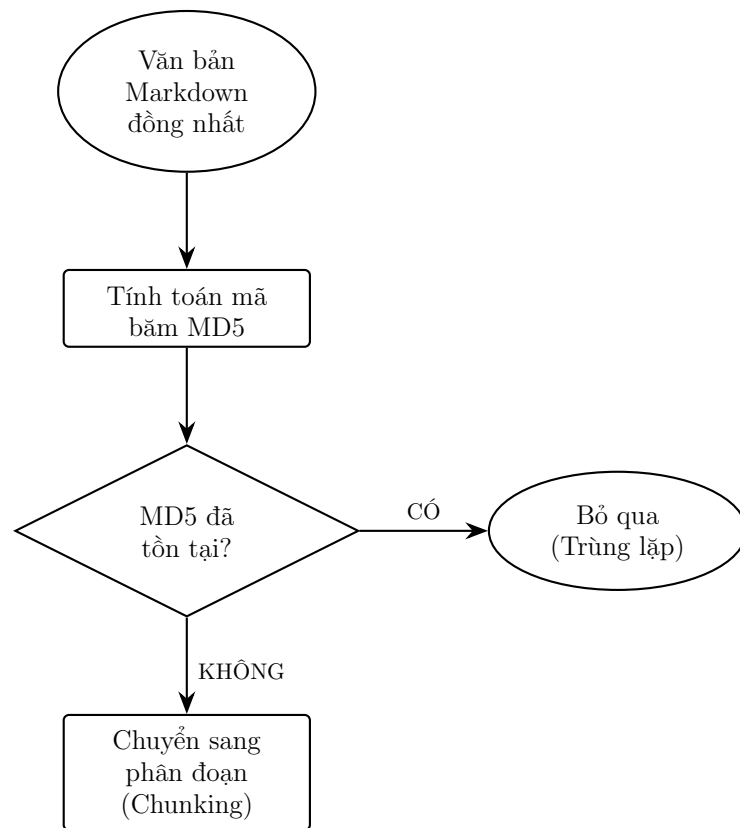
Nhờ đường ống Hybrid này, MinerU khắc phục hoàn toàn những nhược điểm của việc đọc tệp bằng `textract` hay `pypdf` thông thường, cung cấp dòng dữ liệu đầu vào sắc nét và có độ tin cậy tuyệt đối cho mô hình ngôn ngữ phía sau.

3.1.2 Luồng logic xử lý và cơ chế kiểm tra trùng lặp

Một trong những thành phần then chốt giúp hệ thống vận hành hiệu quả và tiết kiệm chi phí là cơ chế kiểm soát dữ liệu tại cổng vào. Sau khi văn bản được trích xuất từ các nguồn đa dạng (PDF, Office, Web) và đưa về định dạng Markdown đồng nhất, hệ thống thực hiện quy trình kiểm tra trùng lặp (Deduplication) dựa trên mã băm nội dung.

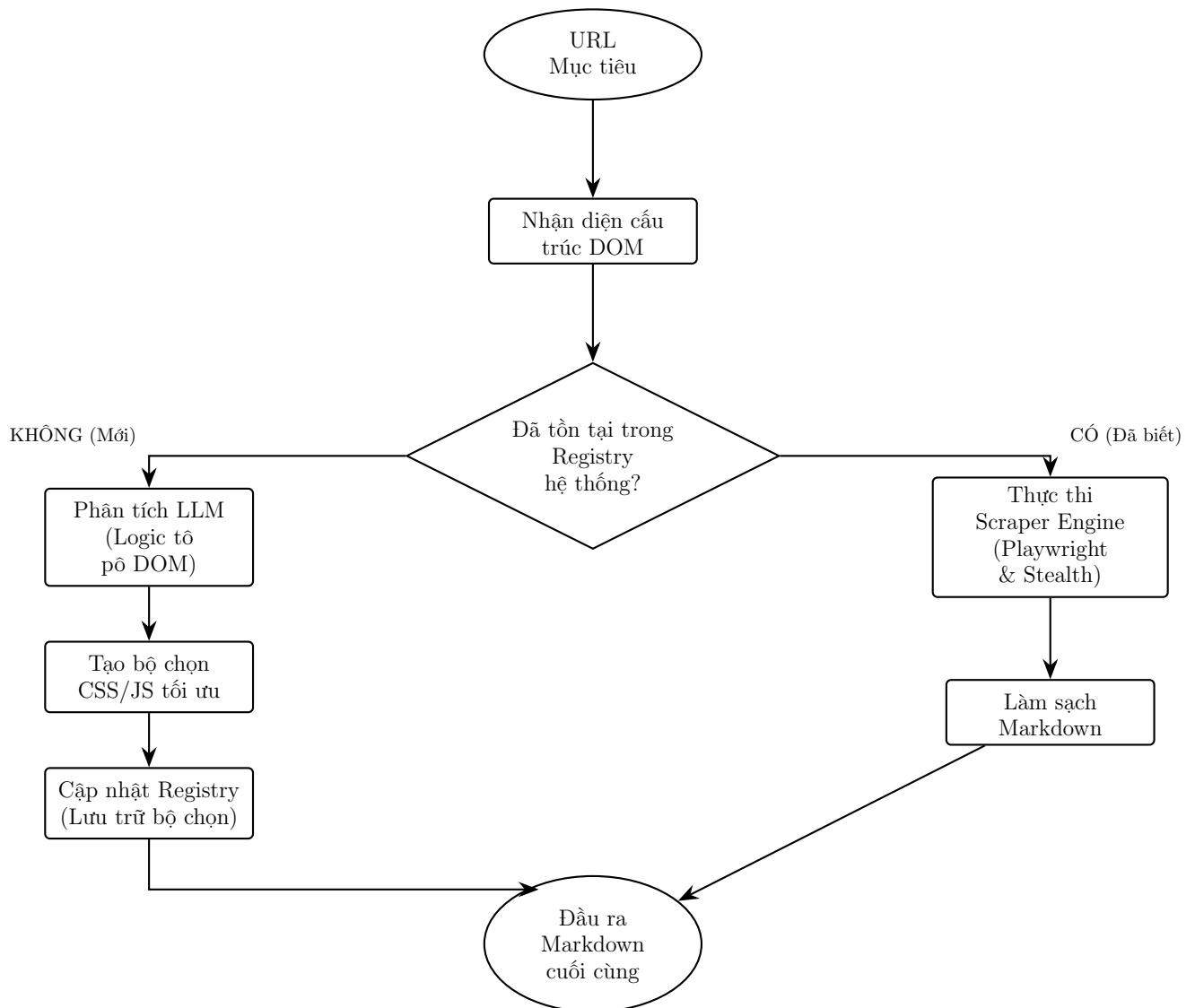
Cụ thể, hệ thống sử dụng thuật toán băm MD5 để tạo ra một "dấu vân tay" kỹ thuật số duy nhất cho toàn bộ nội dung văn bản. Mã băm này sau đó được đối chiếu với cơ sở dữ liệu tri thức hiện tại, dẫn đến hai kịch bản xử lý chính. Trong trường hợp trùng khớp hoàn toàn, nếu mã MD5 của tài liệu mới đã tồn tại, hệ thống sẽ ngay lập tức dừng quy trình xử

lý nhằm ngăn chặn việc nạp lại dữ liệu dư thừa, bảo vệ tính toàn vẹn của đồ thị tri thức và tiết kiệm tối đa tài nguyên tính toán. Ngược lại, đối với trường hợp nội dung mới hoặc có sự thay đổi, tài liệu được coi là nguồn tri thức mới và được chuyển tiếp sang giai đoạn phân đoạn. Ngay cả khi tài liệu chỉ có một thay đổi nhỏ, hệ thống vẫn sẽ thực hiện lại quy trình để đảm bảo tính cập nhật, trong khi sự trùng lặp về mặt thực thể sẽ được tự động hợp nhất tại bước xây dựng đồ thị sau này.



Hình 3.3: Luồng logic kiểm tra trùng lặp bằng cơ chế băm nội dung

3.1.2.1 Hệ thống thu thập tri thức từ trang web



Hình 3.4: Quy trình thu thập tri thức từ trang web với Smart Scraper

Hệ thống cào web thông minh hoạt động theo quy trình tự động hóa từ việc nhận diện cấu trúc trang đến trích xuất dữ liệu. Logic cốt lõi dựa trên công cụ Smart Scraper Engine với cơ chế vận hành qua bốn bước chính: lấy dấu vân tay, đối chiếu kho lưu trữ, phân tích cấu trúc bằng mô hình ngôn ngữ và thực thi bộ phân giải.

Quy trình bắt đầu bằng việc tạo ra một bản tóm tắt cấu trúc cây DOM của trang web, tập trung vào các thuộc tính định danh như thẻ HTML,

tên lớp và ID. Hệ thống trích xuất một bộ khung xương kỹ thuật của trang web để làm cơ sở cho việc nhận diện. Sau đó, bản tóm tắt này được đối chiếu với kho lưu trữ các bộ phân giải đã biết. Mỗi mục trong kho lưu trữ chứa các thông tin về dấu hiệu nhận biết, bộ chọn nội dung chính, các thành phần cần loại bỏ và chiến lược chờ đợi phù hợp cho từng loại công nghệ web như React, Vue, hay các hệ quản trị nội dung truyền thống.

Trong trường hợp gặp một trang web mới không có trong kho lưu trữ, hệ thống sẽ gửi một phần cấu trúc DOM tiêu biểu cho mô hình ngôn ngữ lớn. Mô hình này có nhiệm vụ phân tích logic phân bổ nội dung và tự động sinh ra mã lệnh điều hướng. Mã lệnh này bao gồm các bộ chọn CSS tối ưu để trích xuất văn bản có ý nghĩa và các quy tắc loại bỏ những thành phần gây nhiễu như trình đơn điều hướng, biểu ngữ quảng cáo hoặc các đoạn mã kịch bản. Bộ phân giải mới sinh ra sẽ được kiểm tra và lưu lại vào kho lưu trữ để sử dụng cho các lần truy cập sau, giúp tối ưu hóa hiệu suất và giảm thiểu chi phí gọi API.

Đối với các tên miền mặc định như `fit.hcmus.edu.vn` và `hcmus.edu.vn`, hệ thống đã được cấu hình sẵn các bộ phân giải chuyên biệt cho nền tảng DotNetNuke. Do đặc thù của nền tảng này thường chứa nhiều nội dung lồng ghép trong các khung phức tạp, hệ thống sử dụng các bộ chọn mục tiêu như `.dnn_contentPane` và `#dnn_ContentPane` để định vị chính xác vùng chứa thông tin học thuật. Quy trình trích xuất văn bản tại đây không chỉ lấy chữ thuần túy mà còn thực hiện chuyển đổi các liên kết quan trọng sang định dạng Markdown để bảo toàn thông tin tham chiếu.

Đặc biệt, đối với các trang có tính tương tác cao như danh mục môn học hoặc đề cương chi tiết, hệ thống áp dụng chế độ tương tác vét cạn. Chế độ này sử dụng thư viện Playwright để tự động thực hiện các hành động như nhấn vào các nút mở rộng, cuộn trang để tải thêm dữ liệu và chờ đợi các tiến trình AJAX hoàn tất. Điều này đảm bảo rằng toàn bộ tri thức về chương trình đào tạo, mô tả môn học và thông tin giảng viên được thu thập đầy đủ nhất, ngay cả khi chúng nằm sau các thành phần giao diện động. Dữ liệu sau khi trích xuất sẽ được làm sạch và chuẩn hóa trước

khi đưa vào các bước phân đoạn và xây dựng đồ thị tri thức tiếp theo.

3.1.3 Chiến lược phân đoạn và chuẩn hóa văn bản

Quá trình xử lý khởi đầu bằng việc làm sạch dữ liệu đầu vào, loại bỏ các ký tự lỗi và đảm bảo văn bản tuân thủ chuẩn mã hóa quốc tế Unicode (UTF-8). Bước tiền xử lý này không chỉ chuẩn hóa định dạng mà còn loại bỏ các thẻ HTML dư thừa, khoảng trắng không cần thiết và các ký tự điều khiển có thể gây nhiễu cho mô hình ngôn ngữ ở các bước xử lý tiếp theo.

Về mặt kỹ thuật, hệ thống áp dụng song song hai chiến lược phân đoạn chính nhằm phục vụ hai mục tiêu khác biệt trong kiến trúc tổng thể: chiến lược phân đoạn theo kích thước cố định phục vụ trích xuất đồ thị tri thức và chiến lược phân đoạn phân cấp đa phương thức phục vụ truy xuất ngữ cảnh chính xác.

Chiến lược thứ nhất, phân đoạn theo kích thước cố định (fixed-size chunking), được thiết kế chuyên biệt để phục vụ quy trình trích xuất thực thể và quan hệ nhằm xây dựng đồ thị tri thức trong LightRAG [9]. Phương pháp này chia toàn bộ văn bản đã chuẩn hóa thành các đoạn có độ dài xác định trước, thường trong khoảng từ 512 đến 1200 token, kèm theo một vùng chồng lấp (overlap window) chiếm khoảng 10–20% kích thước đoạn giữa hai đoạn liên tiếp.

Cơ chế chồng lấp này đóng vai trò như một “cầu nối ngữ cảnh” (context bridge): khi mô hình ngôn ngữ xử lý đoạn thứ i , nó đã có sẵn phần đuôi của đoạn thứ $i - 1$ làm nền tảng, từ đó tránh được hiện tượng cắt đứt giữa chừng một câu hoặc một luận điểm quan trọng. Cụ thể, với kích thước đoạn mặc định là 1200 token và vùng chồng lấp 100 token, hệ thống đảm bảo rằng các thực thể và mối quan hệ nằm gần ranh giới giữa hai đoạn liên tiếp vẫn được mô hình ngôn ngữ lớn nhận diện đầy đủ trong ít nhất một đoạn.

Chiến lược phân đoạn kích thước cố định được lựa chọn cho tác vụ trích xuất đồ thị do quy trình trích xuất thực thể và quan hệ trong LightRAG

yêu cầu mỗi đoạn văn phải đủ dài để chứa ngữ cảnh phong phú, giúp mô hình ngôn ngữ lớn nhận diện chính xác các thực thể và suy luận được mối liên hệ giữa chúng. Các đoạn quá ngắn sẽ thiếu ngữ cảnh, dẫn đến việc bỏ sót các quan hệ ngầm định giữa các câu không liền kề.

Chiến lược thứ hai, phân đoạn phân cấp (hierarchical chunking), được thiết kế nhằm bảo toàn tối đa cấu trúc và ngữ nghĩa của tài liệu gốc, phục vụ cho việc truy xuất ngữ cảnh chính xác khi hệ thống nhận truy vấn từ người dùng. Chiến lược này tham khảo từ MultiDocFusion **shin-etal-2025-multidocfusion**, một hệ thống phân đoạn phân cấp được đề xuất cho các tài liệu dài và phức tạp, đã chứng minh hiệu quả vượt trội trong việc cải thiện độ chính xác truy xuất từ 8–15% so với các phương pháp phân đoạn truyền thống.

Khác với phân đoạn kích thước cố định vốn xử lý văn bản như một chuỗi ký tự thuần túy, chiến lược phân cấp đa phương thức nhận diện rằng tài liệu học thuật không chỉ chứa văn bản tuyến tính mà còn bao gồm nhiều thành phần có cấu trúc và ngữ nghĩa khác nhau. Do đó, quy trình phân đoạn không thể chỉ dựa trên số lượng token mà cần xem xét ba loại cấu trúc: (i) cấu trúc phân cấp (structural), tức mối quan hệ cha-con giữa các tiêu đề, mục, đoạn văn; (ii) cấu trúc bố cục (layout), tức vị trí vật lý, cột, vùng bao đóng của các thành phần trên trang; và (iii) cấu trúc ngữ nghĩa (semantic), tức mối quan hệ về nghĩa giữa các đoạn nội dung dù không kề nhau về vị trí vật lý.

Quy trình phân đoạn phân cấp đa phương thức trong hệ thống được triển khai qua bốn giai đoạn tuần tự, tương ứng với bốn bước chính trong kiến trúc MultiDocFusion **shin-etal-2025-multidocfusion**:

Giai đoạn 1: Phân tích cấu trúc tài liệu bằng thị giác máy tính (Vision-based Document Parsing). Hệ thống VMiner (đã trình bày ở mục trước) sử dụng các mô hình học sâu chuyên biệt để phân tích cấu trúc hình học của từng trang tài liệu. Quá trình này tự động phát hiện và phân loại các vùng nội dung thành các loại thành phần riêng biệt: vùng tiêu đề (heading), khối văn bản (text block), bảng biểu (table), hình ảnh và biểu

đồ (figure/chart), danh sách (list), chú thích (footnote) và công thức toán học. Mỗi vùng được xác định bởi một hộp bao đóng (bounding box) chứa tọa độ chính xác trên trang, làm cơ sở cho việc sắp xếp thứ tự đọc hợp lý và phân loại nội dung ở các bước tiếp theo.

Giai đoạn 2: Trích xuất nội dung và phân loại thành phần đa phương thức. Sau khi nhận diện các vùng nội dung, hệ thống tiến hành trích xuất nội dung tương ứng từ từng vùng. Đối với các vùng văn bản thuần túy, nội dung được trích xuất bằng các công cụ nhận dạng ký tự quang học (OCR) đã tối ưu hóa cho tiếng Việt. Đối với bảng biểu, hệ thống tái tạo cấu trúc dữ liệu dưới dạng Markdown hoặc HTML, bảo toàn nguyên vẹn thông tin hàng, cột và tiêu đề. Đối với hình ảnh và biểu đồ, hệ thống sinh mô tả văn bản tự động thông qua mô hình ngôn ngữ thị giác (Vision Language Model, VLM), kết hợp với chú thích (caption) gốc và văn bản ngữ cảnh xung quanh hình ảnh trong tài liệu. Bước phân loại này đảm bảo mỗi thành phần được xử lý bởi chiến lược phù hợp nhất với đặc trưng dữ liệu của nó.

Giai đoạn 3: Xây dựng cây cấu trúc phân cấp (Hierarchical Structure Parsing). Đây là giai đoạn cốt lõi phân biệt chiến lược phân cấp với các phương pháp phân đoạn truyền thống. Hệ thống tái cấu trúc toàn bộ tài liệu thành một cây phân cấp (hierarchical tree) phản ánh tổ chức logic của nội dung, trong đó gốc cây là tài liệu, các nút trung gian là các mục (section) và tiểu mục (sub-section), còn các nút lá là các đơn vị nội dung cụ thể như đoạn văn, bảng, hình ảnh. Cấu trúc phân cấp này được suy luận từ hệ thống tiêu đề (heading hierarchy) trong tài liệu, cụ thể các cấp tiêu đề H1, H2, H3 tương ứng với các mức độ lồng nhau khác nhau trong cây. Trong trường hợp tài liệu không có tiêu đề rõ ràng hoặc cấu trúc tiêu đề bị nhiễu, hệ thống bổ sung cơ chế phân tích dựa trên mô hình ngôn ngữ lớn (tương tự kỹ thuật DSHP-LLM trong MultiDocFusion **shin-etal-2025-multidocfusion**) để suy luận ranh giới ngữ nghĩa giữa các phần. Mỗi nút trong cây được gán một đường dẫn phân cấp (breadcrumb path) dạng: [Tài liệu] > [Chương X] > [Mục X.Y]

> [Tiêu mục X.Y.Z], cung cấp ngữ cảnh vị trí chính xác cho từng đơn vị nội dung.

Giai đoạn 4: Gom nhóm và sinh phân đoạn theo chiến lược DFS (DFS-based Grouping). Giai đoạn cuối cùng thực hiện việc duyệt cây phân cấp theo chiều sâu (Depth-First Search) để gom nhóm các nút lá thành các phân đoạn hoàn chỉnh, tuân thủ giới hạn kích thước token tối đa đã được cấu hình. Thuật toán duyệt DFS đảm bảo rằng các nút con thuộc cùng một nút cha sẽ được ưu tiên gom vào cùng một phân đoạn, từ đó bảo toàn tính liên kết ngữ nghĩa theo cấu trúc phân cấp gốc. Cụ thể, quy tắc gom nhóm được áp dụng như sau:

- Nếu thành phần là một đoạn văn ngắn (dưới 100 token): gộp với các đoạn văn tiếp theo thuộc cùng mục (section) cho đến khi đạt ngưỡng kích thước tối thiểu.

- Nếu thành phần là một đoạn văn dài (vượt quá giới hạn token): áp dụng kỹ thuật tách nhận biết ranh giới câu (sentence-aware splitting) để tránh cắt ngang ý tưởng.

- Nếu thành phần là bảng biểu: giữ nguyên toàn bộ bảng như một phân đoạn độc lập kèm theo chú thích bảng và ngữ cảnh mục chứa bảng. Đối với bảng quá lớn vượt giới hạn token, áp dụng chiến lược tách theo nhóm hàng kèm giữ nguyên hàng tiêu đề.

- Nếu thành phần là hình ảnh hoặc biểu đồ: tạo phân đoạn hình ảnh chứa mô tả sinh bởi VLM, chú thích (caption) gốc và đoạn văn bản ngữ cảnh trước/sau hình ảnh, thiết lập liên kết chéo giữa các thành phần này.

- Nếu thành phần là slide (đối với tài liệu PPTX): giữ nguyên toàn bộ nội dung slide gồm tiêu đề, khung văn bản, danh sách và ghi chú diễn giả làm một phân đoạn; tách khi nội dung vượt giới hạn.

Mỗi phân đoạn được tạo ra sẽ được nhúng (inject) thêm thông tin đường dẫn phân cấp (breadcrumb context) vào đầu nội dung, theo định dạng:

```
[Document: {tên_tài_liệu}] > [{đường_dẫn_mục}]
{nội_dung_gốc}
```

Kỹ thuật nhúng ngữ cảnh phân cấp này giúp mô hình ngôn ngữ lớn có đủ thông tin định vị ngay cả khi phân đoạn bị tách khỏi tài liệu gốc. Breadcrumb được nhúng vào đầu nội dung nhưng không được tính vào số lượng token của phần nội dung chính, nhằm không ảnh hưởng đến chất lượng biểu diễn vector nhúng (embedding) của nội dung thực.

Bảng 3.1 trình bày so sánh chi tiết giữa hai chiến lược phân đoạn được áp dụng trong hệ thống.

Bảng 3.1: So sánh hai chiến lược phân đoạn trong hệ thống

Tiêu chí	Chiến lược 1: Kích thước cố định	Chiến lược 2: Phân cấp đa phương thức
Mục đích chính	Trích xuất thực thể và quan hệ cho đồ thị tri thức	Truy xuất ngữ cảnh chính xác cho RAG
Đơn vị phân đoạn	Cửa sổ token cố định (1200 token)	Đơn vị ngữ nghĩa tự nhiên (đoạn văn, bảng, hình)
Bảo toàn cấu trúc	Không, xử lý văn bản thuần tuyến tính	Có, bảo toàn cấu trúc phân cấp gốc
Ngữ cảnh vị trí	Dựa vào vùng chồng lấp	Breadcrumb path nhúng vào mỗi chunk
Ưu điểm	Đơn giản, phân bổ tải đều, phù hợp cho trích xuất hàng loạt	Bảo toàn ngữ nghĩa, truy xuất chính xác, hỗ trợ đa phương thức
Hạn chế	Có thể cắt ngang ý tưởng	Phức tạp hơn, phụ thuộc vào chất lượng phân tích cấu trúc

Sự phối hợp giữa hai chiến lược phân đoạn tạo nên một kiến trúc lưu trữ kép (dual-indexing architecture) phục vụ cho hai luồng truy xuất khác nhau trong hệ thống. Chiến lược phân đoạn kích thước cố định cung cấp nguyên liệu đầu vào cho quy trình xây dựng đồ thị tri thức, nơi các thực thể và quan hệ được trích xuất và lưu trữ trong cơ sở dữ liệu đồ thị, phục vụ

cho các chế độ truy vấn cục bộ (local) và toàn cục (global) của LightRAG. Trong khi đó, chiến lược phân đoạn phân cấp đa phương thức cung cấp các phân đoạn có cấu trúc ngữ nghĩa chặt chẽ, được lưu trữ trong cơ sở dữ liệu vector kèm theo siêu dữ liệu phong phú (metadata), phục vụ cho việc truy xuất ngữ cảnh chính xác khi hệ thống cần bổ sung thông tin chi tiết từ tài liệu gốc. Kiến trúc kép này đảm bảo rằng hệ thống vừa có khả năng lập luận đa bước trên đồ thị tri thức, vừa có khả năng truy xuất ngữ cảnh tài liệu gốc một cách trung thực và đầy đủ. Phương pháp thứ nhất — phân đoạn theo kích thước cố định (fixed-size chunking) — chia văn bản thành các đoạn có độ dài xác định trước (thường từ 512 đến 1200 token), kèm theo một vùng chồng lấp (overlap window) chiếm khoảng 10–20% kích thước đoạn giữa hai đoạn liên tiếp. Cơ chế chồng lấp này đóng vai trò như một “cầu nối ngữ cảnh”: khi mô hình xử lý đoạn thứ i , nó đã có sẵn phần đuôi của đoạn thứ $i-1$ làm nền tảng, từ đó tránh được hiện tượng cắt đứt giữa chừng một câu hoặc một luận điểm quan trọng.

Phương pháp thứ hai — phân đoạn theo cấu trúc tự nhiên (structure-aware chunking) — khai thác các dấu hiệu cấu trúc sẵn có trong tài liệu như tiêu đề, tiêu đề phụ, danh sách có đánh số, hay các đoạn văn được phân tách bởi ký tự xuống dòng kép. Hệ thống sử dụng các biểu thức chính quy (regex) kết hợp với phân tích cú pháp nhẹ (lightweight parsing) để nhận diện ranh giới tự nhiên này. Ưu điểm của phương pháp này là mỗi đoạn văn giữ nguyên tính nguyên vẹn ngữ nghĩa — ví dụ, toàn bộ một mục trong tài liệu kỹ thuật sẽ không bị tách ra giữa chừng.

3.1.3.1 Nghiên cứu lựa chọn mô hình nhúng (Embedding Model) và giải pháp triển khai Production

Để tối ưu hóa phân hệ truy xuất ngữ cảnh (Retriever) trong kiến trúc RAG và cải thiện trực tiếp chỉ số độ gọi bối cảnh (Context Recall), hệ thống đề xuất sử dụng mô hình nhúng mã nguồn mở `intfloat/multilingual-e5-large-instruct` được phát triển bởi Microsoft Research [23]. Mô hình này sở hữu kiến trúc biểu diễn hai chiều Encoder sâu sắc, được huấn luyện

dựa trên tập ngữ liệu khổng lồ đa ngôn ngữ và đã chứng minh năng lực vượt trội khi đạt điểm số cao nhất trong bảng xếp hạng năng lực xử lý tiếng Việt tiêu chuẩn VN-MTEB (Vietnamese Massive Text Embedding Benchmark) công bố tại hội nghị khoa học EACL 2026 [20]. Qua nghiên cứu thực nghiệm, sự lựa chọn này đem lại các giá trị tối ưu vượt bậc cùng chiến lược xử lý các hạn chế vật lý bao gồm:

- Cơ chế học có chỉ dẫn theo tác vụ (Task-aware Instruction Tuning): Điểm cải tiến cốt lõi của phiên bản Instruct so với các kiến trúc nhúng phẳng thông thường là khả năng hiểu sâu sắc mục đích nghiệp vụ thông qua câu lệnh chỉ dẫn hệ thống [27]. Trước khi thực hiện vector hóa, hệ thống sẽ tự động cấu hình chèn tiền tố ngữ cảnh định hướng: chèn **"query:"** vào trước câu hỏi thảo luận học vụ của sinh viên và chèn **"passage:"** vào trước các phân đoạn tài liệu bối cảnh thô. Cơ chế này giúp mô hình chủ động căn chỉnh ma trận biểu diễn không gian, kéo các vector câu hỏi và vector bối cảnh chứa câu trả lời học vụ lại gần nhau hơn, giải quyết bài toán truy xuất bối cảnh chính xác [23].

- Khả năng căn chỉnh đa ngữ (Cross-lingual Alignment): Do đặc thù của kho tri thức học tập ngành Công nghệ thông tin đan xen dày đặc thuật ngữ tiếng Anh và tiếng Việt, mô hình E5 có khả năng ánh xạ các thực thể ngữ nghĩa đồng dạng của hai ngôn ngữ về cùng các vùng lân cận trong không gian vector 1024 chiều [23]. Điều này giúp hệ thống phản hồi chính xác về bối cảnh ngay cả khi sinh viên đặt câu hỏi bằng tiếng Việt nhưng tài liệu đề cương được lưu trữ lại bằng tiếng Anh [23].

- Hạn chế kỹ thuật về giới hạn vật lý và giải pháp tối ưu: Hạn chế lớn nhất của mô hình nhúng này là cửa sổ ngữ cảnh đầu vào (Context Window) bị giới hạn nghiêm ngặt ở mức 512 tokens [23]. Nếu phân đoạn văn bản vượt quá giới hạn này, mô hình sẽ kích hoạt cơ chế tự động cắt tỉa phần đuôi, gây thất thoát nghiêm trọng thông tin cấu trúc môn học. Để xử lý triệt để hạn chế này, hệ thống LightRAG đã thu hẹp cấu hình phân đoạn tịnh tiến (Chunking), hạ kích thước mỗi mảnh (Chunk Size) từ 1200 tokens mặc định xuống mức 400–500 tokens, phối hợp với cửa sổ dịch

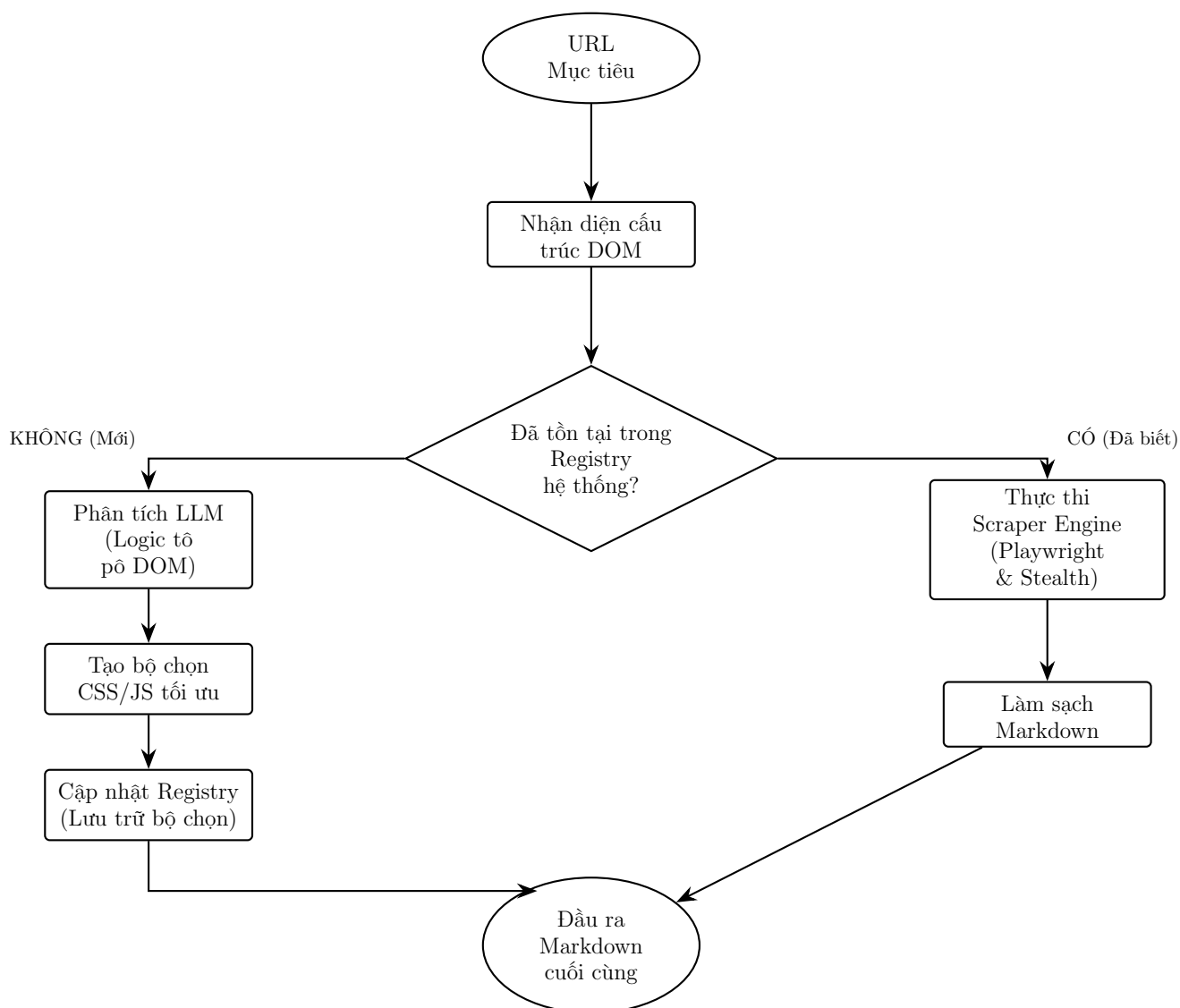
chuyển chồng lấp 50 tokens. Chiến lược này giúp bối cảnh văn bản lọt gọn vào giới hạn vật lý của mô hình biểu diễn mà vẫn bảo toàn ngữ nghĩa.

Về mặt kiến trúc vận hành thực tế hệ thống, mô hình nhúng mở multilingual-e5-large-instruct sở hữu dung lượng tham số ở mức trung bình (560 triệu tham số) [23], mở ra hai kịch bản triển khai linh hoạt đáp ứng hoàn hảo từng môi trường hạ tầng:

- Môi trường phát triển và thử nghiệm (Local Deployment): Mô hình cho phép tải trực tiếp về phần cứng cá nhân thông qua thư viện sentence-transformers và thực hiện xử lý vector hóa offline [23]. Cơ chế này mang lại tính độc lập và tính toàn vẹn cao (Self-contained) cho đồ án. Khi hội đồng hoặc giảng viên sao chép mã nguồn, hệ thống có thể biên dịch và chạy ngay lập tức mà không cần kết nối mạng bên ngoài, không tốn chi phí duy trì token và loại bỏ sự phụ thuộc vào API Key của bên thứ ba [23].

- Môi trường vận hành thực tế (Production Hosting API): Để đưa hệ thống ra thị trường phục vụ hàng ngàn sinh viên truy cập đồng thời, hệ thống chuyển sang mô hình Self-hosted API độc lập. Mô hình nhúng được đóng gói và triển khai thành một vi dịch vụ (Microservice) riêng biệt trên máy chủ đám mây (Cloud) có hỗ trợ GPU tăng tốc thông qua các framework inference chuyên dụng như TEI (Text Embeddings Inference) của Hugging Face hoặc Triton Inference Server của NVIDIA. Kiến trúc này kích hoạt tính năng xử lý hàng đợi động (Dynamic Batching) và lượng hóa ma trận về định dạng Float16, giúp nén tối đa độ trễ sinh vector nhúng thời gian thực xuống dưới 10 mili giây (ms), đáp ứng trọn vẹn áp lực tải diện rộng trong Production.

3.1.4 Hệ thống thu thập tri thức từ trang web



Hình 3.5: Quy trình thu thập tri thức từ trang web với Smart Scraper

Hệ thống cào web thông minh hoạt động theo quy trình tự động hóa từ việc nhận diện cấu trúc trang đến trích xuất dữ liệu. Logic cốt lõi dựa trên công cụ Smart Scraper Engine với cơ chế vận hành qua bốn bước chính: lấy dấu vân tay, đối chiếu kho lưu trữ, phân tích cấu trúc bằng mô hình ngôn ngữ và thực thi bộ phân giải.

Quy trình bắt đầu bằng việc tạo ra một bản tóm tắt cấu trúc cây DOM của trang web, tập trung vào các thuộc tính định danh như thẻ HTML,

tên lớp và ID. Hệ thống trích xuất một bộ khung xương kỹ thuật của trang web để làm cơ sở cho việc nhận diện. Sau đó, bản tóm tắt này được đối chiếu với kho lưu trữ các bộ phân giải đã biết. Mỗi mục trong kho lưu trữ chứa các thông tin về dấu hiệu nhận biết, bộ chọn nội dung chính, các thành phần cần loại bỏ và chiến lược chờ đợi phù hợp cho từng loại công nghệ web như React, Vue, hay các hệ quản trị nội dung truyền thống.

Trong trường hợp gặp một trang web mới không có trong kho lưu trữ, hệ thống sẽ gửi một phần cấu trúc DOM tiêu biểu cho mô hình ngôn ngữ lớn. Mô hình này có nhiệm vụ phân tích logic phân bổ nội dung và tự động sinh ra mã lệnh điều hướng. Mã lệnh này bao gồm các bộ chọn CSS tối ưu để trích xuất văn bản có ý nghĩa và các quy tắc loại bỏ những thành phần gây nhiễu như trình đơn điều hướng, biểu ngữ quảng cáo hoặc các đoạn mã kịch bản. Bộ phân giải mới sinh ra sẽ được kiểm tra và lưu lại vào kho lưu trữ để sử dụng cho các lần truy cập sau, giúp tối ưu hóa hiệu suất và giảm thiểu chi phí gọi API.

Đối với các tên miền mặc định như `fit.hcmus.edu.vn` và `hcmus.edu.vn`, hệ thống đã được cấu hình sẵn các bộ phân giải chuyên biệt cho nền tảng DotNetNuke. Do đặc thù của nền tảng này thường chứa nhiều nội dung lồng ghép trong các khung phức tạp, hệ thống sử dụng các bộ chọn mục tiêu như `.dnn_contentPane` và `#dnn_ContentPane` để định vị chính xác vùng chứa thông tin học thuật. Quy trình trích xuất văn bản tại đây không chỉ lấy chữ thuần túy mà còn thực hiện chuyển đổi các liên kết quan trọng sang định dạng Markdown để bảo toàn thông tin tham chiếu.

Đặc biệt, đối với các trang có tính tương tác cao như danh mục môn học hoặc đề cương chi tiết, hệ thống áp dụng chế độ tương tác vét cạn. Chế độ này sử dụng thư viện Playwright để tự động thực hiện các hành động như nhấn vào các nút mở rộng, cuộn trang để tải thêm dữ liệu và chờ đợi các tiến trình AJAX hoàn tất. Điều này đảm bảo rằng toàn bộ tri thức về chương trình đào tạo, mô tả môn học và thông tin giảng viên được thu thập đầy đủ nhất, ngay cả khi chúng nằm sau các thành phần giao diện động. Dữ liệu sau khi trích xuất sẽ được làm sạch và chuẩn hóa trước

khi đưa vào các bước phân đoạn và xây dựng đồ thị tri thức tiếp theo.

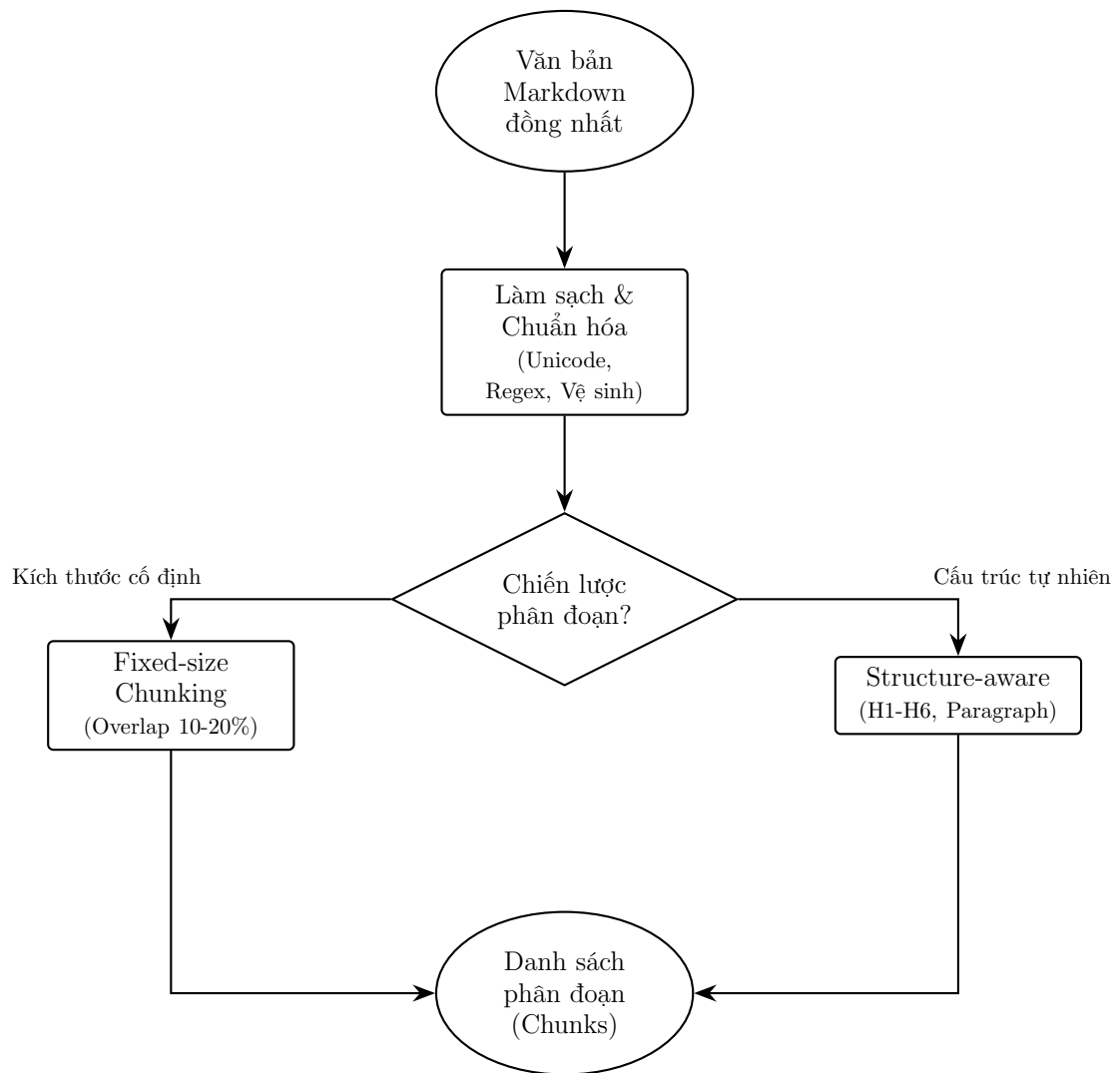
3.1.5 Quy trình điều phối và phân đoạn dữ liệu

Sau khi dữ liệu vượt qua bước kiểm tra trùng lặp, hệ thống bước vào giai đoạn điều phối và phân đoạn. Đây là quy trình then chốt nhằm chuyển đổi các văn bản Markdown dài thành những đơn vị tri thức có kích thước tối ưu, sẵn sàng cho việc trích xuất thực thể và quan hệ. Quy trình điều phối này không chỉ đơn thuần là cắt nhỏ văn bản mà còn đảm bảo tính logic và ngữ cảnh giữa các phần.

LightRAG [9] thực hiện quy trình này thông qua hai chiến lược phân đoạn chính để bảo toàn tính toàn vẹn tri thức:

Phương pháp thứ nhất, phân đoạn theo kích thước cố định (fixed-size chunking), chia văn bản thành các đoạn có độ dài xác định trước (thường từ 512 đến 1200 token), kèm theo một vùng chồng lấp (overlap window) chiếm khoảng 10–20% kích thước đoạn giữa hai đoạn liên tiếp. Cơ chế chồng lấp này đóng vai trò như một “cầu nối ngữ cảnh”: khi mô hình xử lý đoạn thứ i , nó đã có sẵn phần đuôi của đoạn thứ $i-1$ làm nền tảng, từ đó tránh được hiện tượng cắt đứt giữa chừng một câu hoặc một luận điểm quan trọng.

Phương pháp thứ hai, phân đoạn theo cấu trúc tự nhiên (structure-aware chunking), khai thác các dấu hiệu cấu trúc sẵn có trong tài liệu như tiêu đề, tiêu đề phụ, danh sách có đánh số, hay các đoạn văn được phân tách bởi ký tự xuống dòng kép. Hệ thống sử dụng các biểu thức chính quy (regex) kết hợp với phân tích cú pháp nhẹ (lightweight parsing) để nhận diện ranh giới tự nhiên này. Ưu điểm của phương pháp này là mỗi đoạn văn giữ nguyên tính nguyên vẹn ngữ nghĩa — ví dụ, toàn bộ một mục trong tài liệu kỹ thuật sẽ không bị tách ra giữa chừng.



Hình 3.6: Luồng logic xử lý từ dữ liệu thô đến giai đoạn phân đoạn

3.1.6 Cơ chế trích xuất thực thể và xây dựng quan hệ

Sau khi văn bản được chia thành các phân đoạn (chunk), LightRAG thực hiện quy trình trích xuất tri thức để chuyển đổi dữ liệu phi cấu trúc thành các thành phần đồ thị tri thức tường minh. Khác với các hệ thống RAG truyền thống thường bỏ qua mối liên kết giữa các đoạn văn, cơ chế này tập trung vào việc nhận diện các thực thể (Entities) và thiết lập các quan hệ (Relationships) xuyên suốt tài liệu.

3.1.6.1 Chiến lược trích xuất đa nhiệm trong một lần gọi (One-step Extraction)

Để tối ưu hóa chi phí tính toán và duy trì tính nhất quán ngữ nghĩa, LightRAG sử dụng một lời nhắc có cấu trúc (structured prompt) yêu cầu mô hình ngôn ngữ lớn (LLM) thực hiện đồng thời hai nhiệm vụ trích xuất trong một lần gọi API duy nhất.

Nhiệm vụ 1: Nhận diện thực thể (Entity Extraction). Mô hình được yêu cầu xác định tất cả các thực thể quan trọng xuất hiện trong chunk. Đối với mỗi thực thể, LLM phải trích xuất các thuộc tính cốt lõi bao gồm tên định danh (Entity Name), loại thực thể (Entity Type) như PERSON, ORGANIZATION, CONCEPT, EVENT,... và một đoạn văn bản ngắn mô tả (Entity Description) giải thích vai trò, đặc điểm và ngữ cảnh của thực thể đó. Khác với NER truyền thống [18], phương pháp này khai thác tối đa khả năng hiểu ngữ nghĩa sâu của LLM để tạo ra các mô tả tri thức phong phú.

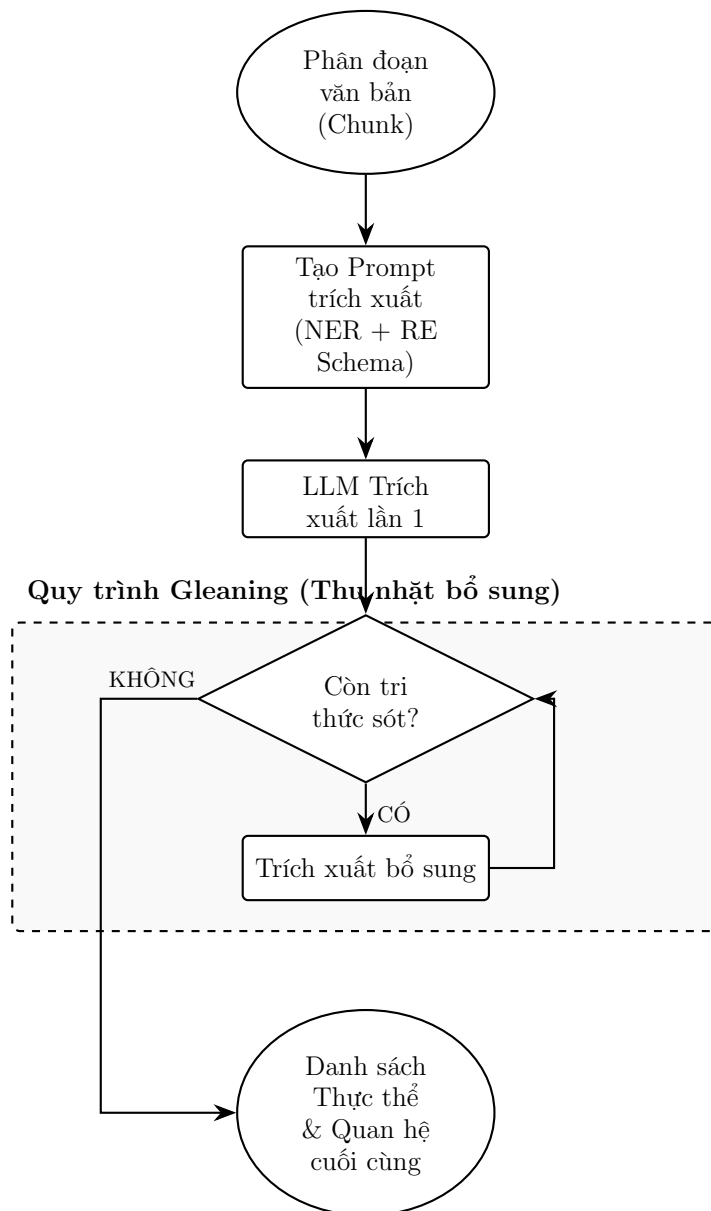
Nhiệm vụ 2: Trích xuất quan hệ (Relation Extraction). Sau khi xác định các thực thể, mô hình thực hiện quét các cặp thực thể để tìm kiếm các mối liên kết có ý nghĩa. Các thuộc tính trích xuất bao gồm hai thực thể tham gia (Source & Target), các từ khóa định nghĩa bản chất liên kết (Relationship Keywords) và mô tả chi tiết (Relationship Description) lý do tại sao hai thực thể này có liên quan đến nhau dựa trên bằng chứng trong

văn bản.

3.1.6.2 Cơ chế Gleaning: Tự chỉnh sửa và Thu nhặt bổ sung

Một trong những thách thức lớn nhất của việc trích xuất tri thức bằng LLM là hiện tượng bỏ sót thông tin do giới hạn về khả năng chú ý (attention) khi xử lý các đoạn văn phức tạp. LightRAG giải quyết vấn đề này thông qua kỹ thuật *Gleaning* (thu nhặt bổ sung).

Sau vòng trích xuất đầu tiên, hệ thống không kết thúc ngay mà thực hiện gửi lại danh sách tri thức vừa tìm được cho LLM kèm theo một yêu cầu phản biện: “Dựa trên đoạn văn này và danh sách thực thể/quan hệ đã có, liệu còn bất kỳ chi tiết nào bị bỏ sót không?”. Nếu mô hình phát hiện thêm tri thức mới, nó sẽ trả về kết quả bổ sung để hệ thống hợp nhất vào kết quả ban đầu. Quy trình này lặp lại cho đến khi mô hình xác nhận không còn thông tin sót hoặc đạt giới hạn số vòng lặp cấu hình.



Hình 3.7: Luồng logic quy trình trích xuất tri thức và cơ chế Gleaning

Kết quả cuối cùng là một tập hợp các thực thể và quan hệ có cấu trúc, sẵn sàng cho bước hợp nhất đồ thị (Graph Merging). Cơ chế này đảm bảo rằng mỗi đoạn văn bản đều được khai thác triệt để tri thức, tạo tiền đề cho việc xây dựng một đồ thị tri thức dày đặc và chính xác.

3.1.7 Luồng xử lý Prompt linh hoạt cho đa dạng nguồn dữ liệu

Một thách thức lớn trong việc xây dựng đồ thị tri thức cho môi trường học thuật là sự khác biệt sâu sắc về cấu trúc và ngôn từ giữa các nguồn tài liệu. Đề cương chi tiết môn học thường mang tính quy phạm, ngắn gọn và sử dụng dày đặc các mã ký hiệu viết tắt như G1, G2 (mục tiêu môn học), CLO (chuẩn đầu ra), hay các thuật ngữ quản lý đào tạo như tích lũy, tiên quyết. Trong khi đó, dữ liệu từ hệ thống website của khoa lại mang tính chất thông tin đại chúng, chứa nhiều tin tức, sự kiện và mô tả tổng quát. Một lời nhắc trích xuất tổng quát duy nhất khó có thể bao quát tốt cả hai dạng tài liệu này: lời nhắc tối ưu cho đề cương sẽ bỏ sót ngữ cảnh tin tức, còn lời nhắc tối ưu cho website lại không nhận diện được các mã ký hiệu chuyên ngành. Để giải quyết vấn đề này, hệ thống triển khai mô-đun luồng xử lý lời nhắc linh hoạt (Dynamic Prompt Flow) đặt tại `lightrag/prompt_flow`, với cơ chế tự động điều phối lời nhắc theo miền tri thức (domain) của từng phân đoạn.

Về mặt thiết kế, mô-đun áp dụng đồng thời ba mẫu thiết kế phần mềm: *Registry*, *Strategy* và *Protocol*. Mỗi miền tri thức được hiện thực thành một mô-đun chiến lược (strategy) tuân thủ giao diện `EntityExtractionModule`, bắt buộc cung cấp đúng bốn hằng số lời nhắc: lời nhắc hệ thống (`ENTITY_EXTRACTION_SYSTEM_PROMPT`), tập ví dụ mẫu (`ENTITY_EXTRACTION_EXAMPLES`), lời nhắc người dùng (`ENTITY_EXTRACTION_USER_PROMPT`) và lời nhắc thu nhật bổ sung (`ENTITY_CONTINUE_EXTRACTION_USER_PROMPT`). Bộ điều phối (`PromptDispatcher`) duy trì một sổ đăng ký (`Registry`) ánh xạ tên miền sang mô-đun tương ứng; khi gặp một nhãn miền chưa được đăng ký, hệ thống tự động lùi về miền mặc định `curriculum`. Thiết kế này cho phép bổ sung một miền tri thức mới mà không phải sửa đổi mã nguồn của tầng trích xuất, đảm bảo khả năng mở rộng.

Hệ thống hiện hỗ trợ bốn miền tri thức đặc thù cho dữ liệu đào tạo

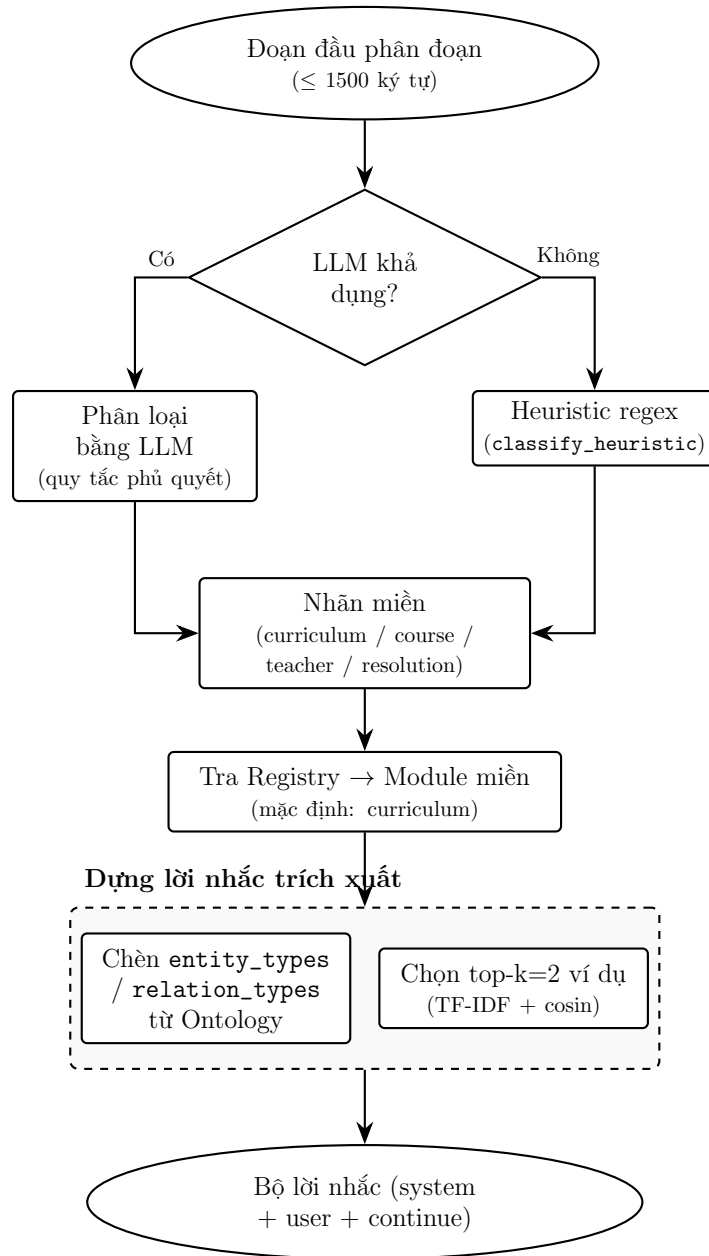
bậc đại học. Miền **curriculum** dành cho chương trình đào tạo, tập trung vào ngành, khoa, khối kiến thức, môn học, mục tiêu chương trình (PO), chuẩn đầu ra (ELO), số tín chỉ và điều kiện tiên quyết. Miền **course** dành cho đề cương chi tiết học phần, ưu tiên các chuẩn đầu ra môn học (CLO), nội dung chương, hình thức kiểm tra và tài liệu tham khảo. Miền **teacher** dành cho danh sách nhân sự giảng dạy, phân biệt giảng viên cơ hữu, thỉnh giảng và cán bộ hỗ trợ cùng học hàm, học vị. Cuối cùng, miền **resolution** dành cho các văn bản pháp lý, hành chính thuần túy như nghị quyết hội đồng trường hay quy chế tổ chức.

Việc xác định miền của một phân đoạn được thực hiện theo cơ chế hai tầng nhằm cân bằng giữa độ chính xác và khả năng chịu lỗi. Tầng thứ nhất sử dụng một mô hình ngôn ngữ lớn đóng vai trò bộ phân loại nhẹ (lightweight classifier): hệ thống chỉ gửi khoảng 1500 ký tự đầu của phân đoạn kèm theo một lời nhắc phân loại ngắn gọn, yêu cầu mô hình trả về đúng một nhãn miền. Do các văn bản giáo dục Việt Nam thường mở đầu bằng phần thủ tục hành chính (“QUYẾT ĐỊNH V/v Ban hành...”, “Căn cứ...”), lời nhắc phân loại được bổ sung một quy tắc phủ quyết (critical override): nếu phần mở đầu hành chính này thực chất nhằm ban hành chương trình đào tạo hoặc đề cương môn học, văn bản phải được phân vào **curriculum** hoặc **course** thay vì bị nhầm thành **resolution**. Tầng thứ hai là một bộ phân loại dựa trên biểu thức chính quy và tính điểm theo từ khóa đặc trưng (**classify_heuristic**), đóng vai trò phương án dự phòng khi mô hình ngôn ngữ không khả dụng hoặc trả về kết quả không hợp lệ.

Sau khi xác định được miền, bộ điều phối nạp các kiểu thực thể và kiểu quan hệ đặc thù từ lớp bản thể học (ontology) của miền đó vào các vị trí giữ chỗ **{entity_types}** và **{relation_types}** trong lời nhắc. Ví dụ, miền **curriculum** định nghĩa các kiểu thực thể như **Major**, **KnowledgeBlock**, **Course**, **LearningOutcome** (ELO), **ProgramObjective** (PO) cùng các kiểu quan hệ như **PREREQUISITE_OF** (học trước), **CONTAINS** (khối kiến thức chứa môn học) hay **MAPS_TO** (ELO liên kết với PO); khi miền không khai báo bản thể học, hệ thống dùng quan hệ tổng quát **RELATES_TO** làm mặc định.

Việc áp đặt sẵn bộ kiểu chuyên biệt giúp định hướng mô hình ngôn ngữ tập trung vào tri thức có giá trị nhất của miền và duy trì tính nhất quán về định danh (Entity Canonicalization) ngay từ bước sơ khởi, từ đó giảm thiểu số lượng thực thể phân mảnh và tạo điều kiện cho việc hợp nhất các nút đồng nghĩa ở các bước sau. Cần lưu ý rằng ở giai đoạn hiện tại, việc đối chiếu kết quả trích xuất với bản thể học chỉ dừng ở mức ghi cảnh báo (warn-only), còn cơ chế loại bỏ tự động dữ liệu vi phạm được để ngỏ như một điểm mở rộng trong tương lai.

Để kiểm soát chi phí token, mô-đun không chèn toàn bộ tập ví dụ mẫu vào mỗi lần gọi mà áp dụng cơ chế chọn ví dụ động (dynamic few-shot selection). Một bộ chỉ mục TF-IDF kết hợp độ tương đồng cosin được xây dựng thuần Python (không phụ thuộc thư viện ngoài) để xếp hạng các ví dụ theo mức độ liên quan với nội dung phân đoạn hiện tại, sau đó chỉ giữ lại hai ví dụ ($\text{top-k} = 2$) phù hợp nhất. Cách làm này giảm đáng kể lượng token tiêu thụ so với việc nhúng toàn bộ ví dụ, đồng thời vẫn bảo toàn chất lượng trích xuất nhờ luôn cung cấp đúng ngữ cảnh mẫu sát với phân đoạn. Trong trường hợp việc xếp hạng gặp lỗi, hệ thống tự động lùi về phương án trả về các ví dụ đầu danh sách, bảo đảm quy trình không bị gián đoạn. Hình 3.8 mô tả tổng quan luồng điều phối lời nhắc từ phân đoạn đầu vào đến bộ lời nhắc trích xuất hoàn chỉnh.



Hình 3.8: Luồng điều phối lời nhắc linh hoạt theo miền tri thức (Dynamic Prompt Flow)

3.1.8 Kiểm chứng tri thức bằng chuẩn SHACL và cơ chế Staging

Để đảm bảo tính toàn vẹn và chất lượng của dữ liệu tri thức trước khi đưa vào đồ thị chính thức, hệ thống tích hợp một bộ kiểm chứng lấy cảm

hứng từ chuẩn SHACL (Shapes Constraint Language) [22]. SHACL cung cấp một ngôn ngữ để mô tả các ràng buộc về cấu trúc và ngữ nghĩa của dữ liệu đồ thị, cho phép tự động hóa việc phát hiện các lỗi phổ biến trong kết quả trích xuất bằng mô hình ngôn ngữ lớn [14]. Toàn bộ logic kiểm chứng được hiện thực trong mô-đun `lightrag/shacl`, tổ chức thành một quy trình ba lớp bảo vệ (3-Guard) thực thi tuần tự trên kết quả trích xuất của từng phân đoạn.

Lớp bảo vệ thứ nhất (Guard 1) – Khử trùng lặp. Hệ thống đối chiếu mã định danh phân đoạn (`chunk_id`) với kho lưu trữ hiện có. Nếu phân đoạn đã tồn tại, hệ thống chuyển sang trạng thái nạp lại (`RE_INGEST`) và điều chỉnh cách tính bậc kết nối của thực thể để tránh cộng dồn trùng lặp; ngược lại, phân đoạn mới được cho phép tiếp tục (`PROCEED`). Lớp này ngăn việc xử lý lại dữ liệu dư thừa và bảo vệ tính toàn vẹn của đồ thị.

Lớp bảo vệ thứ hai (Guard 2) – Kiểm tra cấu trúc và đối chiếu bằng chứng. Đây là trọng tâm của quy trình. Mỗi thực thể và quan hệ được kiểm tra qua một tập các định nghĩa hình dạng (Shapes). Đối với thực thể, các quy tắc bao gồm: bắt buộc có tên hợp lệ (`missing_name`, mức lỗi), bắt buộc có kiểu xác định khác “UNKNOWN” (`missing_type`, mức lỗi), bắt buộc có siêu dữ liệu nguồn gốc `source_id` (`no_provenance`, mức lỗi), và cảnh báo thực thể bị cô lập khi số kết nối nhỏ hơn ngưỡng tối thiểu (`isolated_entity`). Đáng chú ý, số kết nối của một thực thể được tính bằng cách kết hợp bậc cục bộ (trong phạm vi phân đoạn) với bậc toàn cục truy vấn đồng thời từ cơ sở dữ liệu đồ thị Neo4j, nhằm tránh báo cô lập sai đối với những thực thể đã có liên kết ở các phân đoạn trước. Đối với quan hệ, hệ thống kiểm tra sự tồn tại của thực thể nguồn và đích, phát hiện quan hệ tự trở (`self_loop`), độ dài tối thiểu của mô tả (mặc định 15 ký tự) và dải giá trị hợp lệ của trọng số trong khoảng $[0, 1]$. Ngoài ra, một bộ chặn ngôn ngữ (language guard) sẽ từ chối mọi giá trị chứa ký tự nằm ngoài bảng chữ Latinh và tiếng Việt (`contains_foreign_script`), loại bỏ các trường hợp mô hình ngôn ngữ vô tình sinh ra ký tự ngoại lai. Mỗi vi phạm được gán một trong ba mức nghiêm trọng: lỗi (error), cảnh báo

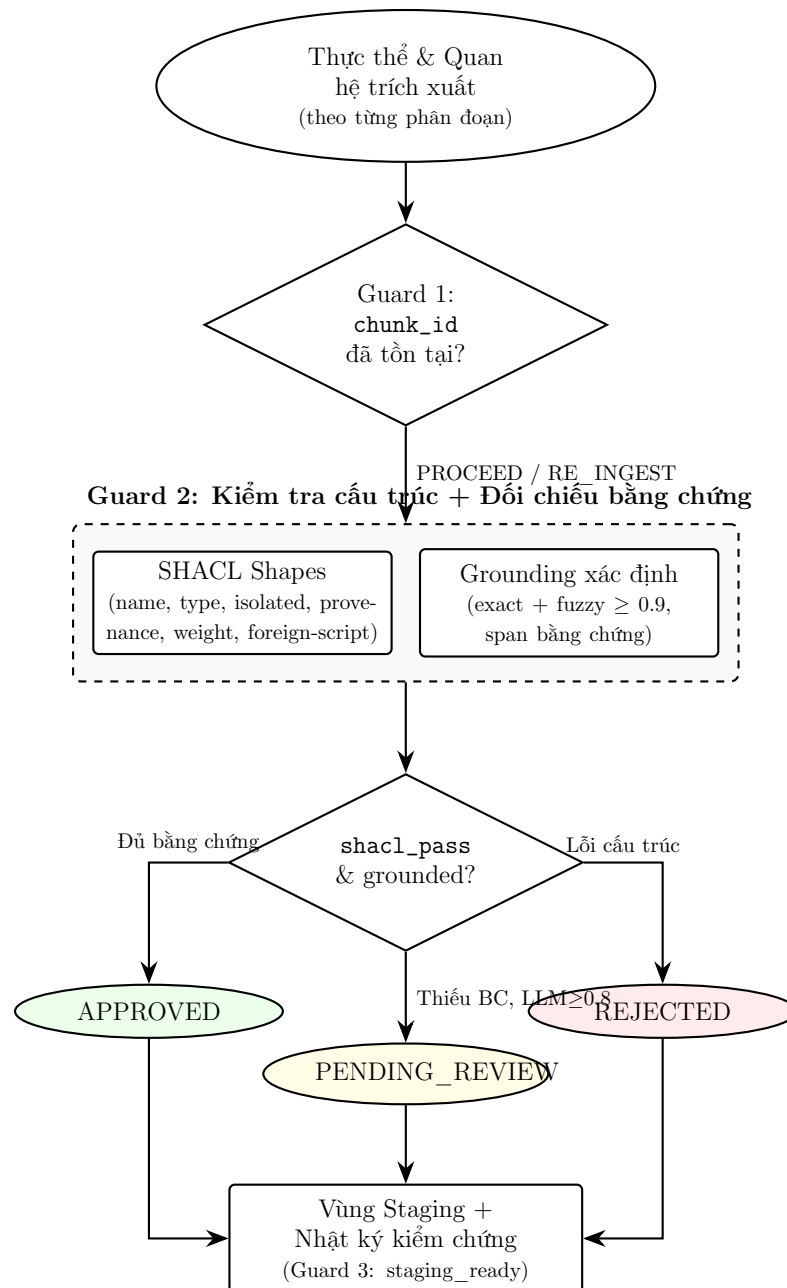
(warning) hoặc thông tin (info).

Điểm khác biệt cốt lõi của bộ kiểm chứng là cơ chế đối chiếu bằng chứng văn bản một cách xác định (deterministic text-grounding). Thay vì hỏi mô hình ngôn ngữ “thực thể này có trong văn bản không?” rồi tin vào câu trả lời, hệ thống tự tính toán bằng chứng: trước tiên tìm khớp chính xác chuỗi con (điểm số 1.0), nếu không có thì dùng so khớp mờ (fuzzy matching qua `rapidfuzz` hoặc `difflib`) với ngưỡng chấp nhận 0.9. Kết quả trả về không phải một ý kiến mà là chính đoạn văn bản khớp được kèm theo vị trí ký tự trong phân đoạn – đóng vai trò bằng chứng có thể kiểm tra lại. Đối với quan hệ, hệ thống yêu cầu cả hai đầu mút đều có bằng chứng và đồng xuất hiện trong cùng một cửa sổ 200 ký tự, qua đó củng cố độ tin cậy rằng quan hệ là có thật chứ không phải hai lần nhắc rời rạc.

Quyết định cuối cùng cho mỗi thực thể và quan hệ được tổng hợp tại một tầng quyết định thuần túy theo nguyên tắc phân cấp niềm tin: các kiểm tra xác định (cấu trúc và bằng chứng văn bản) giữ quyền quyết định, còn mô hình ngôn ngữ phản biện (critic LLM) chỉ được phép nâng mức xem xét chứ không được tự ý chấp nhận. Cụ thể, nếu vi phạm cấu trúc ở mức lỗi thì dữ liệu bị đề xuất từ chối (**REJECTED**); nếu hợp lệ về cấu trúc và có bằng chứng văn bản thì được phê duyệt (**APPROVED**); nếu hợp lệ về cấu trúc nhưng thiếu bằng chứng xác định, dữ liệu chỉ được đưa vào danh sách chờ xem xét (**PENDING_REVIEW**) khi độ tin cậy của mô hình phản biện đạt ngưỡng 0.8, ngược lại vẫn bị từ chối. Các thực thể bị cô lập hoặc quan hệ tự trở cũng được tự động chuyển sang trạng thái chờ xem xét.

Kết quả kiểm chứng trực tiếp điều hướng dữ liệu vào vùng đệm Staging thay vì hợp nhất ngay vào đồ thị chính thức. Mỗi mục dữ liệu được lưu kèm danh sách vi phạm (`shacl_violations`), cờ hợp lệ (`shacl_pass`), trạng thái đề xuất (`status_suggestion`) và bằng chứng đối chiếu (`grounding`). Lớp bảo vệ thứ ba (Guard 3) tổng hợp báo cáo trên toàn bộ phân đoạn và chỉ đánh dấu sẵn sàng nạp vào đồ thị (`guard3_staging_ready`) khi không còn bất kỳ mục nào bị từ chối. Hình 3.9 minh họa toàn bộ quy trình ba lớp

bảo vệ. Sự kết hợp giữa kiểm chứng theo Shapes, đối chiếu bằng chứng xác định và cơ chế Staging tạo nên một quy trình kiểm soát chất lượng chặt chẽ, vừa tự động phát hiện sai sót của mô hình ngôn ngữ, vừa cung cấp nhật ký minh bạch để người quản trị can thiệp, chỉnh sửa và phê duyệt, qua đó bảo đảm đồ thị tri thức cuối cùng luôn nhất quán và đáng tin cậy.



Hình 3.9: Quy trình kiểm chứng tri thức ba lớp bảo vệ (3-Guard) dựa trên SHACL và đối chiếu bằng chứng

3.1.9 Xử lý thực thể trùng lặp trong văn bản học thuật

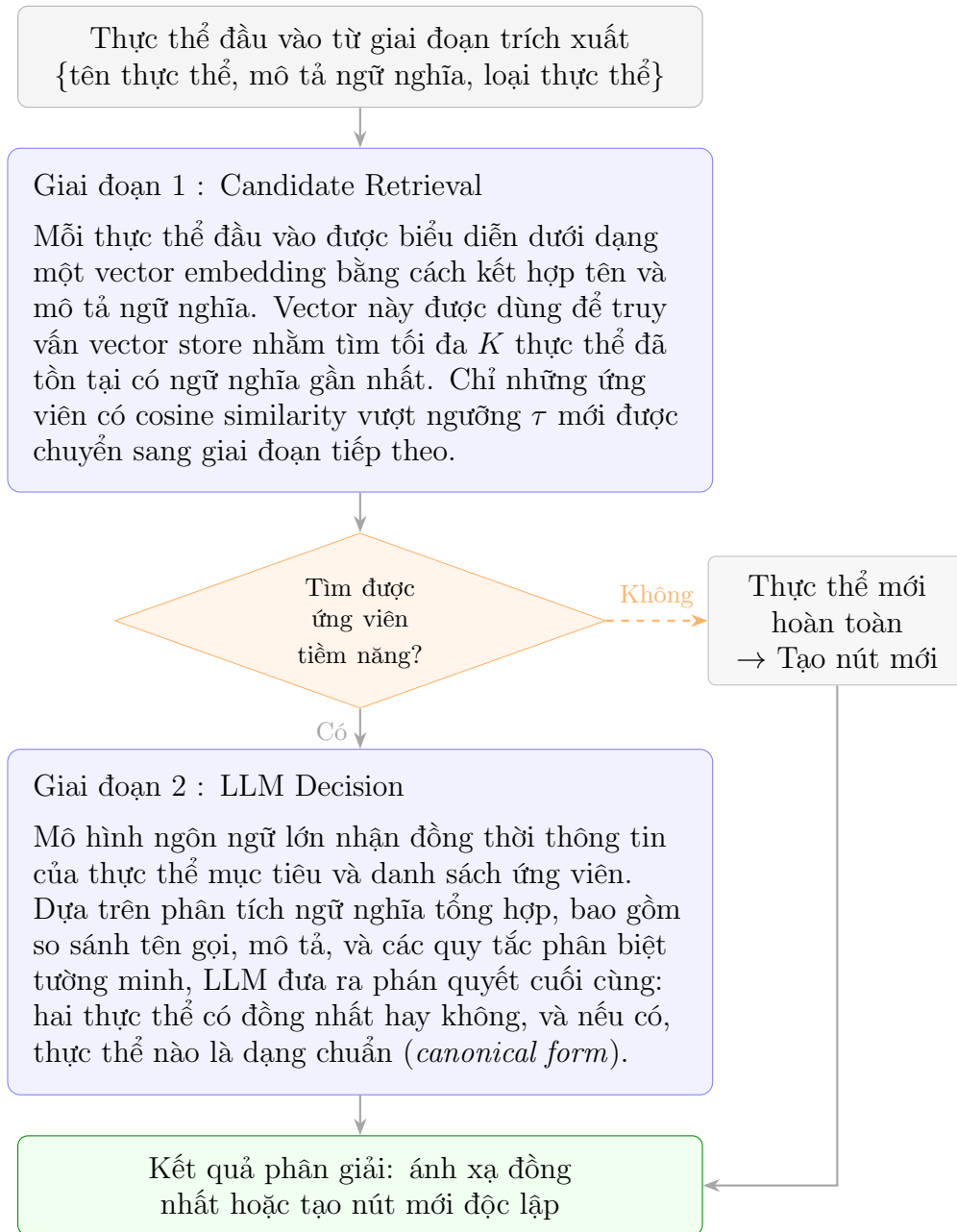
Để xử lý vấn đề thực thể trùng lặp đảm bảo truy xuất đầy đủ thông tin, đề tài đề xuất một module phân giải thực thể (*entity resolution*) được tích hợp vào hai giai đoạn, hoạt động trước khi bất kỳ nút nào được ghi vào đồ thị. Module này được thiết kế theo kiến trúc hai giai đoạn xử lý nối tiếp: Candidate Retrieval (Truy hồi ứng viên) và LLM Decision (LLM đưa ra quyết định).

Cách tiếp cận này được xây dựng trên nền tảng của hai công trình nghiên cứu trong lĩnh vực căn chỉnh thực thể (*entity alignment*) giữa các knowledge graph:

HELEA **helea2024** chứng minh hiệu quả của pipeline hai giai đoạn: đầu tiên truy hồi ứng viên bằng embedding similarity, sau đó dùng mô hình ngôn ngữ lớn để xếp hạng lại (*reranking*). Cơ chế này vừa đảm bảo không bỏ sót ứng viên tiềm năng, vừa tận dụng khả năng suy luận ngữ nghĩa sâu của LLM.

EasyEA **easyea2024** chứng minh rằng mô hình ngôn ngữ lớn, khi được cung cấp đủ ngữ cảnh, có khả năng đưa ra quyết định căn chỉnh thực thể chính xác theo phương thức *zero-shot* không cần dữ liệu huấn luyện nhân, phù hợp với bối cảnh corpus tiếng Việt học thuật hiện tại.

Hình 3.10 Trình bày kiến trúc tổng thể của nhất đồ thị.



Hình 3.10: Kiến trúc hai giai đoạn của module phân giải thực thể. Giai đoạn Candidate Retrieval sử dụng embedding similarity để truy hồi các thực thể ngữ nghĩa tương đồng từ vector store. Giai đoạn LLM Decision chỉ được kích hoạt khi có ứng viên vượt ngưỡng τ , giúp tiết kiệm chi phí gọi LLM cho những thực thể hoàn toàn mới.

Giai đoạn 1: Candidate Retrieval. Mỗi thực thể cần phân giải được biểu diễn dưới dạng một vector embedding bằng cách kết hợp cả tên lẫn mô tả của thực thể đó thành một chuỗi đầu vào thống nhất. Cách biểu

diễn này cho phép vector nắm bắt đồng thời hai chiều thông tin: chiều định danh (tên gọi) và chiều ngữ nghĩa (nội dung, thuộc tính). Sau đó, hệ thống thực hiện tìm kiếm láng giềng gần nhất trong vector store để truy hồi tối đa K thực thể đã tồn tại trong knowledge graph có vector gần nhất với thực thể đang xét. Ngưỡng tương đồng cosine τ đóng vai trò bộ lọc: chỉ những ứng viên có $\cos(\mathbf{v}_1, \mathbf{v}_2) \geq \tau$ mới được coi là *ứng viên tiềm năng* và đưa vào giai đoạn tiếp theo. Cơ chế này tương đồng với giai đoạn truy hồi ứng viên trong kiến trúc HELEA **helea2024**, đảm bảo không bỏ sót các cặp thực thể có ngữ nghĩa gần nhau.

Giai đoạn 2: LLM Decision. Khi giai đoạn Candidate Retrieval xác định được ít nhất một ứng viên tiềm năng, mô hình ngôn ngữ lớn được kích hoạt để đưa ra phán quyết cuối cùng. Không giống với các phương pháp truyền thống chỉ dựa vào điểm số cosine similarity, LLM có khả năng suy luận ở mức ngữ nghĩa sâu hơn, cũng như phân biệt các thực thể bên ngoài tương tự nhưng thực chất khác biệt. Chẳng hạn, mô hình xác định “*Nhập môn Lập trình*” và “*Lập trình Hướng đối tượng*” là hai học phần khác nhau mặc dù đều thuộc nhóm lập trình, trong khi “*HCMUS*” và “*Trường Đại học Khoa học Tự nhiên*” là cùng một tổ chức.

3.1.9.1 Thiết kế quan hệ SAME_AS: Liên kết thay vì xóa bỏ

Khi mô hình ngôn ngữ lớn xác nhận hai thực thể là đồng nhất, hệ thống có 2 lựa chọn: *hợp nhất vật lý* hai nút thành một (xóa một nút, gộp toàn bộ dữ liệu vào nút còn lại), hay *tạo quan hệ liên kết* giữa chúng và giữ nguyên cả hai? Đề tài lựa chọn giải pháp thứ hai thông qua quan hệ SAME_AS.

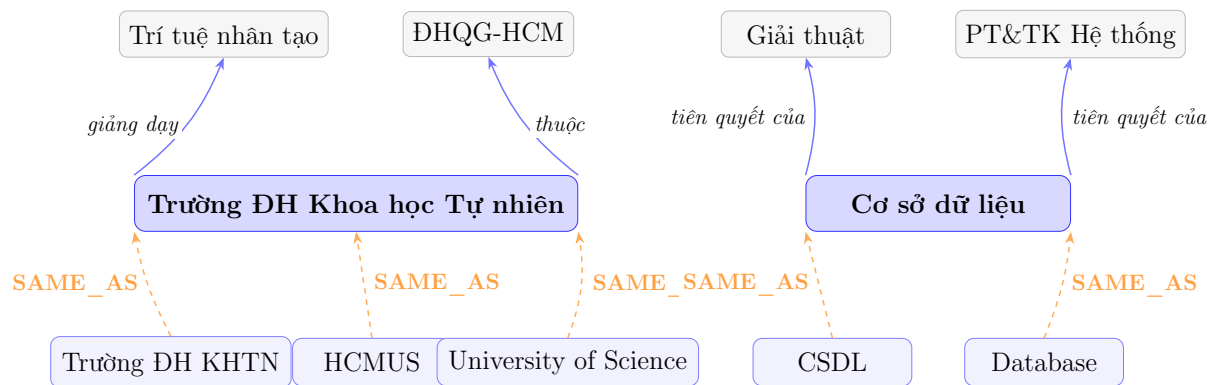
Quyết định này không phải là sự thỏa hiệp mà xuất phát từ ba lý do cơ bản. *Thứ nhất*, cả HELEA lẫn EasyEA đều tiếp cận entity alignment theo hướng tạo ra *tập hợp cặp đồng nhất* giữa hai knowledge graph độc lập không ai trong số họ đề xuất xóa bỏ hoặc hợp nhất vật lý các nút. Quan hệ SAME_AS là biểu diễn ngữ nghĩa chuẩn của kết quả alignment, tương đương với thuộc tính owl:sameAs trong các chuẩn ngữ nghĩa web (*Semantic*

Web) **owlsemweb2004**. Thứ hai, hợp nhất vật lý là thao tác không thể hoàn tác: nếu mô hình ngôn ngữ lớn đưa ra quyết định nhầm, dữ liệu của nút bị xóa không thể phục hồi. Ngược lại, một quan hệ SAME_AS có thể được xóa bỏ đơn giản nếu phát hiện sai sót, trong khi toàn bộ thông tin gốc vẫn được bảo toàn. Thứ ba, thiết kế SAME_AS bảo toàn *nguồn gốc ngữ nghĩa (provenance)*: mỗi nút vẫn duy trì liên kết đến đúng các tài liệu đã sử dụng hình thức tên đó, phục vụ cho việc truy vết và kiểm chứng thông tin sau này.

Về mặt hành vi trong knowledge graph, thiết kế này cho phép hệ thống truy vấn tổng hợp thông tin từ cả hai nút thông qua phép duyệt cạnh SAME_AS. Ví dụ, khi người dùng hỏi về “HCMUS”, hệ thống không chỉ truy xuất thông tin gắn với nút “HCMUS” mà còn có thể traverse sang nút “Trường Đại học Khoa học Tự nhiên” để bổ sung thêm tri thức liên quan, cho ra câu trả lời toàn diện hơn.

3.1.9.2 Kết quả trong knowledge graph

Minh họa cấu trúc của knowledge graph sau khi áp dụng module phân giải thực thể với dữ liệu từ corpus học thuật.



Hình 3.11: Ví dụ cấu trúc knowledge graph sau khi áp dụng phân giải thực thể trên corpus học thuật HCMUS. Các nút màu đậm là thực thể dạng chuẩn (*canonical*); các nút màu nhạt là hình thức tên thay thế (*alias*) được liên kết về nút chuẩn qua quan hệ SAME_AS (đường nét đứt cam). Quan hệ ngữ nghĩa thông thường (mũi tên xanh) chỉ xuất phát từ nút chuẩn, đảm bảo tri thức không bị phân tán.

3.1.9.3 Đồng bộ hóa với vector store

Song song với việc xây dựng đồ thị quan hệ, hệ thống duy trì một cơ sở dữ liệu vector (*vector store*) được đồng bộ liên tục. Sau mỗi thao tác hợp nhất, toàn bộ thực thể và quan hệ - với mô tả đã được tổng hợp - đều được cập nhật vào vector store. Kiến trúc lưu trữ kép này (*dual storage*) là nền tảng cho hai chế độ truy vấn của LightRAG: đồ thị tri thức phục vụ duyệt cấu trúc quan hệ tường minh, trong khi vector store cho phép tìm kiếm ngữ nghĩa linh hoạt khi câu hỏi của người dùng không khớp chính xác với bất kỳ tên thực thể nào đã được lập chỉ mục.

3.1.9.4 So sánh với công trình liên quan

Bảng 3.2 tổng hợp mức độ tương đồng về mặt thiết kế giữa module phân giải thực thể của đề tài và hai công trình nghiên cứu tham chiếu.

Bảng 3.2: So sánh các thành phần thiết kế với công trình liên quan

Thành phần	HELEA	EasyEA	Phương án đề tài
Embedding kết hợp tên & mô tả	✓	✓	Áp dụng. Biểu diễn thực thể qua vector kết hợp hai chiều ngữ nghĩa.
Truy hồi Top- K qua vector similarity	✓	✓	Áp dụng với lọc ngưỡng cosine τ .
LLM ra quyết định cuối cùng	✓	✓	Áp dụng theo zero-shot.
Quan hệ SAME_AS (thay vì merge vật lý)	✓	✓	Áp dụng. Bảo toàn cả hai nút và provenance.
Ngữ cảnh triple láng giềng trong prompt	—	✓	Chưa tích hợp. Xác định là hướng phát triển tiếp theo.
Fine-tuning encoder với hard negatives	✓	—	Không áp dụng (chọn hướng zero-shot).

Hai điểm khác biệt chính so với các công trình tham chiếu cần được làm rõ. Đối với việc không fine-tuning encoder, đây là quyết định có chủ ý: corpus tiếng Việt trong lĩnh vực học thuật gần như không có tập dữ liệu căn chỉnh thực thể được gán nhãn, khiến việc huấn luyện không khả thi. Phương án zero-shot mà EasyEA kiểm chứng giải quyết đúng hạn chế này. Đối với ngữ cảnh triple láng giềng, EasyEA chỉ ra rằng việc bổ sung các bộ ba quan hệ liên quan của ứng viên vào prompt giúp LLM phân biệt tốt hơn các trường hợp thực thể mơ hồ - ví dụ, một thực thể là *học phần* và một thực thể là *phần mềm* có thể cùng mang tên “Cơ sở dữ liệu” nhưng có bộ quan hệ hoàn toàn khác nhau trong knowledge graph. Việc tích hợp thành phần này được xác định là hướng phát triển ưu tiên trong giai đoạn tiếp theo.

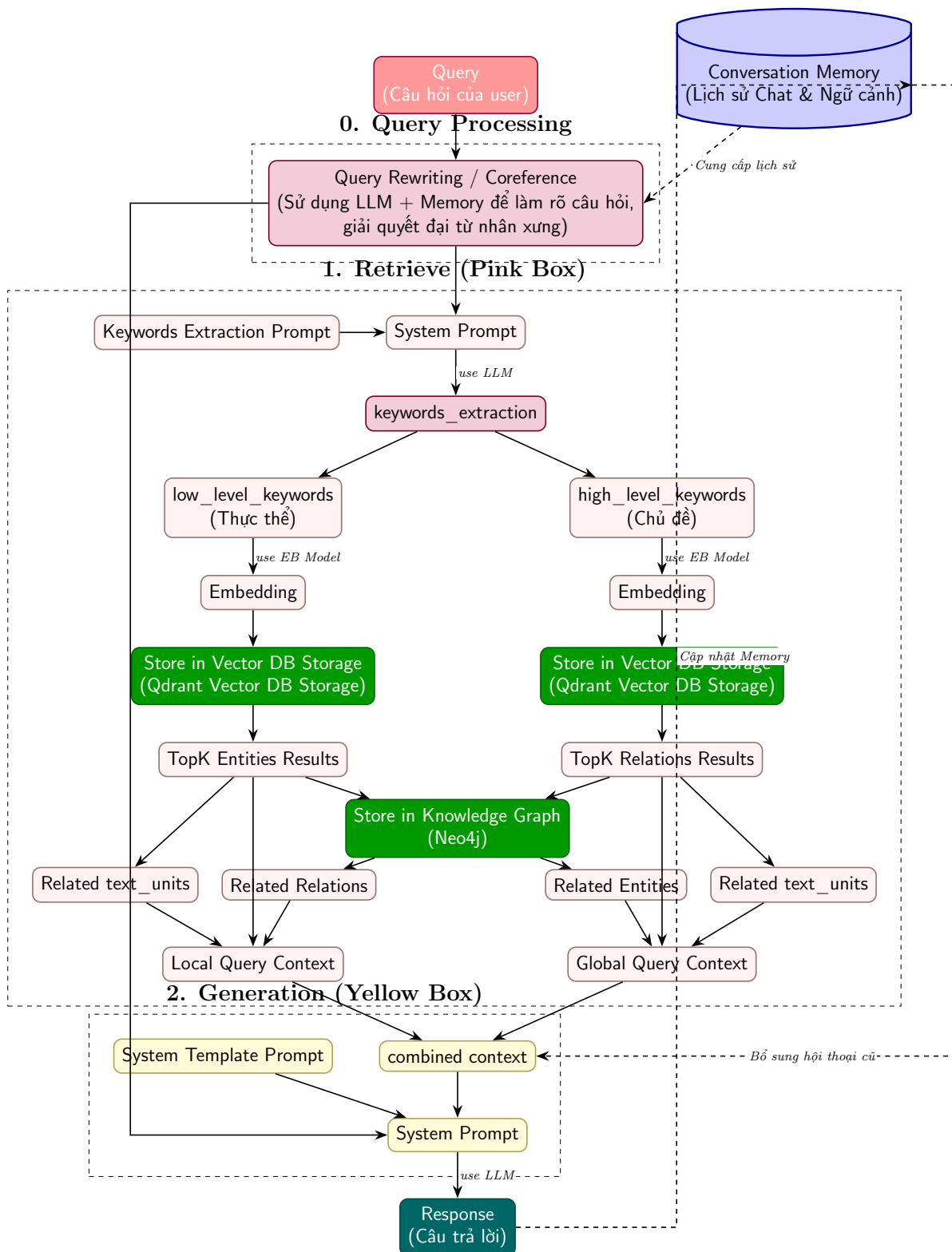
3.1.10 Quản lý và duy trì bền vững hệ thống

LightRAG thiết kế hệ thống quản trị với triết lý nhất quán tăng dần (incremental consistency): thay vì xây dựng lại toàn bộ đồ thị mỗi khi có tài liệu mới, hệ thống chỉ xử lý các đoạn văn mới hoặc đã thay đổi, sau đó hợp nhất kết quả vào đồ thị hiện có. Cơ chế làm sạch tự động (auto-cleaning) thường xuyên kiểm tra tính nhất quán giữa đồ thị tri thức và nội dung tài liệu gốc: nếu một tài liệu bị xóa khỏi kho dữ liệu, tất cả các nút và cạnh có nguồn gốc duy nhất từ tài liệu đó sẽ tự động bị loại bỏ, trong khi các nút được hỗ trợ bởi nhiều nguồn sẽ chỉ được cập nhật lại mô tả.

Người quản trị có thể can thiệp thủ công thông qua các API quản trị: hiệu chỉnh nhãn thực thể (entity label editing), cập nhật mô tả hoặc hợp nhất thủ công hai thực thể bị nhận diện nhầm là khác nhau. Sau bất kỳ thay đổi nào, hệ thống kích hoạt quy trình đồng bộ hóa tự động: véc-tơ nhúng của các thực thể bị ảnh hưởng được tính toán lại và cập nhật trong cơ sở dữ liệu vector, đảm bảo sự nhất quán tuyệt đối giữa hai lớp lưu trữ.

Cuối cùng, LightRAG hỗ trợ tùy chỉnh lời nhắc trích xuất theo lĩnh vực (domain-specific prompt tuning): người dùng có thể cung cấp danh sách

các kiểu thực thể ưu tiên và các mẫu quan hệ đặc thù, từ đó định hướng mô hình tập trung vào tri thức có giá trị nhất trong từng miền chuyên môn cụ thể.



Hình 3.12: Quy trình truy vấn đa chế độ và cơ chế tối ưu phản hồi trong LightRAG

Một trong những ưu điểm nổi bật nhất của LightRAG so với các hệ thống RAG truyền thống là khả năng thực hiện các truy vấn đa chế độ, cho phép hệ thống đáp ứng linh hoạt các loại câu hỏi khác nhau từ người học. Hệ thống hỗ trợ bốn chế độ truy vấn chính: Naive, Local, Global và Mix.

Chế độ truy vấn cơ bản (Naive Query): Đây là phương pháp tìm kiếm truyền thống dựa trên độ tương đồng vector. Hệ thống truy xuất các đoạn văn bản (chunks) có bản nhúng vector gần nhất với câu hỏi mà không tận dụng cấu trúc đồ thị. Chế độ này phù hợp cho các câu hỏi đơn giản, yêu cầu trích xuất thông tin trực tiếp từ một đoạn văn cụ thể.

Chế độ truy vấn cục bộ (Local Query): Chế độ này được tối ưu hóa cho các câu hỏi chi tiết về một khái niệm cụ thể (Low-level Query). Hệ thống sử dụng cơ chế tìm kiếm vector để xác định các thực thể liên quan nhất, sau đó mở rộng truy xuất các nút lân cận và các mối quan hệ trực tiếp trong đồ thị tri thức. Quy trình này cung cấp một ngữ cảnh tri thức tập trung, giúp mô hình ngôn ngữ trả lời chính xác các thuộc tính và mối quan hệ vi mô của thực thể.

Chế độ truy vấn toàn cục (Global Query): Đối với các câu hỏi mang tính tổng quát cao (High-level Query), đòi hỏi sự hiểu biết về toàn bộ kho dữ liệu, chế độ toàn cục phát huy vai trò chủ đạo. Thay vì tìm kiếm đoạn văn rời rạc, hệ thống tận dụng cấu trúc đồ thị ở mức độ vĩ mô bằng cách tổng hợp các mô tả tóm tắt của thực thể và quan hệ quan trọng. Cách tiếp cận này giúp mô hình ngôn ngữ bao quát được các chủ đề lớn và đưa ra những nhận định mang tính hệ thống vượt qua giới hạn của sở ngữ cảnh thông thường.

Chế độ truy vấn hỗn hợp (Mix Query): Đây là sự kết hợp giữa tìm kiếm cục bộ và toàn cục. Chế độ Mix đồng thời truy xuất các chi tiết cụ thể từ đồ thị lân cận và các tóm tắt tri thức vĩ mô từ cấp độ toàn cục. Quy trình xử lý này đảm bảo câu trả lời vừa có độ chính xác của dữ liệu chi tiết, vừa có chiều sâu của lập luận hệ thống, đáp ứng tốt nhất cho các nhiệm vụ phân tích tri thức đa tầng.

3.2 Mô hình hóa trạng thái hệ thống (Agent State)

Phân hệ trợ lý ảo được xây dựng trên nền tảng LangGraph và thiết kế dưới dạng một đồ thị trạng thái có hướng (state graph), trong đó mỗi nút đại diện cho một tác vụ tính toán và các cạnh đại diện cho luồng điều khiển. Trạng thái dùng chung của hệ thống được mô hình hóa bằng kiểu **AgentState** và có thể biểu diễn bởi tập hợp $S = \{Q, U, H, A, D, K, M, G, F\}$. Trong đó, Q (Query) là câu hỏi đầu vào, U (User ID) là mã định danh người dùng phục vụ cá nhân hóa, và H (Chat History) là lịch sử hội thoại ngắn hạn đã được tóm tắt. Thành phần A (Analysis) lưu kết quả phân tích ý định, D (Decision) là quyết định điều phối với ba giá trị `direct_answer`, `retrieve` hoặc `reject`, còn K (Keywords) là tập từ khóa truy xuất được tách thành hai mức cao và thấp. Cuối cùng, M (Long-term Memory) là hồ sơ người dùng dài hạn, G (Generation Request) là gói ngữ cảnh đầy đủ để sinh câu trả lời, và F (Final Response) là câu trả lời cuối cùng trả về cho người dùng. So với các kiến trúc có vòng lặp tự đánh giá, hệ thống đề xuất chủ trương giữ luồng điều phối tuyến tính và tắt định để tối ưu độ trễ phản hồi, dồn chất lượng vào khâu phân tích đầu vào và truy xuất tri thức thay vì hậu kiểm nhiều vòng.

3.2.1 Kiến trúc các tác vụ (Agents)

Hệ thống bao gồm năm thành phần xử lý chuyên biệt, được hiện thực dưới dạng các hàm trạng thái (stateless functions) và phối hợp theo một đồ thị có hướng do LangGraph điều phối, thay vì các tác nhân tự hành lập kế hoạch nhiều lượt.

Thứ nhất, Analyze Agent (`analyze_agent`) là thành phần trung tâm của khâu định tuyến. Bằng một lần gọi mô hình ngôn ngữ lớn với định dạng đầu ra JSON, agent này phân loại ý định người dùng, đưa ra quyết định điều phối (`direct_answer`, `retrieve` hoặc `reject`), chọn chế độ truy

vấn LightRAG, viết lại câu hỏi thành dạng độc lập (standalone query), và trích xuất hai tập từ khóa: từ khóa mức thấp (cụ thể, như mã môn học, tên giảng viên) và mức cao (khái quát, như chính sách, so sánh). Agent cũng đảm nhiệm vai trò kiểm duyệt: khi phát hiện nội dung không phù hợp, quyết định bị ép sang **reject**.

Thứ hai, Generator Agent (**generator**) chịu trách nhiệm dựng ngữ cảnh sinh chứ không trực tiếp gọi mô hình. Với nhánh truy xuất, persona của trợ lý được đặt vào trường `user_prompt` để LightRAG vẫn giữ được khuôn mẫu nội bộ và chèn ngữ cảnh tri thức `context_data`; với nhánh trả lời trực tiếp, agent dựng một `system_prompt` đầy đủ; còn với nhánh từ chối, agent trả về một thông điệp từ chối lịch sự đã soạn sẵn.

Thứ ba, Summarizer Agent (**summarizer**) nén lịch sử hội thoại khi vượt quá bốn lượt thành một đến hai câu súc tích, nhằm giảm số token mà Analyze Agent phải xử lý và duy trì ngữ cảnh cốt lõi qua các phiên dài.

Thứ tư, Long-term Memory Agent (**longterm**) quản lý hồ sơ người dùng bền vững trên MongoDB. Thay vì tóm tắt lại toàn bộ hồ sơ, agent chỉ trích xuất các sự kiện ổn định (chuyên ngành, sở thích trả lời, định hướng học tập dài hạn) dưới dạng JSON rồi hợp nhất chúng vào hồ sơ hiện có bằng mã Python có khử trùng lặp, tránh hiện tượng trôi dạt nội dung tóm tắt.

Cuối cùng, LLM Factory (11m) là lớp trừu tượng hóa nhà cung cấp mô hình, cho phép chuyển đổi linh hoạt giữa OpenAI và Ollama (mô hình cục bộ) qua biến môi trường và tự chuẩn hóa các tham số khác biệt giữa hai nền tảng.

3.2.2 Quy trình điều phối và Luồng dữ liệu

Đồ thị điều phối được biên dịch cùng một bộ lưu điểm kiểm tra (checkpoint) trên MongoDB và gồm bốn nút thực thi tuần tự: **analyzer** → **prepare_context** → **generate** → (có điều kiện) **update_long_term** → **END**. Quy trình thực thi được mô tả qua các bước logic sau:

Bước 1: Phân tích và Định tuyến. Khi nhận câu hỏi Q , nút **analyzer** gọi *Analyze Agent* (sau khi đã tóm tắt lịch sử qua *Summarizer*) để xác định ý định, quyết định điều phối, chế độ truy vấn và các từ khóa. Kết quả phân tích được chuẩn hóa: nếu nội dung không phù hợp thì quyết định bị ép sang **reject**; nếu giá trị không hợp lệ thì lùi an toàn về **retrieve**.

Bước 2: Chuẩn bị ngữ cảnh. Nút **prepare_context** không gọi mô hình ngôn ngữ. Nó chỉ nạp bộ nhớ dài hạn M từ MongoDB khi khâu phân tích xác định là cần thiết, rồi dựng gói yêu cầu sinh G (**generation_request**) chứa câu hỏi, chế độ truy vấn, persona và các từ khóa. Việc tách riêng bước này giúp luồng phát trực tuyến (streaming) có thể dừng tại đây trước khi sinh.

Bước 3: Sinh câu trả lời. Nút **generate** phân nhánh theo quyết định điều phối: với **retrieve**, hệ thống gọi LightRAG kèm các từ khóa đã trích sẵn để truy xuất tri thức từ đồ thị; với **direct_answer**, hệ thống gọi trực tiếp mô hình ngôn ngữ mà không truy xuất (phù hợp cho chào hỏi, trò chuyện hoặc lời khuyên học tập chung); với **reject**, hệ thống trả về thông điệp từ chối đã soạn sẵn.

Bước 4: Cập nhật bộ nhớ dài hạn (có điều kiện). Khác với cập nhật theo chu kỳ cố định, hệ thống chỉ kích hoạt nút **update_long_term** khi khâu phân tích đánh dấu cờ **should_update_long_term** - tức khi người dùng cung cấp một sự kiện ổn định đáng ghi nhớ. Khi đó, *Long-term Memory Agent* trích xuất và hợp nhất thông tin mới vào hồ sơ; ngược lại, đồ thị kết thúc ngay sau bước sinh để giảm độ trễ.

3.2.3 Cơ chế truy xuất tri thức (LightRAG Integration)

Khác với RAG truyền thống dựa thuần túy trên độ tương đồng vector, hệ thống sử dụng LightRAG để truy xuất trên cấu trúc đồ thị tri thức (Knowledge Graph). Điểm tối ưu quan trọng là các từ khóa đã được *Analyze Agent* trích xuất sẵn ở mức cao và mức thấp được truyền thẳng

vào LightRAG:

$$Q_{RAG} = Q_{standalone} + K_{high} + K_{low}$$

Nhờ vậy, LightRAG bỏ qua được bước tự trích xuất từ khóa nội bộ (vốn cần thêm một lần gọi mô hình), giúp giảm độ trễ mà vẫn định hướng truy xuất chính xác. Tùy theo độ khái quát của câu hỏi, hệ thống lựa chọn một trong các chế độ truy vấn **naive**, **local**, **global**, **hybrid** hoặc **mix** đã trình bày ở phần trên. Việc viết lại câu hỏi thành dạng độc lập kết hợp với từ khóa giúp khắc phục hiện tượng mất ngữ cảnh khi câu hỏi của sinh viên quá ngắn hoặc mơ hồ. Toàn bộ trạng thái hội thoại được lưu bền vững qua cơ chế checkpoint trên MongoDB, cho phép khôi phục đúng bước đang xử lý khi có gián đoạn.

3.2.4 Thuật toán điều phối (Pseudocode)

Algorithm 1 Quy trình điều phối Agent

Require: *query*, *user_id* ▷ Đầu vào từ sinh viên
Ensure: *answer* ▷ Câu trả lời cuối cùng

- 1: *routing* $\leftarrow$ AnalyzeAgent(*query*, Summarize(*history*))
- 2: *decision* $\leftarrow$ *routing.decision* ▷ direct_answer | retrieve | reject
- 3: **if** *decision* = “reject” **then**
- 4: **return** REJECT_MESSAGE
- 5: *memory* $\leftarrow$ LoadFromMongo(*user_id*) ▷ chỉ khi *routing.needs_lt*
- 6: *genReq* $\leftarrow$ BuildGenerationRequest(*routing*, *memory*)
- 7: **if** *decision* = “retrieve” **then**
- 8: *answer* $\leftarrow$ LightRAG(*genReq.query*, *genReq.mode*, K_{high} , K_{low})
- 9: **else**
- 10: *answer* $\leftarrow$ LLM(*genReq.system_prompt*, *query*)
- 11: **if** *routing.should_update_lt* **then**
- 12: UpdateMemory(*user_id*, *query*, *answer*) ▷ trích xuất + hợp nhất hồ sơ
- 13: **return** *answer*

3.3 Thiết lập thực nghiệm

3.3.1 Thiết lập thực nghiệm phân hệ trích xuất (MinerU)

3.3.1.1 Đối tượng nghiên cứu

Đối tượng nghiên cứu chính của phân hệ này là dữ liệu tri thức dạng văn bản phi cấu trúc trong môi trường học thuật, cụ thể là các tài liệu liên quan đến Chương trình đào tạo, quy chế học vụ, sổ tay sinh viên, và lộ trình môn học tại các trường đại học. Các công cụ trích xuất tài liệu đa phương thức sẽ được sử dụng để xử lý nguồn dữ liệu này.

3.3.1.2 Môi trường thực nghiệm

Hệ thống được triển khai thực nghiệm trên hạ tầng máy chủ đám mây nhằm đảm bảo khả năng tính toán mạnh mẽ cho các tác vụ xử lý ảnh và văn bản. Quá trình xử lý tài liệu đầu vào sử dụng công cụ MinerU (từ phiên bản 3.2.3) tích hợp các mô hình Vision-Language Model (VLM), OCR và Mathematical Formula Recognition (MFR).

3.3.1.3 Mô tả dữ liệu và quá trình thu thập

Để thực nghiệm và đánh giá chất lượng của bộ giải mã MinerU, nghiên cứu đã tiến hành thu thập tập dữ liệu Chương trình đào tạo chuẩn của sáu ngành và khối ngành thuộc Khoa, bao gồm: Cử nhân tài năng, Công nghệ thông tin, Hệ thống thông tin, Khoa học máy tính, Kỹ thuật phần mềm, và Trí tuệ nhân tạo. Nhằm đảm bảo tính đại diện, tài liệu được phân rã theo từng khóa tuyển sinh từ năm 2022 đến năm 2025.

3.3.1.4 Độ đo để đánh giá (Performance Metrics)

Phương pháp đánh giá được thực hiện thông qua việc đối sánh kết quả trích xuất tự động với nhãn chuẩn (Ground Truth). Để đánh giá mức độ chính xác cơ bản, nghiên cứu sử dụng CER (Character Error Rate) và WER (Word Error Rate) nhằm phản ánh tỷ lệ sai sót ở cấp độ ký tự và từ. Tương tự, Edit Distance Sim (Độ tương đồng khoảng cách sửa đổi) được áp dụng để đo lường sự sai khác chuỗi tổng thể. Nhằm định lượng mức độ trùng khớp văn bản sâu hơn, BLEU và METEOR được sử dụng để đánh giá độ bảo toàn n-gram. Đặc biệt, đối với cấu trúc dữ liệu phức tạp, TEDS-Struct và TEDS-Content (Tree-Edit-Distance-based Similarity) đóng vai trò đo lường chuyên biệt cho mức độ bảo toàn cấu trúc bảng biểu.

Biện luận về độ quan trọng của các chỉ số: Đối với kiến trúc truy xuất tăng cường đồ thị, các chỉ số BLEU, TEDS-Struct, TEDS-Content và Edit Distance Sim đóng vai trò then chốt nhất. Nếu mô hình bóc tách thất bại trong việc tái tạo cấu trúc bảng (TEDS-Struct thấp), lượng tri thức sẽ bị suy biến. Chỉ số BLEU cao minh chứng văn bản bảo toàn chính xác các cụm từ gốc, giúp LLM không sai lệch ngữ cảnh. Edit Distance Sim đảm bảo tính nguyên vẹn của mã học phần, tạo tiền đề để thuật toán đối soát và hợp nhất thực thể hoạt động ổn định.

3.3.2 Thiết lập thực nghiệm Hệ thống Đồ thị tri thức và Đa tác nhân

3.3.2.1 Đối tượng nghiên cứu

Bên cạnh dữ liệu tài liệu, đối tượng nghiên cứu trong hệ thống bao gồm các mô hình ngôn ngữ lớn (LLMs) và cơ sở dữ liệu đồ thị tri thức nhằm phục vụ việc xây dựng kiến trúc truy xuất tăng cường (GraphRAG). Hệ thống hướng tới việc phục vụ nhóm người dùng cuối là sinh viên, giúp họ giải đáp các thắc mắc học vụ một cách chính xác.

3.3.2.2 Môi trường thực nghiệm

Đối với quá trình đánh giá chất lượng đồ thị tri thức bằng phương pháp LLM-as-a-Judge, nhóm nghiên cứu đã lựa chọn mô hình Qwen/Qwen2.5-72B-Instruct làm trọng tài (Judge). Đây là dòng mô hình mã nguồn mở hàng đầu hiện nay, có khả năng tư duy logic (reasoning) và am hiểu sâu sắc ngữ pháp tiếng Việt, giúp phân tách được các khái niệm chuyên môn phức tạp mà các mô hình nhỏ thường nhầm lẫn. Các quy trình phân tích động, điều phối đa tác nhân (Multi-agent) được xây dựng dựa trên nền tảng LangGraph, trong khi việc lưu trữ và truy vấn đồ thị được quản lý bởi cơ sở dữ liệu đồ thị đồ thị (Neo4j).

3.3.2.3 Mô tả dữ liệu và quá trình thu thập

Về phía đồ thị tri thức, hệ thống đã xây dựng hai phiên bản đồ thị để đối soát chất lượng: Phiên bản gốc (Origin - kiến trúc LightRAG nguyên bản) và Phiên bản tối ưu (tích hợp chuẩn ngôn ngữ SHACL). Việc lấy mẫu đánh giá đồ thị được thực hiện ở các quy mô $N=10$, $N=200$, và $N=500$, dựa trên lý thuyết hội tụ tri thức để đảm bảo bao phủ các quan hệ hiếm gặp và đạt độ tin cậy thống kê 95%.

Bên cạnh đó, nhằm đánh giá khả năng sinh câu trả lời của hệ thống (RAG Metrics), một bộ dữ liệu kiểm thử (Benchmark Dataset) gồm 200 câu hỏi chuẩn đã được xây dựng. Các câu hỏi này được trích xuất từ các tình huống thực tế của sinh viên Khoa Công nghệ Thông tin trên các diễn đàn, xoay quanh chủ đề quy chế đào tạo, sổ tay học vụ và lộ trình môn học.

3.3.2.4 Độ đo để đánh giá (Performance Metrics)

Đánh giá chất lượng đồ thị tri thức. Khi triển khai kiến trúc LightRAG vào bài toán dữ liệu có tính hệ thống cao, rủi ro lớn nhất là việc mô hình ngôn ngữ sẽ gặp "ảo giác" (Hallucination) hoặc thiết lập các chuỗi quan

hệ rác. Hệ thống ứng dụng phương pháp phân tích tĩnh và động (Hybrid Assessment) thông qua ba trụ cột:

- Nhóm 1 - Đánh giá cấu trúc đồ thị (SHACL): Chuẩn SHACL của W3C được lựa chọn vì có thể kiểm tra ràng buộc cấu trúc đồ thị mà không yêu cầu tập dữ liệu nhãn thủ công. Nó giúp tìm kiếm các Nút mồ côi (Dangling Nodes) thông qua duyệt đồ thị có hướng để phát hiện các đỉnh không có mối quan hệ kết nối, ngăn chặn tình trạng tri thức bị phân mảnh. Đồng thời, Độ phủ văn bản (Chunk Coverage) được đo lường để đảm bảo không bỏ sót thông tin trọng yếu.

- Nhóm 2 - Đánh giá độ hữu cơ của quan hệ (LLM-as-a-Judge): Phương pháp này đánh giá độ lắng đọng trích xuất, với tiêu chí cốt lõi là Độ có cơ sở (Groundedness). Nó bắt buộc mọi mối quan hệ phải tồn tại chứng cứ tường minh từ các đoạn văn bản gốc, ngăn chặn hiện tượng âm tính giả và ảo giác của LLM.

- Nhóm 3 - Đánh giá hợp nhất thực thể (V-Measure): Đo lường hiệu quả việc hợp nhất các thực thể trùng lặp, giải quyết bài toán deduplication. Hệ thống theo dõi tỷ lệ Bỏ sót gộp (C1 Rate - Under-merge) nhằm giảm thiểu tình trạng một thực thể bị phân tách thành nhiều nút. Trái lại, Độ thuần khiết (C2 Rate - Purity - Over-merge) được duy trì để ngăn chặn việc đồng nhất hóa sai lầm các thực thể mang bản chất riêng biệt. Từ sự kết hợp của những yếu tố trên, Overall V-Measure được tính toán như một chỉ số tổng quát đại diện cho sự cân bằng hoàn hảo.

Đánh giá chất lượng sinh văn bản (RAG Metrics). Khung đánh giá RAGAS (Retrieval-Augmented Generation Assessment) được sử dụng với các chỉ số: - Độ liên quan của câu trả lời (Answer Relevance): Đảm bảo phản hồi đáp ứng đúng trọng tâm truy vấn. - Độ trung thực (Faithfulness): Đo lường khả năng bám sát ngữ cảnh từ đồ thị, hạn chế "ảo giác". - Độ gọi bối cảnh (Context Recall): Đánh giá khả năng truy xuất thành công các khái niệm phân tán.

Đánh giá hiệu năng điều phối (Agentic Metrics). Đo lường độ chính xác trong việc điều phối luồng xử lý (Routing Accuracy) của Tác nhân

Orchestrator và Tốc độ phản hồi trung bình (Average Latency) từ lúc nhận câu hỏi đến khi xuất ra câu trả lời.

Chương 4

Kết quả thí nghiệm và Ứng dụng

4.1 Đánh giá chất lượng MinerU trên tập tài liệu đầu vào

Dựa trên các thiết lập thực nghiệm, nghiên cứu đã tiến hành trích xuất và đo lường trên tập dữ liệu. Khoa học máy tính khóa 2022 là một minh chứng xuất sắc khi đạt CER chỉ 0.0019. Với chỉ số Edit Distance Sim, hầu hết tài liệu thuộc Hệ thống thông tin đều đạt trên 0.99, cho thấy độ chính xác gần như tuyệt đối. CNTT khóa 2022 đạt BLEU lên tới 0.9928. Đối với cấu trúc dữ liệu phức tạp, kết quả thực nghiệm chỉ ra phần lớn các khóa đều đạt TEDS-Struct 1.0000 dù KTPM khóa 2024 có sự sụt giảm nhẹ.

Bảng 4.1: Kết quả chi tiết đánh giá MinerU trên các tập tài liệu Chương trình đào tạo (2022-2025)

Tập tài liệu	CER ↓	WER ↓	Edit Dist Sim ↑	BLEU ↑	METEOR ↑	TEDS-Struct ↑	TEDS-Content ↑
<i>Công nghệ thông tin (CNTT)</i>							
Khóa 2022	0.0008	0.0045	0.9992	0.9928	0.9963	1.0000	0.9999
Khóa 2023	0.1255	0.1571	0.8745	0.7743	0.8149	0.6055	0.5552
Khóa 2024	0.2166	0.1808	0.8151	0.8008	0.8475	0.7416	0.7051
Khóa 2025	0.0006	0.0035	0.9994	0.9926	0.9974	1.0000	0.9999
<i>Cử nhân tài năng (CNTN)</i>							
Khóa 2022	0.0023	0.0107	0.9977	0.9804	0.9912	0.9925	0.9924
Khóa 2023	0.1462	0.1247	0.8720	0.8714	0.8508	0.7298	0.6946
Khóa 2024	0.1643	0.1249	0.8539	0.8538	0.8696	0.9994	0.9632
Khóa 2025	0.0020	0.0100	0.9980	0.9805	0.9914	1.0000	0.9998
<i>Hệ thống thông tin (HTTT)</i>							
Khóa 2022	0.0009	0.0051	0.9991	0.9929	0.9958	1.0000	0.9999
Khóa 2023	0.0011	0.0056	0.9989	0.9892	0.9954	1.0000	0.9997
Khóa 2024	0.0070	0.0284	0.9930	0.9469	0.9736	1.0000	0.9995
Khóa 2025	0.0038	0.0202	0.9962	0.9617	0.9807	1.0000	0.9999
<i>Khoa học máy tính (KHMT)</i>							
Khóa 2022	0.0019	0.0102	0.9981	0.9845	0.9914	1.0000	0.9999
Khóa 2023	0.0018	0.0096	0.9982	0.9832	0.9920	1.0000	0.9997
Khóa 2024	0.0061	0.0188	0.9939	0.9676	0.9853	1.0000	0.9994
Khóa 2025	0.0014	0.0075	0.9986	0.9843	0.9935	0.9976	0.9936
<i>Kỹ thuật phần mềm (KTPM)</i>							
Khóa 2022	0.0012	0.0069	0.9988	0.9891	0.9943	1.0000	1.0000
Khóa 2023	0.0016	0.0084	0.9984	0.9843	0.9930	1.0000	0.9998
Khóa 2024	0.2633	0.2137	0.7760	0.8018	0.8069	0.3342	0.2291
Khóa 2025	0.0031	0.0167	0.9969	0.9678	0.9862	1.0000	0.9997
<i>Trí tuệ nhân tạo (TNTT)</i>							
Khóa 2022	0.0013	0.0074	0.9987	0.9876	0.9939	1.0000	1.0000
Khóa 2023	0.0014	0.0074	0.9986	0.9854	0.9939	1.0000	0.9998
Khóa 2024	0.2266	0.1660	0.8075	0.8117	0.8432	0.4277	0.3081
Khóa 2025	0.0109	0.0201	0.9891	0.9644	0.9849	0.4247	0.3373

4.2 Đánh giá chất lượng đồ thị tri thức

Dưới đây là kết quả thực nghiệm đối soát toàn diện giữa kiến trúc Ligh-tRAG nguyên bản (Origin) và kiến trúc tối ưu (Tối ưu) tích hợp SHACL:

Bảng 4.2: Tổng hợp kết quả đánh giá đồ thị tri thức trên các cấp độ lấy mẫu khác nhau

Chỉ số đánh giá	Cấp độ N=10		Cấp độ N=200		Cấp độ N=500	
	Origin	Tối ưu	Origin	Tối ưu	Origin	Tối ưu
Nút mồ côi (Dangling)	19.4%	5.2%	19.4%	5.2%	19.4%	5.2%
Độ phủ văn bản (Chunk)	36.1%	96.9%	36.1%	96.9%	36.1%	96.9%
Độ có cơ sở (Grounded)	0.910	0.914	0.990	1.000	0.986	0.998
Bỏ sót gộp (C1 Rate)	44.0%	6.0%	45.0%	7.0%	36.8%	8.0%
Độ thuần khiết (C2 Rate)	60.0%	70.0%	71.5%	78.0%	74.4%	78.7%
Overall V-Measure	57.9%	80.2%	62.2%	84.8%	68.3%	84.8%

Phân tích kết quả thực nghiệm cho thấy một sự cải thiện vượt bậc và toàn diện sau khi áp dụng quy trình tối ưu. Cụ thể, tỷ lệ nút mồ côi đã giảm mạnh từ 19.4% xuống chỉ còn 5.2%. Sự chuyển biến này chứng minh rằng việc áp dụng SHACL cùng thuật toán duyệt đồ thị đã ép buộc các thực thể hòa nhập vào ngữ cảnh chung, triệt tiêu hiệu quả các "ốc đảo tri thức" cô lập và đảm bảo tính liên kết logic trong từng câu trả lời của AI. Nhờ vậy, hệ thống cũng ghi nhận một bước đột phá lớn về độ phủ văn bản khi tăng từ 36.1% lên mức ấn tượng 96.9%; điều này khẳng định gần như toàn bộ thông tin trọng yếu đã được "hấp thụ" thành công vào đồ thị tri thức mà không bị bỏ sót. Cùng với đó, chất lượng hợp nhất thực thể cũng chứng kiến sự thăng tiến đáng kể khi chỉ số V-Measure tăng từ khoảng 60% lên 85%, kéo theo tỷ lệ bỏ sót gộp (C1) giảm mạnh từ 44% xuống 8%. Những con số này minh chứng cho việc giải quyết thành công tình trạng phân mảnh thực thể, làm cho đồ thị trở nên tinh gọn, trong khi vẫn cho phép sự can thiệp thủ công qua giao diện để duy trì độ chuẩn xác tuyệt đối. Đáng chú ý nhất, độ có cơ sở (Groundedness) luôn được duy trì ở mức tiệm cận tuyệt đối (0.998 - 1.000), qua đó khẳng định sự kết hợp giữa SHACL và mô hình ngôn ngữ cấp chuyên gia đã loại bỏ hoàn toàn các quan hệ sai lệch, đáp ứng trọn vẹn tiêu chuẩn tin cậy học thuật [19].

4.3 Đánh giá chất lượng truy xuất và sinh văn bản (RAG Metrics)

Chất lượng sinh văn bản của hệ thống được đo lường khắt khe dựa trên khung đánh giá RAGAS (Retrieval-Augmented Generation Assessment) [6], mang lại những kết quả khả quan trên bộ 200 câu hỏi chuẩn. Yếu tố nổi bật nhất là Độ liên quan của câu trả lời (Answer Relevance) đạt mức 88.76%, chứng tỏ các phản hồi sinh ra đáp ứng đúng trọng tâm truy vấn của sinh viên, vượt trội rõ rệt so với các phương pháp RAG truyền thống [17]. Để hỗ trợ cho sự chính xác này, hệ thống duy trì Độ trung thực (Faithfulness) ở mức 76.07%; kết quả này minh chứng cho khả năng bám sát ngữ cảnh từ Đồ thị tri thức, qua đó hạn chế tối đa việc mô hình bị "ảo giác" (Hallucination) hoặc tự bịa đặt thông tin [12]. Thêm vào đó, thông qua cơ chế Graph-aware Retrieval lấy cảm hứng từ GraphRAG [5], Độ gọi bối cảnh (Context Recall) cũng đạt 76.01%, cho thấy hệ thống có khả năng truy xuất thành công các khái niệm phân tán vốn là thách thức lớn đối với kỹ thuật Vector Search thông thường.

4.4 Đánh giá hiệu năng điều phối Đa tác nhân (Agentic Metrics)

Việc tích hợp kiến trúc LangGraph mang lại sự linh hoạt lớn cho hệ thống nhưng đồng thời cũng đặt ra bài toán về hiệu năng tính toán. Quá trình kiểm định cho thấy Tác nhân Orchestrator hoạt động vô cùng hiệu quả với tỷ lệ điều phối đúng công cụ (Routing Accuracy) đạt độ chính xác cao, đặc biệt trong việc phân loại ý định người dùng để điều hướng tới luồng truy xuất phù hợp (Local, Global hoặc Hybrid). Bất chấp sự phức tạp của luồng đa tác nhân, tốc độ phản hồi (Average Latency) vẫn được tối ưu ở mức chấp nhận được, với thời gian trung bình từ lúc nhận câu hỏi đến khi xuất ra câu trả lời hoàn chỉnh đạt 7.32 giây.

4.5 Đánh giá hệ thống từ phía người dùng cuối

Quá trình thử nghiệm thực tế (Beta testing) được tiến hành với sự tham gia của các sinh viên Khoa Công nghệ Thông tin. Hệ thống đã giải quyết trọn vẹn 40 câu hỏi chuẩn, cung cấp lộ trình học tập logic, mở ra cơ hội ứng dụng mạnh mẽ các mô hình ngôn ngữ lớn vào giáo dục [13].

Đặc biệt, hệ thống xử lý hiệu quả các truy vấn phức tạp trên diễn đàn (Forum) chứa từ khóa lỏng và văn phong tự do. Các câu hỏi vi phạm chính sách hoặc mang tính chất spam đều bị Tác nhân phân tích nhận diện và từ chối phản hồi ngay lập tức, đảm bảo môi trường học thuật lành mạnh.

4.6 Thiết kế và xây dựng ứng dụng

4.6.1 Ý tưởng ứng dụng

Sinh viên Khoa Công nghệ Thông tin tại Trường Đại học Khoa học Tự nhiên (HCMUS) đang phải đối mặt với một khối lượng thông tin học vụ lớn và phân tán: từ chương trình đào tạo, đề cương chi tiết từng môn học, quy chế học vụ đến mạng lưới môn tiên quyết và song hành. Để tìm được câu trả lời cho một câu hỏi đơn giản như “Mình cần học gì trước khi học Trí tuệ Nhân tạo?”, sinh viên thường phải lần lượt tra cứu qua nhiều tệp PDF rời rạc hoặc chờ đợi giờ tư vấn của giảng viên.

Ý tưởng trung tâm của hệ thống là xây dựng một trợ lý học tập thông minh có khả năng hiểu sâu về toàn bộ kho tri thức học vụ của Khoa. Hệ thống không chỉ trả lời các câu hỏi đơn lẻ mà còn hiểu được mối liên hệ logic giữa các môn học, chuyên ngành và yêu cầu tốt nghiệp, từ đó đưa ra lời tư vấn cá nhân hóa và có căn cứ rõ ràng.

4.6.1.1 Các tình huống sử dụng tiêu biểu

Tình huống 1: Tra cứu lộ trình môn học. Sinh viên năm hai đang lập kế hoạch học tập đặt câu hỏi: *“Mình muốn học môn Trí tuệ Nhân tạo thì cần học những môn gì trước?”*. Hệ thống AI Agent phân tích câu hỏi, truy xuất đồ thị tri thức để xác định chuỗi môn tiên quyết, sau đó trả lời chi tiết kèm mã môn học và lộ trình học kỳ gợi ý.

Tình huống 2: Kiểm tra điều kiện tốt nghiệp. Sinh viên năm bốn hỏi: *“Mình còn thiếu những gì để đủ điều kiện xét tốt nghiệp?”*. Hệ thống đối chiếu hồ sơ học tập cá nhân được lưu trong bộ nhớ dài hạn với quy định chuẩn đầu ra của Khoa, liệt kê các môn còn thiếu và đưa ra gợi ý đăng ký cho học kỳ tiếp theo.

Tình huống 3: Hỏi đáp về quy chế học vụ. Sinh viên thắc mắc: *“Nếu mình trượt môn tiên quyết thì kỳ sau có được đăng ký môn tiếp theo không?”*. Hệ thống tra cứu quy chế đào tạo tín chỉ của trường, trả lời chính xác với đoạn trích dẫn từ văn bản quy chế tương ứng.

4.6.1.2 Đối tượng người dùng mục tiêu

Hệ thống hướng tới hai nhóm người dùng chính: Sinh viên Khoa Công nghệ Thông tin tại Trường Đại Học Khoa Học Tự Nhiên, những người cần tra cứu thông tin học vụ và lập kế hoạch học tập.

4.6.2 Phân tích và đặc tả yêu cầu

4.6.2.1 Yêu cầu chức năng

Hệ thống được xác định cần đáp ứng các nhóm chức năng chính được liệt kê trong Bảng 4.3.

Bảng 4.3: Danh sách yêu cầu chức năng

STT	Nhóm chức năng	Mô tả
1	Xác thực người dùng	Đăng ký tài khoản, đăng nhập bằng email/mật khẩu, khôi phục mật khẩu qua email.
2	Chat với AI Agents	Đặt câu hỏi học vụ bằng ngôn ngữ tự nhiên, nhận câu trả lời có nguồn gốc từ đồ thị tri thức, xem lại lịch sử hội thoại theo phiên.

4.6.2.2 Yêu cầu phi chức năng

Bảng 4.4: Yêu cầu phi chức năng

STT	Yêu cầu	Ý nghĩa / Mô tả
1	Hiệu suất	Giao diện AI Chat phải đảm bảo tải trong vòng ≤ 2 giây để đảm bảo trải nghiệm người dùng.
2	Khả năng mở rộng	Hệ thống phải mở rộng được để xử lý lượng lớn người dùng và phiên hội thoại đồng thời.
3	Khả dụng	Hệ thống phải hoạt động liên tục với up-time $\geq 99.9\%$ /năm.
4	Đáp ứng	Các thao tác cơ bản như đăng nhập, AI Chat phản hồi nhanh (dưới 1 giây).
5	Khả năng sử dụng	UI/UX đơn giản, dễ hiểu, dễ đặt câu hỏi AI.

4.6.2.3 Danh sách Actor

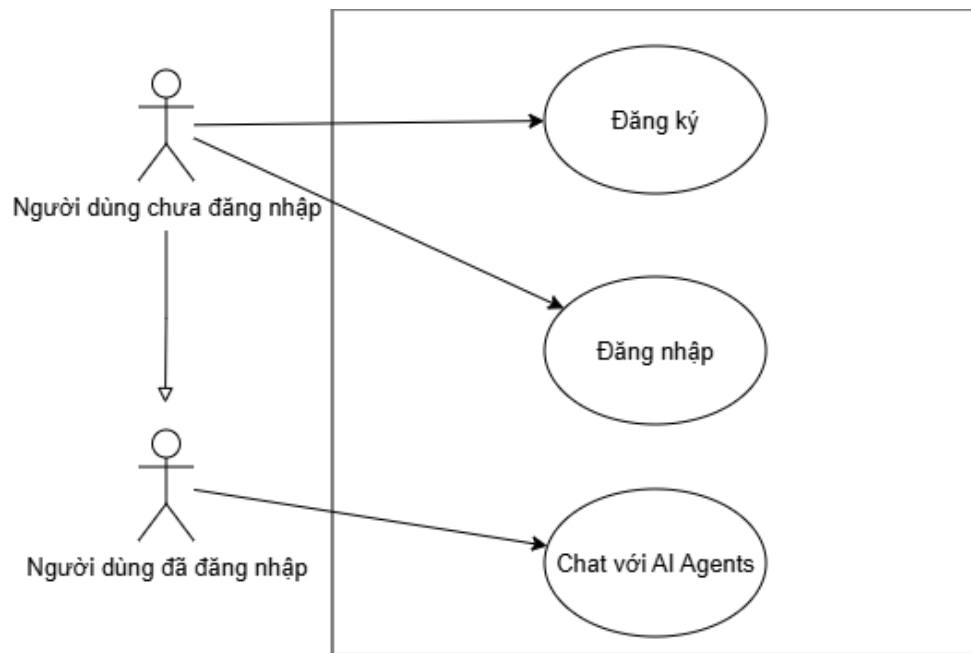
Hệ thống xác định một Actor chính tương tác trực tiếp với toàn bộ các chức năng:

Bảng 4.5: Danh sách Actor

STT	Actor	Ý nghĩa / Ghi chú
1	Người dùng	Sinh viên trực tiếp sử dụng hệ thống, thao tác các chức năng và gửi phản hồi.

4.6.3 Mô hình Use Case

Hệ thống có một phân hệ chính là Sinh viên, tương tác với hệ thống qua ba chức năng: đăng ký tài khoản, đăng nhập và chat với AI Agents. Sơ đồ Use Case tổng quan được thể hiện tại Hình 4.1.



Hình 4.1: Sơ đồ Use Case hệ thống

4.6.3.1 Danh sách Use Case

Bảng 4.6: Danh sách Use Case

STT	Tên Use Case	Mô tả	Actor
1	Đăng ký tài khoản	Sinh viên tạo tài khoản mới trên hệ thống.	Sinh viên
2	Đăng nhập	Sinh viên đăng nhập bằng email và mật khẩu.	
3	Chat với AI Agents	Sinh viên đặt câu hỏi học vụ và nhận tư vấn thông minh từ AI.	

4.6.3.2 Đặc tả Use Case

Use Case: Đăng ký tài khoản

Tên use-case	Đăng ký tài khoản
Tóm tắt	Cho phép sinh viên tạo tài khoản mới để sử dụng hệ thống.
Tác nhân	Sinh viên
Use-case liên quan	Đăng nhập
Điều kiện tiên quyết	Email chưa được đăng ký trong hệ thống.

Dòng sự kiện chính	- Sinh viên truy cập trang đăng ký. - Sinh viên nhập họ tên, email và mật khẩu. - Sinh viên xác nhận mật khẩu và đồng ý điều khoản. - Sinh viên nhấn nút <i>Đăng ký</i>. - Hệ thống kiểm tra hợp lệ và tạo tài khoản. - Hệ thống chuyển hướng sang trang đăng nhập.
Dòng sự kiện phụ	A1. Rẽ nhánh tại bước 2, email đã tồn tại: Hệ thống hiển thị thông báo lỗi và yêu cầu sinh viên sử dụng email khác.
Hậu điều kiện	Tài khoản được tạo thành công; sinh viên có thể đăng nhập vào hệ thống.

Use Case: Đăng nhập

Tên use-case	Đăng nhập hệ thống
Tóm tắt	Cho phép sinh viên xác thực danh tính và truy cập vào hệ thống.
Tác nhân	Sinh viên
Use-case liên quan	Đăng ký tài khoản, Chat với AI Agents
Điều kiện tiên quyết	Sinh viên đã có tài khoản được đăng ký trong hệ thống.

Dòng sự kiện chính	- Sinh viên truy cập trang đăng nhập. - Sinh viên nhập email và mật khẩu. - Sinh viên nhấn nút <i>Đăng nhập</i>. - Hệ thống xác thực thông tin đăng nhập. - Hệ thống cấp phát JWT token và chuyển hướng đến trang chính.
Dòng sự kiện phụ	A1. Rẽ nhánh tại bước 4, thông tin không hợp lệ: Hệ thống hiển thị thông báo lỗi và yêu cầu sinh viên nhập lại. A2. Quên mật khẩu: Sinh viên chọn <i>Quên mật khẩu</i>, hệ thống gửi email chứa liên kết khôi phục.
Hậu điều kiện	Sinh viên đăng nhập thành công và có thể sử dụng đầy đủ chức năng hệ thống.

Use Case: Chat với AI Agents

Tên use-case	Chat với AI Agents
Tóm tắt	Sinh viên đặt câu hỏi về học vụ và nhận câu trả lời thông minh từ AI Agent.
Tác nhân	Sinh viên
Use-case liên quan	Đăng nhập
Điều kiện tiên quyết	Sinh viên đã đăng nhập, dịch vụ AI Agent đang hoạt động.

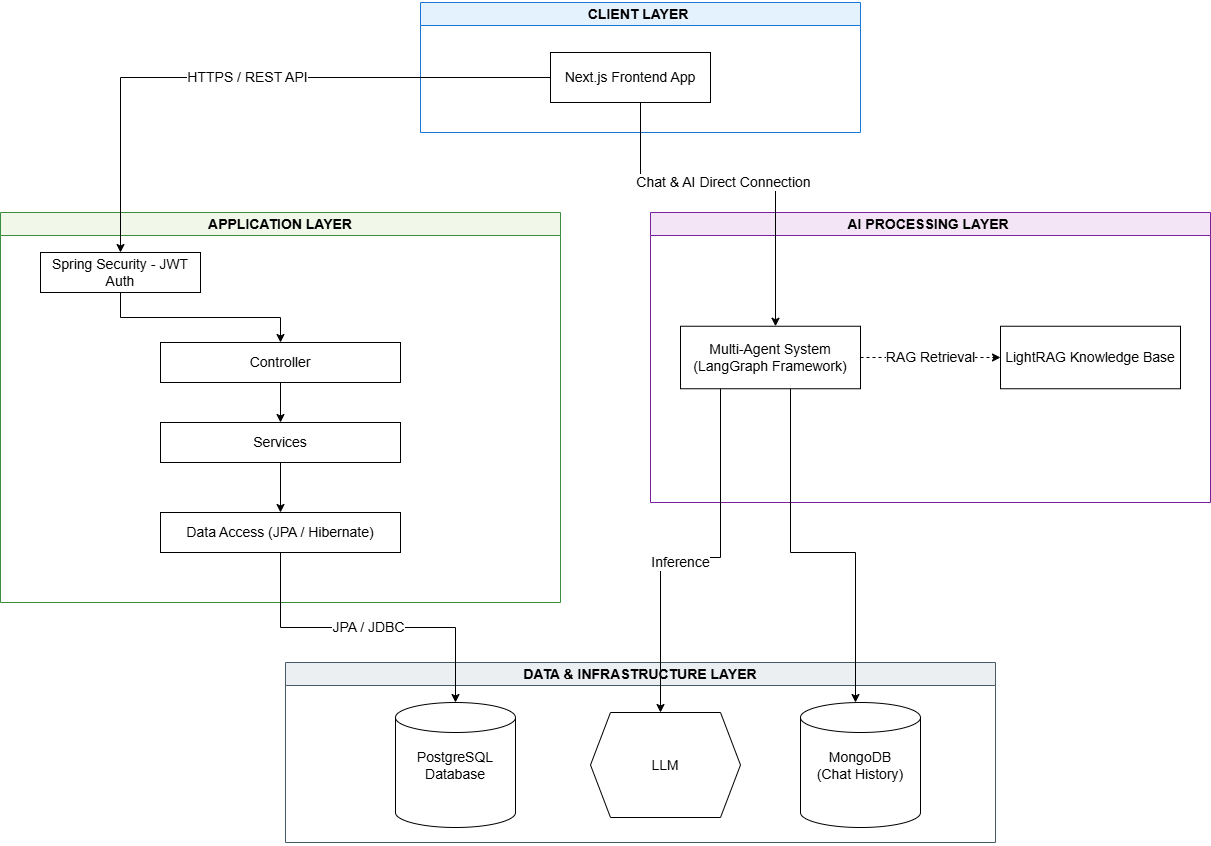
Dòng sự kiện chính	- Sinh viên điều hướng đến phần AI Chat. - Sinh viên nhập câu hỏi liên quan đến học vụ. - Hệ thống chuyển tiếp câu hỏi đến hệ thống AI Agents. - AI Agent phân tích, truy xuất đồ thị tri thức và tạo phản hồi. - Hệ thống hiển thị câu trả lời trong giao diện chat. - Sinh viên có thể đặt câu hỏi tiếp theo, lặp lại bước 2-5.
Dòng sự kiện phụ	A1. Rẽ nhánh tại bước 5, dịch vụ AI không khả dụng: Hệ thống hiển thị thông báo lỗi và gợi ý sinh viên thử lại sau. A2. Câu hỏi ngoài phạm vi: AI phản hồi rằng câu hỏi nằm ngoài phạm vi hỗ trợ của hệ thống.
Hậu điều kiện	Sinh viên nhận được câu trả lời, cuộc hội thoại được lưu vào lịch sử phiên.

4.6.4 Kiến trúc hệ thống

Mô hình kiến trúc tổng thể của hệ thống được đề xuất dựa trên kiến trúc đa tầng (Multi-tier Architecture), nhằm đảm bảo tính tách biệt giữa giao diện người dùng, logic nghiệp vụ và các phân hệ xử lý trí tuệ nhân tạo. Sơ đồ chi tiết được thể hiện tại Hình 4.2.

Sơ lược về kiến trúc được nhóm sử dụng như sau: hệ thống được tổ chức theo mô hình đa tầng gồm bốn tầng chính, trong đó Tầng Giao diện tiếp nhận yêu cầu từ sinh viên và phân phối sang hai luồng xử lý độc lập,

Tầng Nghiệp vụ xử lý logic xác thực và nghiệp vụ, Tầng Điều phối AI đảm nhận toàn bộ xử lý trí tuệ nhân tạo, và Tầng Dữ liệu cung cấp hạ tầng lưu trữ chuyên biệt cho từng loại dữ liệu.



Hình 4.2: Mô hình kiến trúc hệ thống tổng thể

4.6.4.1 Tầng Giao diện (Client Layer)

Tầng Giao diện được xây dựng trên Next.js, là điểm tiếp xúc duy nhất giữa sinh viên và toàn bộ hệ thống. Khi sinh viên sử dụng ứng dụng, tầng này vận hành hai luồng giao tiếp song song tùy theo tính năng được yêu cầu.

Đối với các tác vụ liên quan đến tài khoản và thông tin cá nhân, ứng dụng giao tiếp với Tầng Nghiệp vụ thông qua giao thức HTTPS và REST API. Đây là luồng truyền thống, đảm bảo tính nhất quán và bảo mật cho các thao tác đọc ghi dữ liệu có cấu trúc.

Đối với tính năng Chat tư vấn học vụ, ứng dụng thiết lập kết nối trực

tiếp đến Tầng Điều phối AI (Chat & AI Direct Connection) mà không đi qua Tầng Nghiệp vụ trung gian. Thiết kế giúp giảm thiểu độ trễ và cho phép hiển thị câu trả lời theo từng phần, giúp mang lại trải nghiệm hội thoại tự nhiên và mượt mà hơn cho người dùng.

4.6.4.2 Tầng Nghiệp vụ (Application Layer)

Tầng Nghiệp vụ được xây dựng trên Spring Boot, đảm nhận nhiệm vụ xử lý toàn bộ logic nghiệp vụ và kiểm soát luồng dữ liệu giữa các tầng trong hệ thống.

Mọi yêu cầu từ phía Client trước tiên đều phải đi qua lớp bảo mật Spring Security kết hợp với cơ chế xác thực JWT (JSON Web Token). Giúp kiểm tra tính hợp lệ của token và phân quyền truy cập trước khi cho phép yêu cầu đi tiếp vào bên trong hệ thống.

Sau khi xác thực thành công, yêu cầu được định tuyến đến Controller rồi chuyển xuống lớp Services để thực thi logic nghiệp vụ. Lớp Data Access sử dụng Spring Data JPA và Hibernate để ánh xạ đối tượng và giao tiếp với cơ sở dữ liệu PostgreSQL thông qua giao thức JPA/JDBC.

4.6.4.3 Tầng Điều phối AI (AI Processing Layer)

Tầng Điều phối AI là trung tâm xử lý trí tuệ nhân tạo của hệ thống, tiếp nhận câu hỏi từ sinh viên qua luồng trực tiếp từ Client thông qua Chat & AI Direct Connection.

Khi nhận được câu hỏi, Multi-Agent System được xây dựng trên Lang-Graph Framework sẽ điều phối các tác nhân AI chuyên biệt để phân tích ý định câu hỏi, lập kế hoạch trả lời và tổng hợp kết quả. Trước khi sinh ra câu trả lời, hệ thống kích hoạt cơ chế RAG Retrieval để truy xuất các đoạn thông tin liên quan từ LightRAG Knowledge Base, kho tri thức học vụ đã được xây dựng và lập chỉ mục từ tài liệu của Khoa. Việc bổ sung ngữ cảnh từ tri thức thực tế giúp hạn chế tối đa hiện tượng “ảo giác” (hallucination) của mô hình ngôn ngữ, đảm bảo câu trả lời có căn cứ rõ ràng.

Sau khi có đầy đủ ngữ cảnh, hệ thống dùng LLM để tạo ra câu trả lời cuối cùng và trả về cho sinh viên.

4.6.4.4 Tầng Dữ liệu và Hạ tầng (Data & Infrastructure Layer)

Tầng này gồm ba thành phần chính, mỗi thành phần phục vụ một mục đích riêng biệt.

PostgreSQL được dùng để lưu thông tin tài khoản của sinh viên, bao gồm thông tin đăng nhập, phân quyền và hồ sơ cá nhân. Nhóm chọn PostgreSQL vì hỗ trợ tốt các ràng buộc toàn vẹn dữ liệu, phù hợp với yêu cầu bảo mật cao cho dữ liệu người dùng. Tầng Nghiệp vụ kết nối với PostgreSQL thông qua giao thức JPA/JDBC.

MongoDB lưu lịch sử hội thoại giữa sinh viên và AI Agent theo từng phiên. Do dữ liệu hội thoại không có cấu trúc cố định và cần truy xuất nhanh, MongoDB với cấu trúc tài liệu linh hoạt là lựa chọn phù hợp hơn so với cơ sở dữ liệu quan hệ.

LLM (Large Language Model) là thành phần thực hiện suy luận và sinh ra câu trả lời cuối cùng. Khi Tầng Điều phối AI đã thu thập đủ ngữ cảnh từ Knowledge Graph, yêu cầu Inference được gửi đến LLM để tổng hợp và trả lời cho sinh viên.

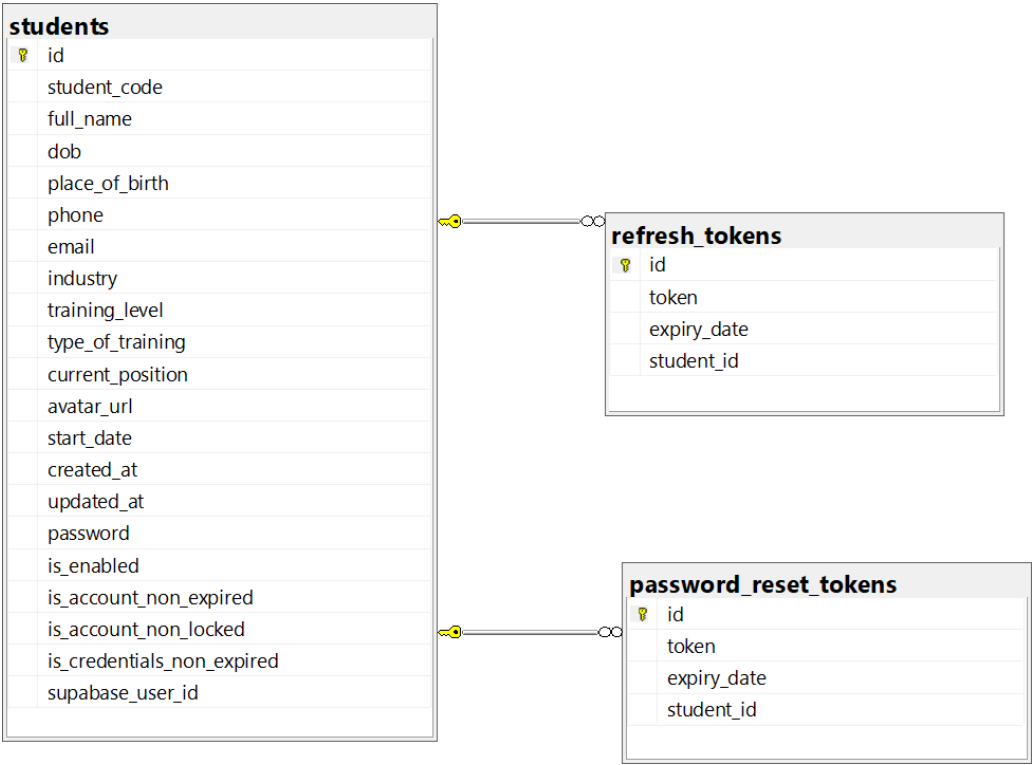
4.6.5 Thiết kế chi tiết các thành phần

Hệ thống sử dụng ba loại cơ sở dữ liệu với vai trò khác nhau: PostgreSQL lưu thông tin tài khoản người dùng, MongoDB lưu lịch sử hội thoại với AI Agent, và Knowledge Graph (LightRAG) lưu toàn bộ tri thức học vụ phục vụ cho tính năng tư vấn.

4.6.5.1 Thiết kế cơ sở dữ liệu

Cơ sở dữ liệu Quan hệ (PostgreSQL) PostgreSQL được dùng để lưu thông tin tài khoản và hồ sơ người dùng trong hệ thống. Cấu trúc quan

hệ giúp đảm bảo tính toàn vẹn dữ liệu và hỗ trợ phân quyền chặt chẽ. Sơ đồ tổng quan của cơ sở dữ liệu được thể hiện tại Hình 4.3.



Hình 4.3: Mô hình sơ đồ cơ sở dữ liệu quan hệ người dùng

Bảng 4.10: Danh sách các bảng trong cơ sở dữ liệu quan hệ

STT	Tên bảng	Ý nghĩa
1	students	Lưu toàn bộ thông tin hồ sơ người dùng: mã sinh viên, họ tên, ngày sinh, email, mật khẩu, ngành học, trình độ đào tạo và các cờ trạng thái tài khoản (<code>is_enabled</code> , <code>is_account_non_locked</code> ,...).

STT	Tên bảng	Ý nghĩa
2	refresh_tokens	Lưu token phiên làm việc (token, expiry_date) để duy trì trạng thái đăng nhập mà không cần xác thực lại.
3	password_reset_tokens	Lưu token bảo mật (token, expiry_date) dùng trong quy trình đặt lại mật khẩu.

Ràng buộc Khóa ngoại

Bảng 4.11: Mô tả khóa ngoại của các bảng trong CSDL quan hệ

STT	Tên Bảng	Tên cột	Bảng tham chiếu	Cột tham chiếu
1	refresh_tokens	student_id	students	id
2	password_reset_tokens	student_id	students	id

Cơ sở dữ liệu Phi cấu trúc (MongoDB) Để phục vụ cho tác nhân duy trì bộ nhớ dài hạn (Long-term Memory Agent) và quản lý trạng thái luồng suy luận trong kiến trúc LangGraph, hệ thống sử dụng MongoDB. Dữ liệu được tổ chức linh hoạt để lưu trữ lịch sử hội thoại và các điểm kiểm soát (checkpoints) của AI Agent.

Bảng 4.12: Danh sách các Collection trong cơ sở dữ liệu hcmus_langgraph

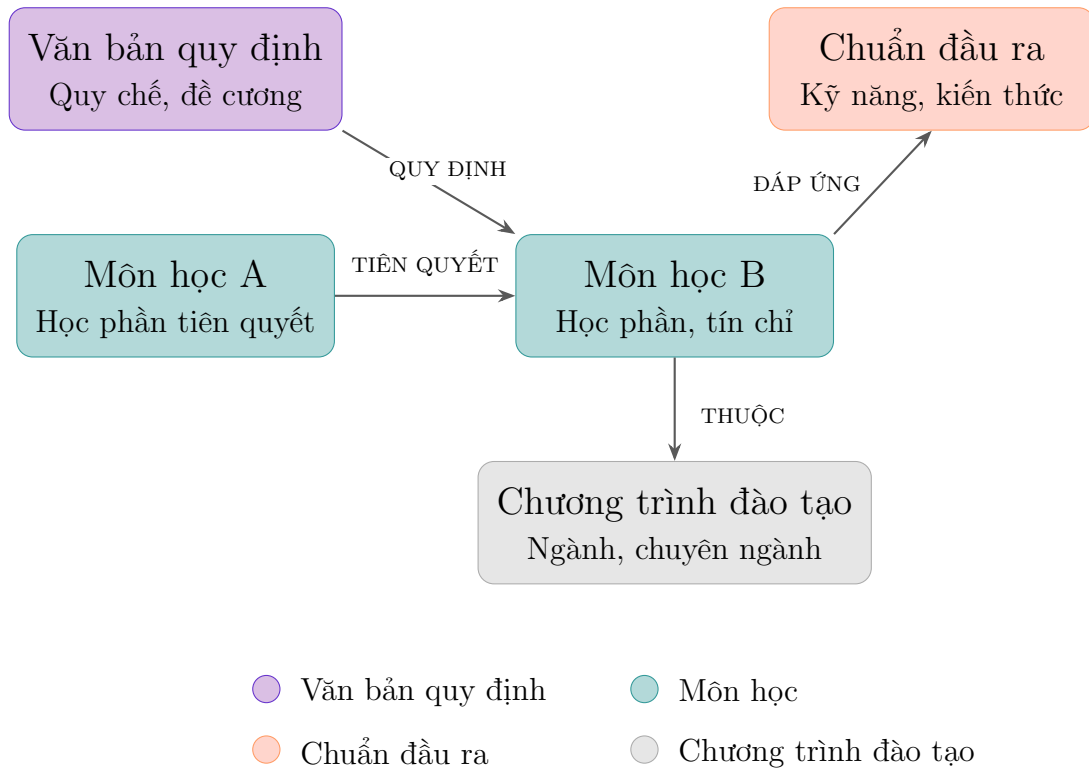
STT	Tên Collection	Ý nghĩa dữ liệu thực tế
1	session_info	Lưu trữ thông tin định danh của phiên chat bao gồm: session_id, user_id, tiêu đề cuộc hội thoại và thời gian cập nhật gần nhất.
2	long_term_memory	Lưu trữ các tri thức hoặc thông tin quan trọng được đúc kết từ các cuộc hội thoại cũ để AI Agent sử dụng cho các lần tương tác sau.
3	checkpoints	Lưu trữ trạng thái hiện tại (State) của luồng LangGraph, cho phép hệ thống phục hồi lại đúng bước đang xử lý khi có lỗi hoặc ngắt quãng.
4	checkpoint_writes	Ghi lại nhật ký các thay đổi của trạng thái Agent qua từng bước trong luồng điều phối.

4.6.5.2 Cơ sở dữ liệu Đồ thị (Knowledge Graph)

Hệ thống sử dụng LightRAG để xây dựng đồ thị tri thức học vụ, phục vụ lưu trữ thông tin phức tạp với nhiều mối quan hệ đan xen. Thay vì cấu trúc bảng (Table), dữ liệu được biểu diễn dưới dạng Thực thể (Nodes) và Mối quan hệ (Edges/Relationships), lưu trữ trong Neo4j.

Cấu trúc đồ thị tri thức được phân tích theo hai lớp: lớp khái niệm (logical view) thể hiện ngữ nghĩa các loại thực thể và quan hệ, và lớp lưu trữ vật lý (physical view) phản ánh cách triển khai thực tế trong Neo4j.

4.6.5.2.1 Lớp khái niệm (logical view) Hình 4.4 thể hiện mô hình khái niệm các loại thực thể và quan hệ chính trong đồ thị tri thức học vụ.



Hình 4.4: Mô hình khái niệm các loại thực thể và quan hệ chính trong đồ thị tri thức học vụ

Về mặt khái niệm, đồ thị gồm bốn nhóm thực thể chính. Môn học là thực thể trung tâm, lưu các học phần cùng thông tin tín chỉ, các môn có thể liên kết tiên quyết với nhau qua quan hệ TIÊN QUYẾT. Chương trình đào tạo nhóm các môn học theo ngành và chuyên ngành qua quan hệ THUỘC. Chuẩn đầu ra mô tả các kỹ năng, kiến thức cần đạt qua quan hệ ĐÁP ỨNG. Văn bản quy định gồm quy chế và đề cương là nguồn gốc tri thức của hệ thống, liên kết qua quan hệ QUY ĐỊNH.

4.6.5.2.2 Lớp lưu trữ vật lý (physical view) Về mặt triển khai trong Neo4j, toàn bộ các quan hệ đều được lưu dưới một relationship type duy nhất là DIRECTED, ngữ nghĩa cụ thể được mã hóa trong thuộc tính keywords do mô hình ngôn ngữ lớn sinh ra trong quá trình trích xuất.

Mỗi thực thể được lưu dưới dạng một node với hai nhãn: nhãn *workspace* (phân vùng dữ liệu, ví dụ base) và nhãn *entity_type* (phân loại ngữ nghĩa, ví dụ Course, Concept). Bảng 4.13 liệt kê các thuộc tính của node.

Bảng 4.13: Thuộc tính của node thực thể trong Knowledge Graph (Neo4j)

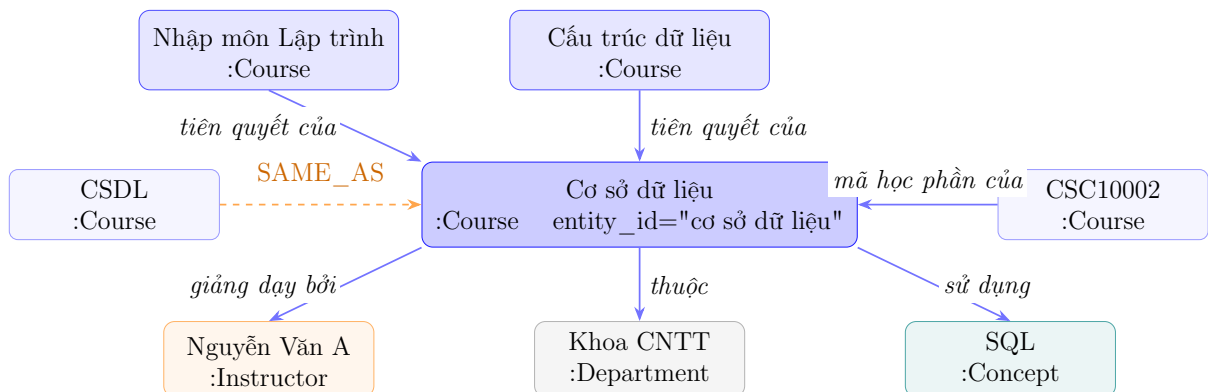
Thuộc tính	Kiểu	Mô tả & Ví dụ
entity_id	string	Tên thực thể (chuẩn hóa chữ thường). VD: "cơ sở dữ liệu"
entity_type	string	Loại thực thể do LLM phân loại. VD: "Course"
description	string	Mô tả tổng hợp từ nhiều tài liệu, các đoạn nối bằng <SEP>. VD: "Môn học về quản lý dữ liệu<SEP>Nghiên cứu SQL"
source_id	string	Danh sách ID chunk nguồn, nối bằng <SEP>. VD: "chunk-abc123<SEP>chunk-def456"
file_path	string	Tài liệu gốc chứa thực thể, nhiều giá trị nối bằng <SEP>. VD: "chương_trình.pdf<SEP>decuong.pdf"
created_at	integer	Unix timestamp thời điểm tạo. VD: 1719571200
truncate	string	Trạng thái giới hạn source_id: "" hoặc "KEEP Old"

Trường hợp đặc biệt là quan hệ đồng nhất thực thể, được biểu diễn bằng cùng type DIRECTED nhưng với keywords = "SAME_AS". Bảng 4.14 mô tả các thuộc tính của relationship.

Bảng 4.14: Thuộc tính của relationship trong Knowledge Graph (Neo4j)

Thuộc tính	Kiểu	Mô tả & Ví dụ
keywords	string	Từ khóa mô tả bản chất quan hệ, do LLM sinh ra. VD: "tiên quyết, học trước" hoặc "SAME_AS"
description	string	Diễn giải quan hệ theo ngữ cảnh tài liệu, nhiều mô tả nối bằng <SEP>. VD: "Nhập môn Lập trình là môn tiên quyết của Cơ sở dữ liệu"
weight	float	Trọng số quan hệ, tỉ lệ với số lần quan hệ xuất hiện. VD: 3.0
source_id	string	ID các chunk nguồn hỗ trợ quan hệ này. VD: "chunk-abc123"
file_path	string	Tài liệu gốc của quan hệ. VD: "chuong_trinh.pdf"
created_at	integer	Unix timestamp thời điểm tạo. VD: 1719571200

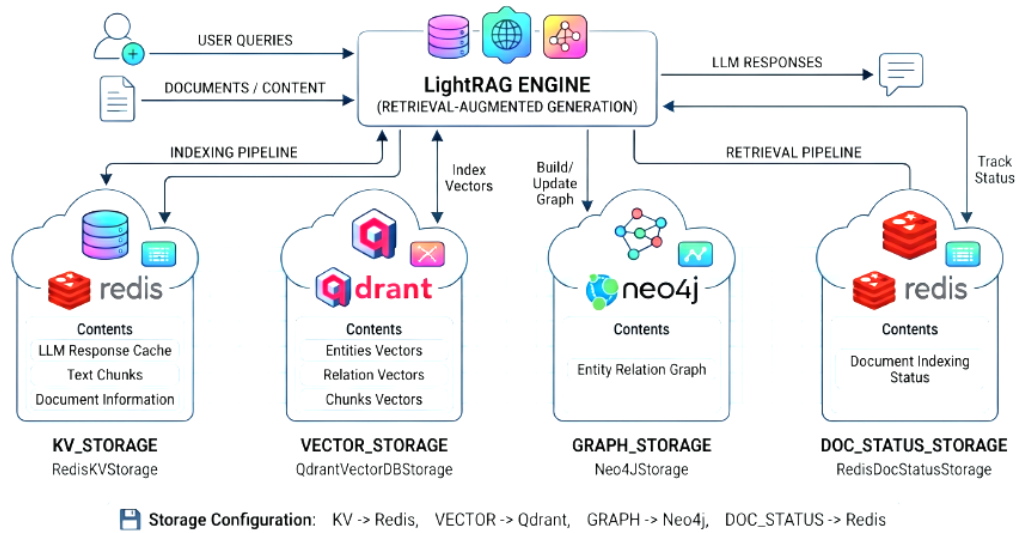
Hình 4.5 minh họa một đoạn đồ thị tri thức được trích xuất từ đề cương môn học, trong đó Cơ sở dữ liệu là thực thể canonical, CSDL và CSC1002 là alias liên kết qua quan hệ SAME_AS, các mũi tên thể hiện quan hệ ngữ nghĩa với nhãn keywords tương ứng.



Hình 4.5: Ví dụ subgraph trích xuất từ đề cương môn học trong Knowledge Graph

4.6.5.3 Kiến trúc triển khai lưu trữ cho LightRAG

Để vận hành hiệu quả các quy trình lập chỉ mục và truy xuất tri thức, hệ thống LightRAG được triển khai với kiến trúc lưu trữ hỗn hợp gồm bốn thành phần chuyên biệt. Sơ đồ tổng quan được thể hiện tại Hình 4.6.



Hình 4.6: Tổng quan kiến trúc lưu trữ và triển khai LightRAG

Thành phần thứ nhất là **KV_STORAGE**, triển khai trên Redis, chịu trách nhiệm lưu trữ bộ nhớ đệm phản hồi LLM, các đoạn văn bản đã phân mảnh và siêu dữ liệu tài liệu gốc. Thành phần thứ hai là **VECTOR_STORAGE**, triển khai trên Qdrant, lưu trữ vector nhúng của thực thể, quan hệ và đoạn văn bản để phục vụ tìm kiếm ngữ nghĩa. Thành phần thứ ba là **GRAPH_STORAGE**, triển khai trên Neo4j, lưu toàn bộ cấu trúc đồ thị tri thức với ngôn ngữ truy vấn Cypher. Thành phần thứ tư là **DOC_STATUS_STORAGE**, triển khai trên Redis, theo dõi trạng thái lập chỉ mục từng tài liệu để tránh xử lý trùng lặp.

Bảng 4.15: Các thành phần lưu trữ trong kiến trúc LightRAG

Thành phần	Công nghệ	Lớp triển khai	Dữ liệu lưu trữ
KV_STORAGE	Redis	RedisKVStorage	LLM Response Cache, Text Chunks, Document Information
VECTOR_STORAGE	Qdrant	QdrantVectorDBStorage	Entities Vectors, Relation Vectors, Chunks Vectors
GRAPH_STORAGE	Neo4j	Neo4JStorage	Entity Relation Graph
DOC_STATUS_STORAGE	Redis	RedisDocStatusStorage	Document Indexing Status

4.6.6 Thiết kế giao diện

4.6.6.1 Màn hình Đăng ký

Hình 4.7: Giao diện Màn hình Đăng ký

Mục đích: Cung cấp biểu mẫu cho phép người dùng mới tạo tài khoản cá nhân để đăng nhập và sử dụng hệ thống LearningPath.

Bao gồm:

1. Các trường nhập dữ liệu: Họ và tên, Mã sinh viên, Email, Mật khẩu, Xác nhận mật khẩu.
2. Nút chức năng "Tạo tài khoản".
3. Liên kết chuyển hướng "Đăng nhập" (dành cho người dùng đã sở hữu tài khoản).

Cách sử dụng:

- Để đăng ký tài khoản mới:

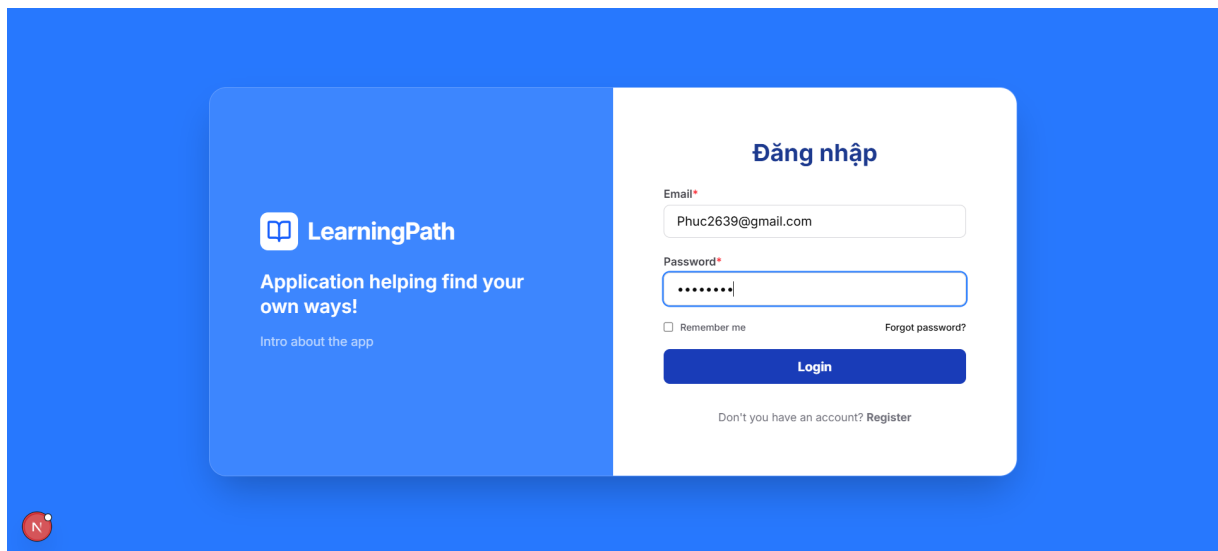
1. Người dùng điền đầy đủ và chính xác các thông tin cá nhân vào từng ô nhập liệu tương ứng.

2. Nhấn nút "Tạo tài khoản".
3. Hệ thống tiến hành kiểm tra tính hợp lệ của dữ liệu đầu vào, nếu hợp lệ sẽ khởi tạo tài khoản mới thành công trên hệ thống và chuyển hướng đến trang đăng nhập.

- **Để quay lại trang Đăng nhập:**

1. Nhấn chọn liên kết "Đăng nhập" ở phần dưới cùng của biểu mẫu để chuyển đổi giao diện nhanh chóng.

4.6.6.2 Màn hình Đăng nhập



Hình 4.8: Giao diện Màn hình Đăng nhập

Mục đích: Xác thực danh tính của người dùng nhằm cấp quyền truy cập vào các tính năng chuyên sâu bên trong hệ thống.

Bao gồm:

1. Các trường nhập liệu: Email và Password (Mật khẩu).
2. Hộp kiểm "Remember me" (Ghi nhớ phiên đăng nhập) và liên kết "Forgot password?" (Quên mật khẩu).

3. Nút chức năng "Login" (Đăng nhập).
4. Liên kết chuyển hướng "Register" (Đăng ký tài khoản mới).

Cách sử dụng:

• **Để truy cập vào hệ thống:**

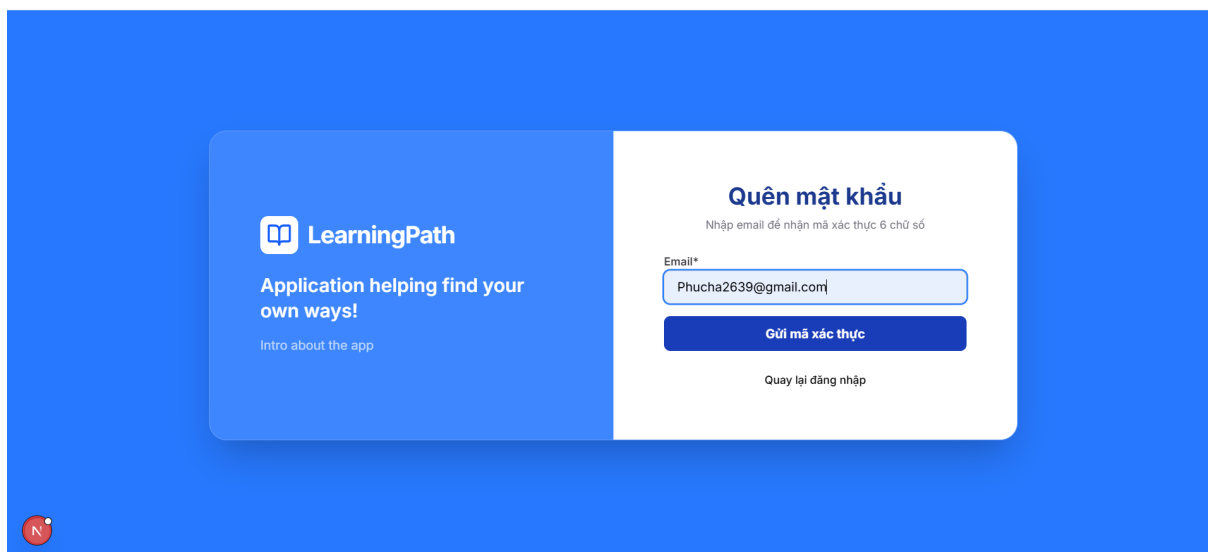
1. Người dùng điền chính xác địa chỉ Email và Mật khẩu đã được đăng ký trước đó.
2. Tích chọn ô "Remember me" nếu muốn duy trì trạng thái đăng nhập cho những lần truy cập sau (tùy chọn).
3. Nhấn nút "Login" để hệ thống đối chiếu xác thực và chuyển hướng vào giao diện chính.

• **Để xử lý khi gặp sự cố mật khẩu hoặc chưa có tài khoản:**

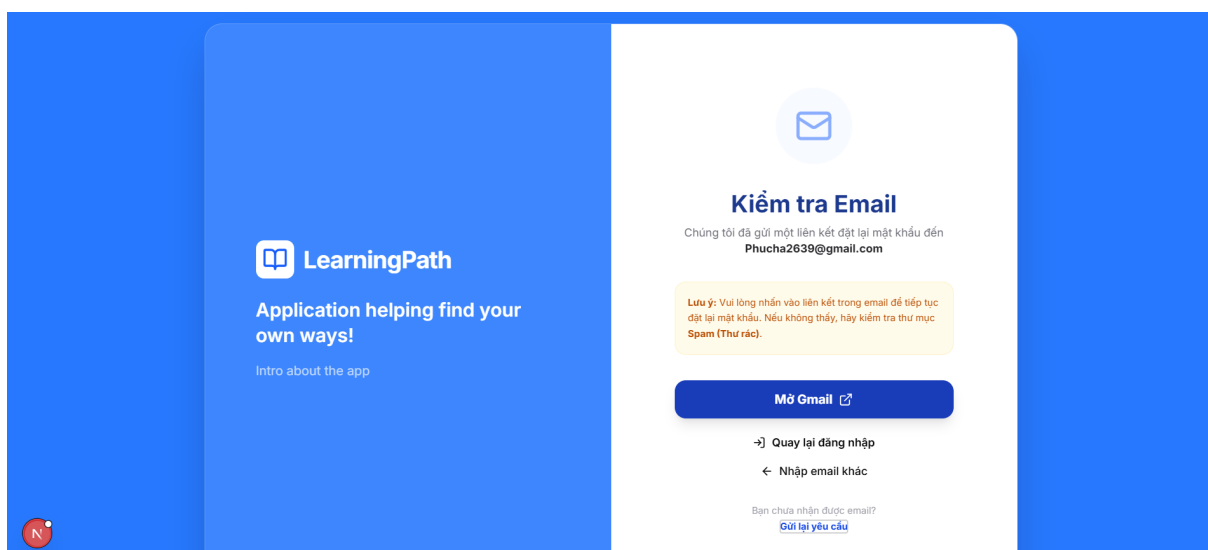
1. Nếu không nhớ mật khẩu, nhấn vào liên kết "Forgot password?" để thực hiện quy trình khôi phục.
2. Nếu chưa có tài khoản, nhấn vào liên kết "Register" để được chuyển hướng đến biểu mẫu đăng ký.

4.6.6.3 Màn hình Quên mật khẩu

Mục đích: Hỗ trợ người dùng cấp lại mật khẩu mới.



Hình 4.9: Giao diện Màn hình Quên mật khẩu - Yêu cầu mã xác thực



Hình 4.10: Giao diện Màn hình Quên mật khẩu - Xác nhận từ Email

Bao gồm:

1. Trường nhập Email để nhận mã khôi phục (Hình 4.9).
2. Nút "Gửi mã xác thực" gửi yêu cầu đến hệ thống (Hình 4.9).
3. Khung hộp thoại thông báo hướng dẫn và kiểm tra thư điện tử (Hình 4.10).

4. Nút "Mở Gmail" (Hình 4.10).
5. Nhóm liên kết hỗ trợ điều hướng bao gồm: "Quay lại đăng nhập", "Nhập email khác" và "Gửi lại yêu cầu" (Hình 4.10).

Cách sử dụng:

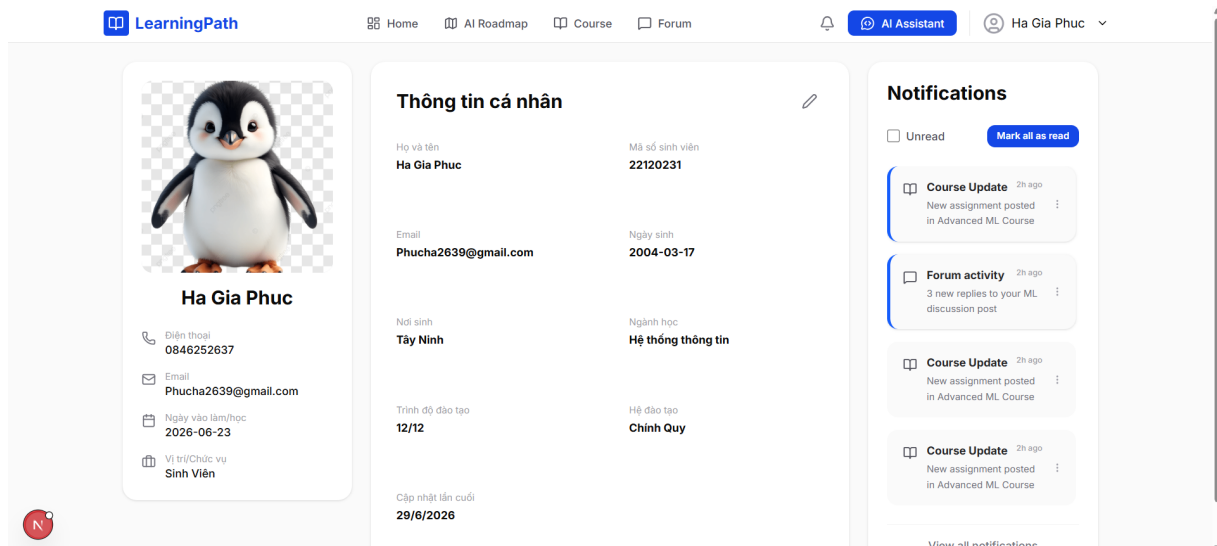
- **Bước 1: Gửi yêu cầu khôi phục mật khẩu từ hệ thống**

1. Người dùng nhập chính xác địa chỉ Email đã dùng để đăng ký tài khoản vào ô dữ liệu.
2. Nhấn nút "Gửi mã xác thực". Hệ thống ghi nhận, gửi mã xác thực đến email của người dùng và chuyển sang giao diện bước 2.

- **Bước 2: Thực hiện kiểm tra hộp thư và tiến hành thiết lập mật khẩu mới**

1. Người dùng nhấn nút "Mở Gmail" để truy cập nhanh kiểm tra hòm thư cá nhân.
2. *Lưu ý bắt buộc:* Kiểm tra kỹ toàn bộ hộp thư chính cũng như thư mục Spam (Thư rác) trong trường hợp bộ lọc của hòm thư giữ lại mã xác thực.
3. Nhấn trực tiếp vào đường dẫn liên kết và thực hiện thay đổi mật khẩu mới.

4.6.6.4 Màn hình Thông tin cá nhân



Hình 4.11: Giao diện Màn hình Thông tin cá nhân

Mục đích: Hiển thị hồ sơ chi tiết của người dùng, và cập nhật thông tin cá nhân.

Bao gồm:

1. **Thông tin tóm tắt:** Hiển thị ảnh đại diện, họ và tên, số điện thoại, email liên hệ, ngày vào học và vai trò người dùng.
2. **Thông tin cá nhân:** Hiển thị chi tiết các dữ liệu học vụ và thông tin cá nhân.

Cách sử dụng:

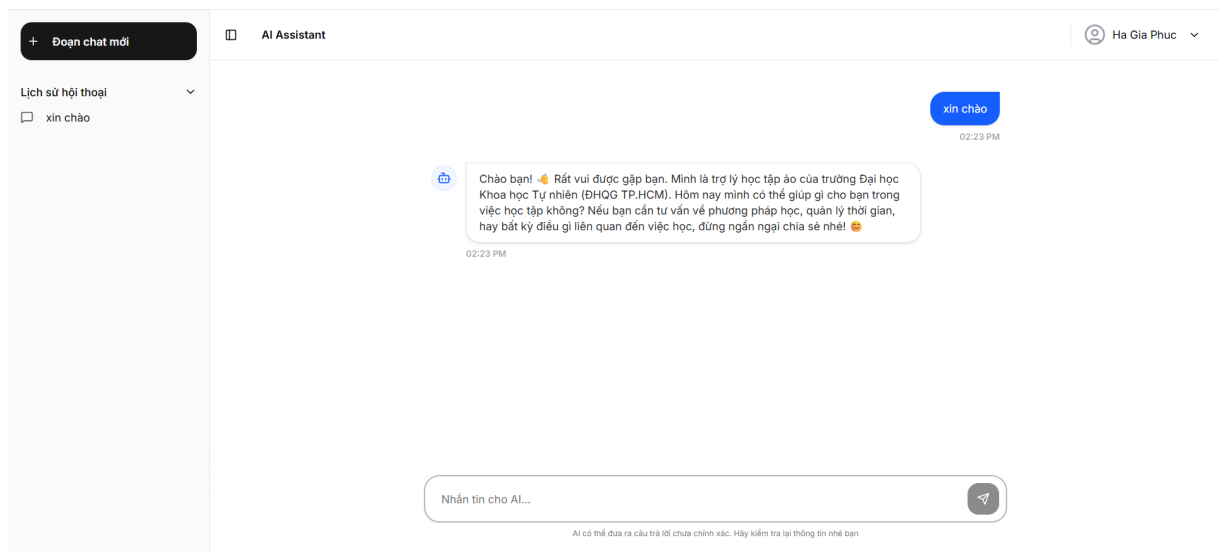
- **Để xem thông tin người dùng:**

1. Truy cập vào trang cá nhân ở góc màn hình bên phải.
2. Hệ thống sẽ tự động truy xuất và hiển thị toàn bộ hồ sơ hiện tại của người dùng lên giao diện.

- **Để cập nhật thông tin:**

1. Tại khu vực "Thông tin cá nhân", người dùng nhấn vào biểu tượng chỉnh sửa ở góc trên bên phải.
2. Hệ thống sẽ mở ra các trường dữ liệu cho phép người dùng thay đổi hoặc bổ sung thông tin.
3. Sau khi nhập liệu xong, người dùng lưu lại để hệ thống cập nhật vào cơ sở dữ liệu.

4.6.6.5 Màn hình AI Chat



Hình 4.12: Giao diện Màn hình AI Chat

Mục đích: Cung cấp không gian trò chuyện trực tiếp giữa sinh viên và AI Agent để được tư vấn thông tin học vụ, lộ trình môn học và quy chế đào tạo.

Bao gồm:

1. Thanh lịch sử hội thoại (bên trái)
2. Khu vực hiển thị hội thoại (trung tâm)
3. Ô nhập câu hỏi và nút gửi (phía dưới)

Cách sử dụng:

- **Để đặt câu hỏi cho AI Agent:**

1. Nhập câu hỏi vào ô nhập liệu ở phía dưới màn hình.
2. Nhấn nút gửi.
3. Hệ thống phân tích câu hỏi, truy xuất đồ thị tri thức và trả về câu trả lời chi tiết trong khu vực hội thoại.

- **Để xem lại lịch sử hội thoại:**

1. Chọn phiên hội thoại tại thanh danh sách bên trái.
2. Hệ thống hiển thị toàn bộ nội dung của phiên đó.

4.6.7 Hiện thực phân hệ Ứng dụng di động (Mobile Application)

Phân hệ di động được phát triển độc lập dựa trên framework Flutter, sử dụng ngôn ngữ lập trình Dart với cơ chế biên dịch trực tiếp sang mã máy native. Kiến trúc mã nguồn được tổ chức theo mô hình quản lý trạng thái tập trung thông qua bộ thư viện *Provider* (gồm *MultiProvider* và *ChangeNotifierProvider*), giúp tách biệt hoàn toàn tầng giao diện (UI) và tầng xử lý logic nghiệp vụ, đồng thời duy trì một luồng dữ liệu đồng nhất trên toàn ứng dụng. Phân hệ di động loại bỏ hoàn toàn các tính năng mạng xã hội/diễn đàn, tập trung chuyên biệt vào việc hỗ trợ thông tin học vụ và cá nhân hóa lộ trình học tập của sinh viên thông qua các phân hệ chức năng thực tế sau:

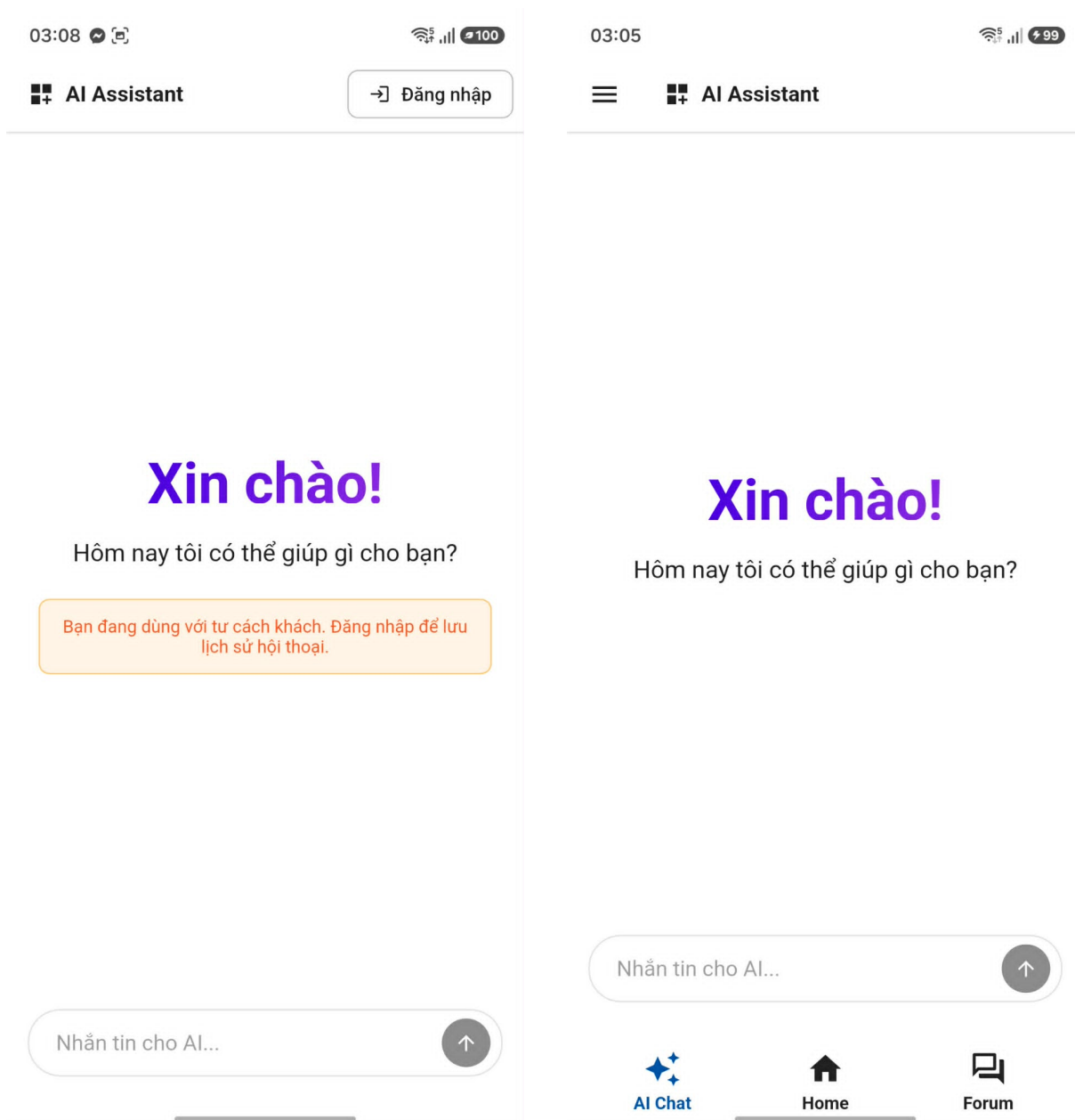
4.6.7.1 Phân hệ Trợ lý ảo AI thông minh và Quản lý ngữ cảnh hội thoại

Đây là tính năng trung tâm của ứng dụng di động, được cấu hình tự động điều phối hiển thị ngay tại màn hình khởi động thông qua lớp

SplashScreen. Giao diện tương tác trò chuyện trực tiếp được thiết kế linh hoạt, tự động phân tách theo trạng thái xác thực của người dùng:

- Chế độ truy cập ẩn danh (Guest Mode): Dành cho người dùng chưa thực hiện đăng nhập vào hệ thống. Khi sinh viên khởi động ứng dụng, luồng điều hướng sẽ đưa thẳng vào màn hình **ChatbotScreen** ở trạng thái khách. Tại đây, sinh viên có thể tương tác trực tiếp với AI để tra cứu thông tin học vụ đại chúng mà không có thanh trình đơn trượt bên trái (Sidebar) và hệ thống tuân thủ nghiêm ngặt cơ chế bảo mật đóng, hoàn toàn không lưu lại lịch sử trò chuyện cục bộ.

- Chế độ thành viên đã xác thực (Authenticated Mode): Trạng thái này tự động được kích hoạt ngay sau khi sinh viên đăng nhập thành công vào hệ thống. Lúc này, màn hình AI Chat sẽ trở thành Tab đầu tiên nằm dưới thanh điều hướng chính (**BottomNavigationBar** của **MainNavScreen**). Hệ thống kích hoạt một thanh Sidebar dạng **Drawer** trượt từ cạnh trái màn hình, hiển thị trực quan toàn bộ danh sách lịch sử các phiên trò chuyện cũ (**ChatSession**). Toàn bộ các phiên trò chuyện này được lưu trữ bền vững tại bộ nhớ đệm thiết bị thông qua gói **SharedPreferences**, tự động sắp xếp theo mốc thời gian tương tác gần nhất và phân tách ranh giới lịch sử theo từng mã định danh tài khoản riêng biệt (**student_id**).



a) Trợ lý AI ở Chế độ Khách (Không lưu lịch sử)

b) Trợ lý AI ở Chế độ Thành viên (Có lịch sử chat)

Hình 4.13: Giao diện tương tác phân hệ Trợ lý ảo AI trên ứng dụng di động

Phân hệ tư vấn thông minh di động tích hợp hai giải pháp tối ưu hóa trải nghiệm hội thoại nâng cao:

- Cơ chế làm giàu truy vấn theo chuỗi hội thoại (Context Concatenation): Để khắc phục hiện tượng mất ngữ cảnh khi sinh viên đặt câu hỏi

quá ngắn hoặc mơ hồ ở các lượt chat sau, ứng dụng tự động kiểm tra mảng dữ liệu tin nhắn. Nếu chuỗi hội thoại đạt từ 3 lượt tương tác trở lên, hệ thống sẽ tự động trích xuất nội dung câu hỏi liền trước (`lastUser`) và câu trả lời liền trước của bot (`lastAI`) để ghép nối có cấu trúc trực tiếp vào câu hỏi hiện tại trước khi thực hiện đóng gói JSON gửi lên API máy chủ LightRAG.

- Bộ phân tích và biên dịch cú pháp Markdown dạng RichText: Để hiển thị câu trả lời từ mô hình ngôn ngữ lớn một cách chuyên nghiệp, ứng dụng hiện thực hóa hàm quét chuỗi ký tự tự động dựa trên biểu thức chính quy. Hệ thống thực hiện bấm nhỏ văn bản theo ký tự xuống dòng kép, nhận diện ký tự thẻ `#` để phóng to cỡ chữ tiêu đề và tự động chuyển đổi màu sắc sang mã màu xanh chủ đạo của trường (`0xFF0054A6`), đồng thời nhận diện các cặp ký tự `**` để gán nhãn kiểu chữ in đậm `FontWeight.bold` cho các thuật ngữ chuyên ngành hoặc mã học phần quan trọng.

4.6.7.2 Phân hệ Xác thực tài khoản và Bắt lỗi hệ thống bảo vệ (Fail-safe Auth)

Phân hệ quản lý toàn bộ vòng đời xác thực của sinh viên bao gồm Đăng nhập (`LoginScreen`), Đăng ký (`RegisterScreen`) và Đăng xuất thông qua kết nối API RESTful mã hóa bảo mật Bearer JWT Token gửi về từ máy chủ Spring Boot. Hệ thống di động tích hợp bộ lọc tự động kích hoạt vì tiến trình `refreshAccessToken` để làm mới mã truy cập ngầm định khi hết hạn mà không gây gián đoạn trải nghiệm của sinh viên.

Đặc biệt, tại chức năng Quên mật khẩu và Đặt lại mật khẩu, ứng dụng tích hợp một cơ chế xử lý ngoại lệ thông minh (*Fail-safe SMTP Exception Handling*):

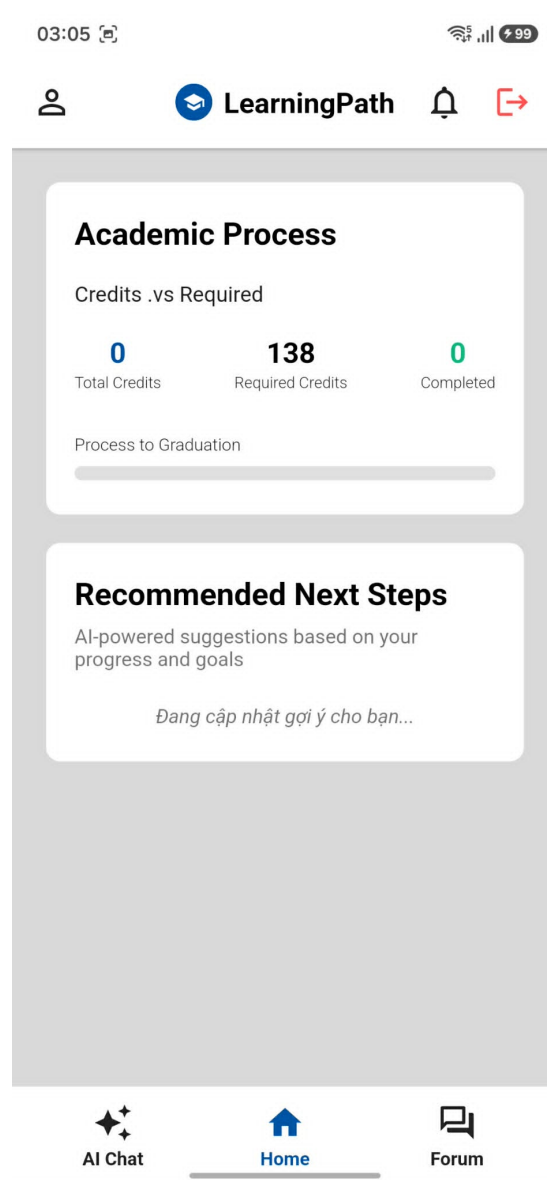
- Khi sinh viên nhấn nút yêu cầu link khôi phục, ứng dụng di động sẽ gọi API `POST /auth/forgot-password`. Trong trường hợp máy chủ đám mây trả về lỗi nghẽn cổng kết nối máy chủ gửi thư (`Mail server connection failed / SocketTimeoutException`), ứng dụng di động sẽ chủ động bắt lấy ngoại lệ này ngay tại tầng kết nối mạng.

- Thay vì vắng lỗi màn hình đỏ, ứng dụng hiển thị một hộp thoại thông báo thông minh thông tin hệ thống đang bảo trì đường truyền gửi thư và tự động kích hoạt mã Token khôi phục giả lập là "DEMO123" để hỗ trợ cho việc Demo báo cáo đồ án. Sinh viên sử dụng mã này điền vào ô xác nhận cùng mật khẩu mới có cấu trúc kiểm tra định dạng Regex nghiêm ngặt (tối thiểu 6 ký tự gồm cả chữ và số) để thực hiện cập nhật cơ sở dữ liệu thành công.

4.6.7.3 Phân hệ Tiến trình học tập, Thông báo và Hồ sơ sinh viên

Phân hệ này chịu trách nhiệm hiển thị thông tin trực quan hóa và đồng bộ hóa trạng thái học vụ của người học, bao gồm các thành phần:

- Thành phần trực quan tiến trình học tập (Academic Progress): Tại màn hình Trang chủ (HomeScreen), ứng dụng thực hiện phân tích số tín chỉ tích lũy hiện hành được thu thập từ trường dữ liệu `totalCreditsEarned` trong API nghiệp vụ. Hệ thống sử dụng widget `LinearProgressIndicator` để dựng đồ thị thanh ngang, tính toán động tỷ lệ phần trăm số môn đã hoàn thành dựa trên cột mốc giới hạn 138 tín chỉ tốt nghiệp bắt buộc của Trường Đại học Khoa học Tự nhiên. Màn hình cũng tự động phân giải danh sách các thẻ gợi ý bước đi tiếp theo cá nhân hóa (`AiRecommendation`) gửi về từ tầng phối hợp AI Agent.



Hình 4.14: Giao diện quản lý Tiến trình tích lũy và Gợi ý học tập cá nhân hóa

- Thành phần Thông báo động (Dynamic Notification Badge): Ứng dụng kết nối lấy danh sách thông báo từ máy chủ, đếm số lượng các phần tử có trường trạng thái `isRead == false`. Số lượng thông báo chưa đọc này được hiển thị động thông qua widget `Stack` dưới dạng một hình tròn đỏ (Badge) nằm đè lên icon thông báo trên thanh `CustomAppBar`, hỗ trợ nút bấm gọi nhanh hàm `markAllNotificationsAsRead` bất đồng bộ để xóa toàn bộ badge thông báo.

- Thành phần Hồ sơ đa cấu trúc và đồng bộ dữ liệu đa phần (Multipart Profile): Hiển thị toàn bộ thông tin cá nhân và kết quả điểm số GPA của sinh viên dưới dạng bố cục lưới các thuộc tính rõ ràng. Khi sinh viên kích hoạt chế độ chỉnh sửa (`_isEditing = true`), ứng dụng hiển thị các ô nhập liệu chuyên biệt, tích hợp hộp thoại chọn ngày dạng lịch (`showDatePicker`) cho các trường dữ liệu thời gian. Sinh viên thay đổi ảnh đại diện thông qua gói thư viện `ImagePicker`, hệ thống tự động đóng gói tệp tin ảnh từ bộ nhớ máy thành luồng dữ liệu nhị phân đa phần (`MultipartFile`) và thực hiện lệnh `http.MultipartRequest` dạng PUT để truyền tải trực tiếp và cập nhật lên máy chủ lưu trữ.

03:05 100

←

Personal Info



Nguyễn Trường Tân

Basic Info

 Phone Chưa cập nhật	 Email tan222799@gmail.com
 Birthday 2000-01-01	 Place of Birth Chưa cập nhật
 Student Code 47047099	 Current Position Sinh viên

Academic Info

 Course Chưa cập nhật	 Training Level Đại học
---	---

Hình 4.15: Giao diện Hồ sơ sinh viên và Quản lý thông tin học vụ cá nhân

Chương 5

Kết luận

5.1 Kết quả đạt được

Đề tài đã nghiên cứu và xây dựng thành công "Hệ thống hỗ trợ thông tin học tập cho sinh viên sử dụng AI Agents". Bằng việc kết hợp sức mạnh của Đồ thị tri thức [11] và kiến trúc điều phối Đa tác nhân [15], hệ thống đã góp phần giải quyết bài toán phân mảnh thông tin trong giáo dục đại học [28].

Những đóng góp chính của đề tài bao gồm ba khía cạnh quan trọng. Thứ nhất, nghiên cứu đã số hóa và cấu trúc hóa thành công kho dữ liệu học vụ phức tạp thành Đồ thị tri thức (LightRAG) có khả năng suy luận nhanh chóng [7]. Thứ hai, hệ thống đã xây dựng một luồng xử lý Đa tác nhân thông minh, cho phép AI tự động chọn lọc công cụ tìm kiếm phù hợp với từng dạng câu hỏi của sinh viên. Cuối cùng, dự án triển khai hệ thống đánh giá tự động thông minh (LLM-as-a-Judge) [29] kết hợp RAGAS [6] giúp đo lường và tinh chỉnh chất lượng câu trả lời một cách khách quan. Nhìn chung, hệ thống đã giúp sinh viên dễ dàng tra cứu, thấu hiểu mối liên hệ giữa các môn học tiên quyết và lập kế hoạch học tập hiệu quả.

5.2 Hạn chế của hệ thống

Dù đạt được những kết quả khả quan, hệ thống vẫn còn tồn tại một số giới hạn về mặt kỹ thuật. Trở ngại lớn nhất hiện nay nằm ở độ trễ xử lý (Latency); do bản chất của việc duy trì hội thoại đa tác nhân [25] và suy luận phức tạp của mô hình LLM, tốc độ phản hồi của hệ thống vẫn còn tương đối chậm, đặc biệt khi có nhiều người dùng truy vấn đồng thời. Kề đến là những hạn chế trong việc số hóa tài liệu tiếng Việt, khi quá trình bóc tách cấu trúc bảng biểu phức tạp từ các file PDF quy chế đào tạo đôi khi vẫn gặp lỗi nhận diện (OCR), dẫn đến việc Đồ thị tri thức bị khuyết dữ liệu ở một vài tiểu mục nhỏ.

5.3 Hướng phát triển trong tương lai

Qua quá trình nghiên cứu và thực hiện đề tài, nhóm đã đạt được các mục tiêu trọng tâm về lý thuyết, giải pháp và thực nghiệm. Về lý thuyết, nhóm xây dựng thành công quy trình đánh giá đồ thị tri thức đa tầng dựa trên các nghiên cứu về hệ thống RAG hiện đại [4], [9], [19]. Về giải pháp, việc đề xuất kiến trúc vùng đệm lưu trữ kết hợp với quy tắc SHACL đã tối ưu hóa cấu trúc đồ thị và đảm bảo tính liên kết chặt chẽ [22]. Thực nghiệm trên toàn bộ kho học liệu của khoa Công nghệ Thông tin đã minh chứng tính ổn định của hệ thống với kết quả vượt trội so với các kiến trúc truyền thống.

Trong thời gian tới, nhóm dự kiến hoàn thiện giao diện cho phép giảng viên trực tiếp tham gia phê duyệt thực thể, hướng tới mục tiêu đạt được độ chính xác tuyệt đối. Đồng thời, việc tự động hóa đồng bộ thuật ngữ Anh-Việt trong đề cương chi tiết sẽ được chú ý phát triển. Cuối cùng, một module gợi ý lộ trình học tập cá nhân hóa dựa trên đồ thị tri thức sư phạm sẽ được tích hợp để hỗ trợ sinh viên trong việc định hướng nghề nghiệp và xây dựng kế hoạch học tập tối ưu. Để khắc phục các hạn chế hiện tại và đưa hệ thống vào ứng dụng thực tiễn ở quy mô toàn trường,

các hướng nghiên cứu tiếp theo sẽ tập trung vào ba nhiệm vụ chính. Trước mắt, nhóm sẽ nâng cấp công cụ xử lý tài liệu thông qua việc nghiên cứu tích hợp các mô hình chuyên biệt để bóc tách chính xác hoàn toàn các bảng biểu môn học từ file PDF. Kế đó, vấn đề tối ưu hóa hiệu năng sẽ được giải quyết bằng việc triển khai kỹ thuật Semantic Caching nhằm lưu trữ lại các câu trả lời cho những câu hỏi thường gặp, giúp giảm đáng kể thời gian phản hồi. Về dài hạn, định hướng quan trọng nhất là mở rộng hệ sinh thái sinh viên bằng cách tích hợp trực tiếp hệ thống AI Agent vào cổng thông tin đào tạo của trường. Điều này sẽ cho phép AI tự động cập nhật lịch học, điểm số và gợi ý đăng ký môn học theo thời gian thực để hỗ trợ giáo dục thông minh toàn diện [13].

Danh mục công trình của tác giả

- Tạp chí ABC
- Tạp chí XYZ

Tài liệu tham khảo

Tiếng Anh

- [4] Dong, X. L. et al., “Knowledge-based trust: Estimating the trustworthiness of web sources,” *Proceedings of the VLDB Endowment*, vol. 8, no. 12, pp. 1638–1649, 2015.
- [5] Edge, D. et al., “From local to global: A graph rag approach to query-focused summarization,” *arXiv preprint arXiv:2404.16130*, 2024.
- [6] Es, S., James, J., Espinosa-Anke, L., and Schockaert, S., “Ragas: Automated evaluation of retrieval augmented generation,” *arXiv preprint arXiv:2309.15217*, 2023.
- [7] Guo, H. et al., “Lightrag: Simple and fast retrieval-augmented generation,” *arXiv preprint arXiv:2410.05779*, 2024.
- [8] Gutiérrez, B. et al., “Hipporag2: Enhancing retrieval-augmented generation with composite knowledge graphs,” in *Advances in Neural Information Processing Systems*, 2025.
- [9] HKUDS. “Lightrag: Institutional memory and personal knowledge management with light-weight retrieval-augmented generation,” Accessed: Apr. 18, 2026. [Online]. Available: <https://github.com/HKUDS/LightRAG>
- [10] Hogan, A. et al., “Knowledge graphs,” *ACM Computing Surveys*, vol. 54, no. 4, pp. 1–37, 2021. [Online]. Available: <https://dl.acm.org/doi/10.1145/3447772>

- [11] Hogan, A. et al., “Knowledge graphs,” *ACM Computing Surveys (CSUR)*, vol. 54, no. 4, pp. 1–37, 2021.
- [12] Ji, Z. et al., “Survey of hallucination in natural language generation,” *ACM Computing Surveys*, vol. 55, no. 12, pp. 1–38, 2023.
- [13] Kasneci, E. et al., “Chatgpt for good? on opportunities and challenges of large language models in education,” *Learning and individual differences*, vol. 103, p. 102 274, 2023.
- [14] Labra Gayo, J. E. et al., “Formalizing and validating wikidata’s property constraints using shacl and sparql,” *ResearchGate*, 2024. [Online]. Available: https://www.researchgate.net/publication/383382328_Formalizing_and_validating_Wikidata's_property_constraints_using_SHACL_and_SPARQL
- [15] LangChain, *Langgraph: Multi-actor workflows with llms*, <https://langchain-ai.github.io/langgraph/>, 2024. Accessed: May 1, 2024.
- [16] Lee, M. et al., “Hypergraphrag: N-ary relation retrieval for complex data,” *arXiv preprint*, 2025.
- [17] Lewis, P. et al., “Retrieval-augmented generation for knowledge-intensive nlp tasks,” *Advances in Neural Information Processing Systems*, vol. 33, pp. 9459–9474, 2020.
- [18] Li, J., Sun, A., Han, J., and Li, C., “A survey on deep learning for named entity recognition,” *IEEE Transactions on Knowledge and Data Engineering*, vol. 34, no. 1, pp. 50–70, 2020. [Online]. Available: <https://ieeexplore.ieee.org/document/9039685>
- [19] Microsoft. “Graphrag: A modular graph-based retrieval-augmented generation system,” Accessed: Apr. 18, 2026. [Online]. Available: <https://github.com/microsoft/graphrag>

- [20] Pham, L., Nguyen, M. N. T., et al., “Vn-mteb: Vietnamese massive text embedding benchmark,” in *Findings of the Association for Computational Linguistics: EACL 2026*, 2026.
- [21] Smith, J. et al., “Og-rag: Ontology-grounded retrieval-augmented generation,” *arXiv preprint*, 2024.
- [22] W3C. “Shapes constraint language (shacl),” Accessed: Apr. 18, 2026. [Online]. Available: <https://www.w3.org/TR/shacl/>
- [23] Wang, L., Yang, N., Huang, X., et al., “Multilingual e5 text embeddings: A technical report,” *arXiv preprint arXiv:2402.05672*, 2024, Microsoft Research.
- [24] Wang, X. et al., “Pathrag: Pruning relational paths for efficient graph-based retrieval,” *arXiv preprint*, 2025.
- [25] Wu, Q. et al., “Autogen: Enabling next-gen llm applications via multi-agent conversation,” *arXiv preprint arXiv:2308.08155*, 2023.
- [26] Xi, Z. et al., “The rise and potential of large language model based agents: A survey,” *arXiv preprint arXiv:2309.07864*, 2023.
- [27] Xplore, I., “Optimal rag system design for textbooks: A comprehensive evaluation and performance enhancement study,” *IEEE Transactions on Learning Technologies*, 2026.
- [28] Zawacki-Richter, O., Marín, V. I., Bond, M., and Gouverneur, F., “Systematic review of research on artificial intelligence applications in higher education—where are the educators?” *International Journal of Educational Technology in Higher Education*, vol. 16, no. 1, pp. 1–27, 2019.
- [29] Zheng, L. et al., “Judging llm-as-a-judge with mt-bench and chatbot arena,” *Advances in Neural Information Processing Systems*, vol. 36, 2023.

Phụ lục A

Ngữ pháp tiếng Việt

Đây là phụ lục.

Phụ lục B

Ngữ pháp tiếng Nôm

Đây là phụ lục 2.